fit@hcmus

MTH00050 – Combinatorial Mathematics

Lecture 7: Tree problems

Lecturer: Bùi Văn Thạch
Lab instructors: Nguyễn Ngọc Toàn, Trần Thị Thảo Nhi, Lê Đức Khoan
{bvthach,nntoan,tttnhi,ldkhoan}@fit.hcmus.edu.vn

(Tentative) Schedule 2026

No.	Date	Location	Topic
1	13/01 16/01	C24 C24/E302	Introduction, Sets, Proofs
2	20/01 23/01	C24 C24/E302	General counting methods
3	27/01 30/01	C24 C24/E302	Inclusion-exclusion principle
4	03/02 06/02	C24 C24/E302	Recurrence relations
5	03/03 06/03	C24 C24/E302	Generating functions
			Mid-term test

No.	Date	Location	Topic
6	10/03 13/03	C24 C24/E302	Graphs (Part I)
7	17/03 20/03	C24 C24/E302	Graphs (Part II)
8	24/03 27/03	C24 C24/E302	Path problems
9	31/03 03/04	C24 C24/E302	Tree problems
10	07/04 10/04	C24 C24/E302	Network flows (Part I)
11	14/04 17/04	C24 C24/E302	Network flows (Part II)
			Final exam

Course topics

0. Introduction
1. Set and counting
2. General counting methods for arrangements and selections
3. Inclusion-exclusion principle
4. Recurrence relations
5. Generating functions
6. Graphs
7. Tree problems
8. Path problems
9. Network flows

Outline

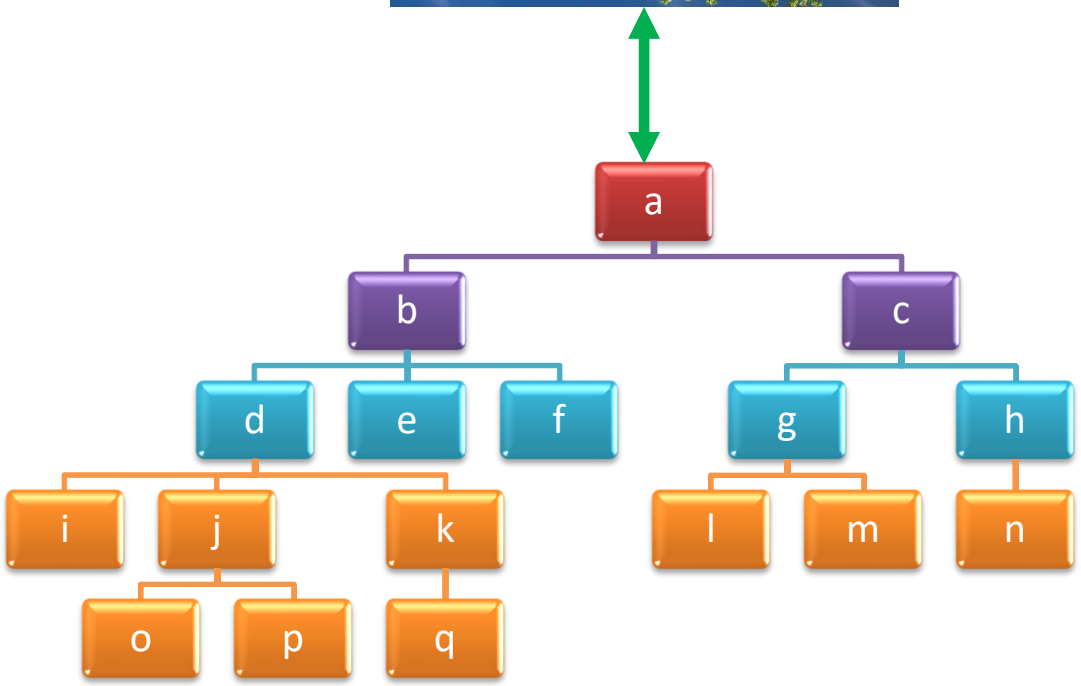
1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

What is tree?

Rotate 180^o



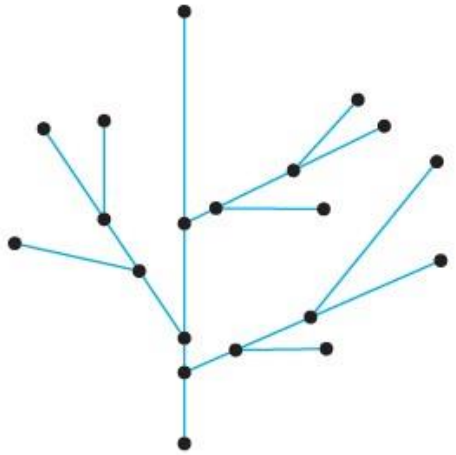
Definition

Definition 1. A graph is said to be circuit-free if, and only if, it has no circuits.

A graph is called a tree if, and only if, it is circuit-free and connected.

A trivial tree is a graph that consists of a single vertex.

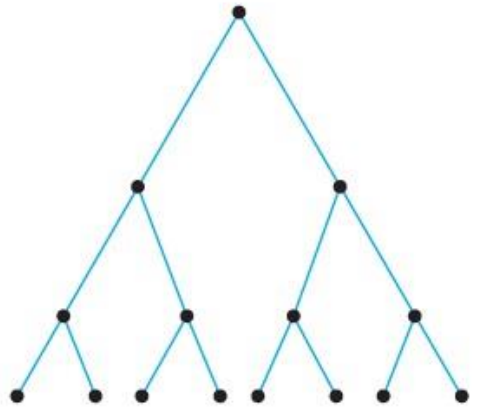
A graph is called a forest if, and only if, it is circuit-free and not connected.



(a)

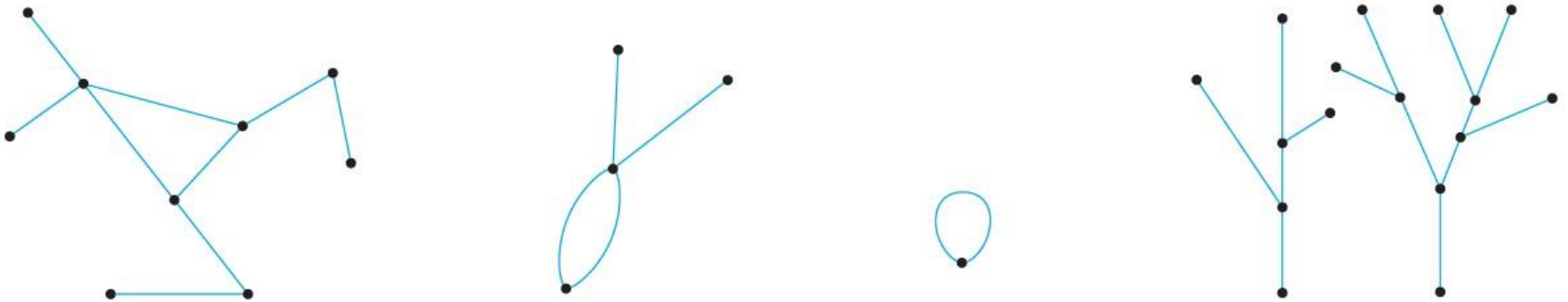
(b)

(c)



(d)

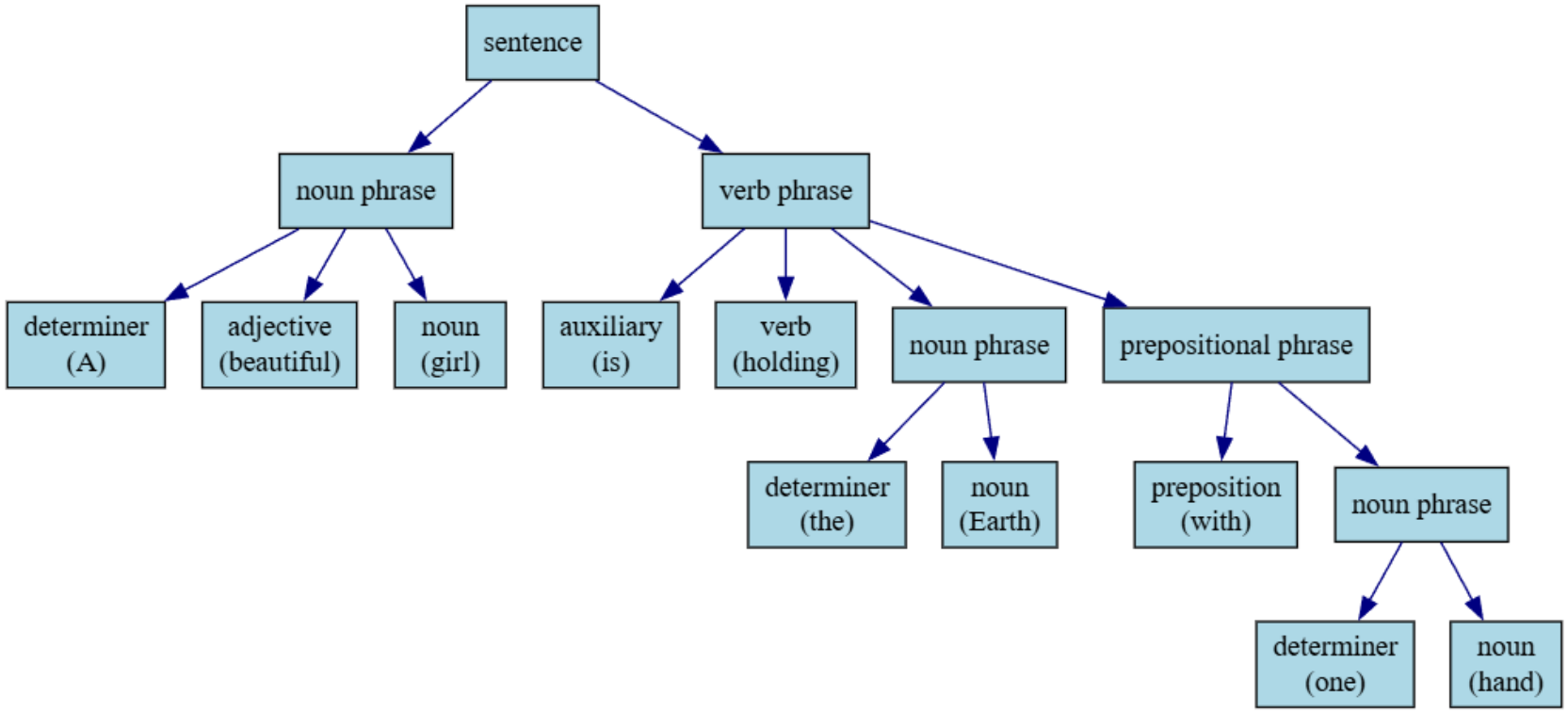
Example: non-trees



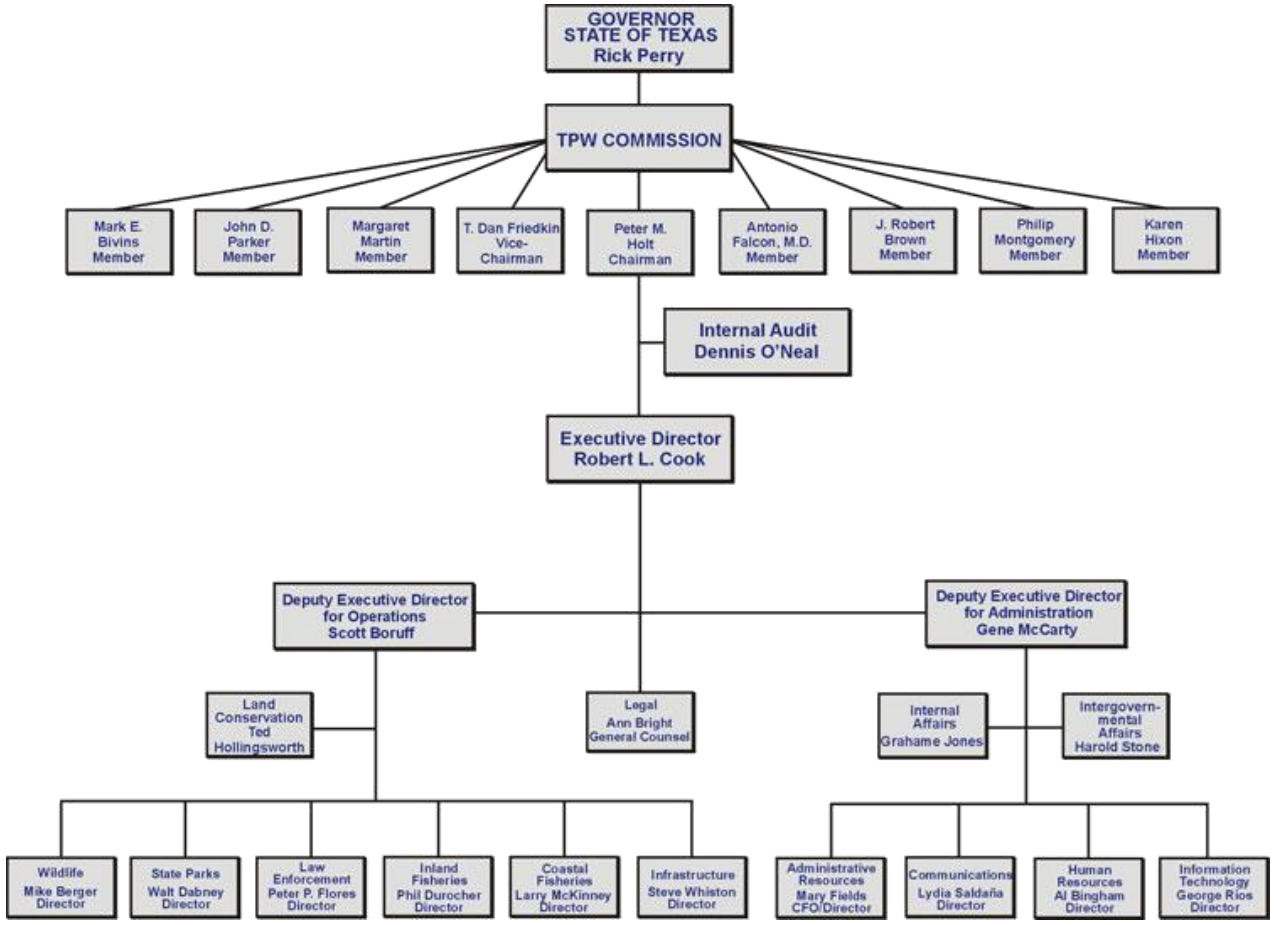
(a) (b) (c) (d)

Example: Parse Tree

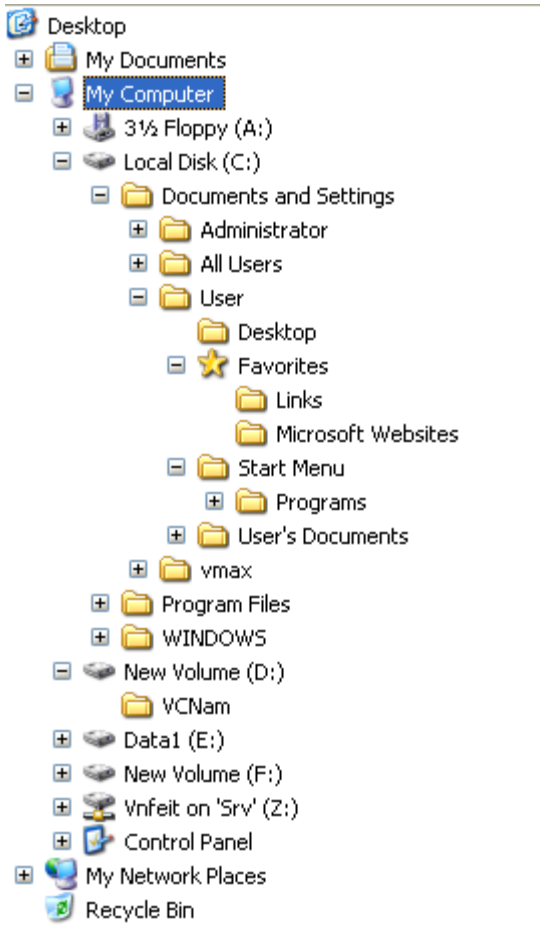
A beautiful girl is holding the Earth with one hand



Example: Organization chart and directory tree

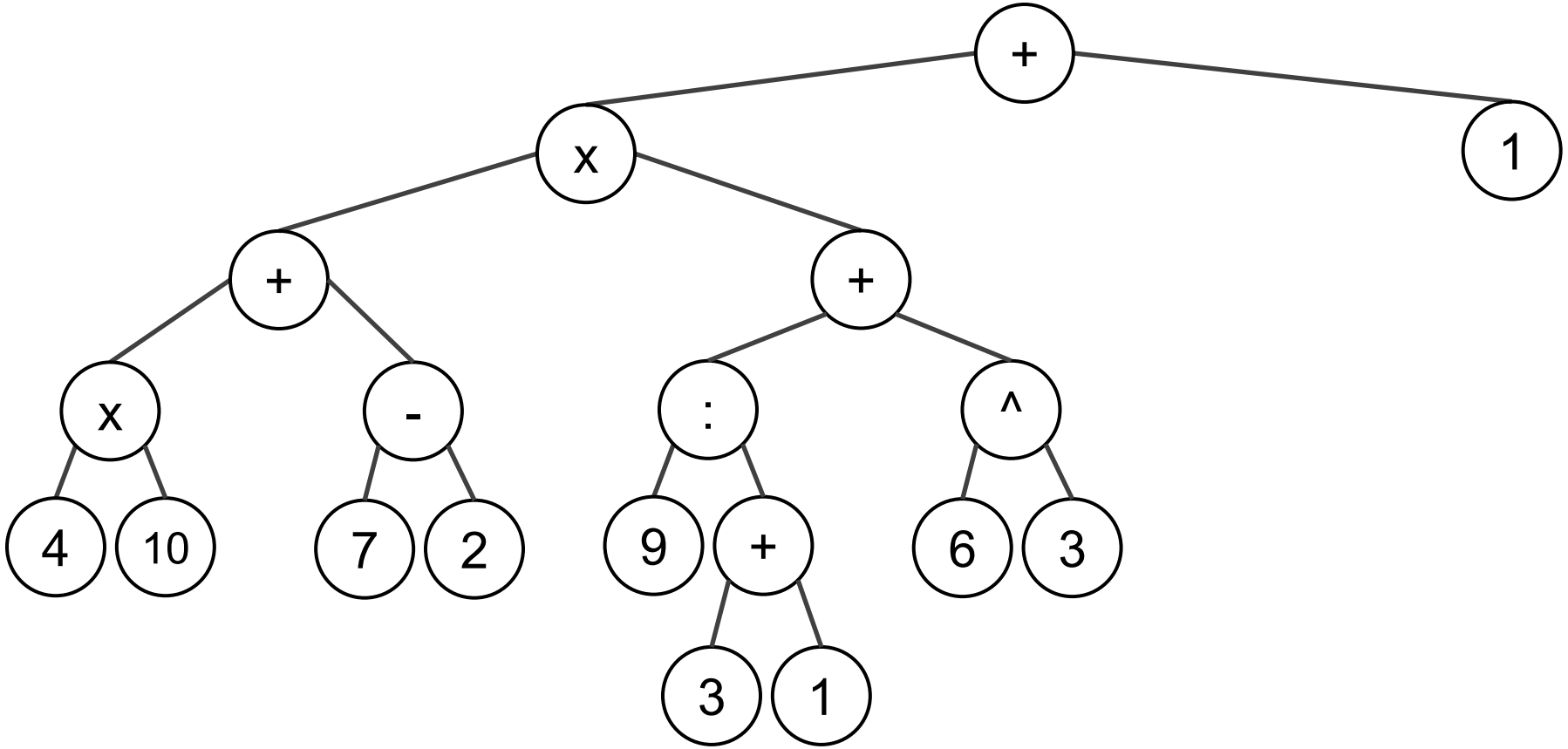


Organizational chart



Directory tree

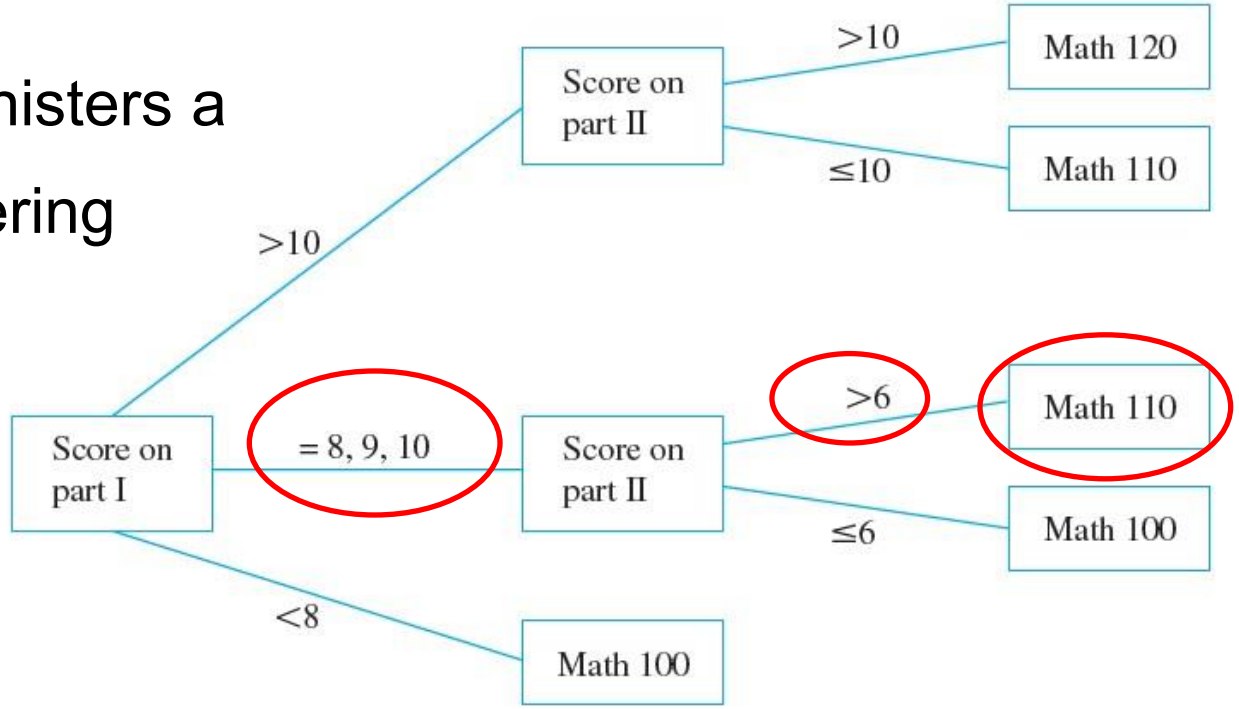
Mathematical expression



$$\left((4 \times 10) + (7 - 2)\right) \times \left(\frac{9}{3+1} + (6^3)\right) + 1$$

Example: a decision tree

- During orientation week, a college administers a mathematics placement exam to all entering students.
- The exam consists of two parts, and placement recommendations are made as indicated by the tree shown beside.
- Read the tree from left to right to decide what course should be recommended for a student who scored 9 on part I and 7 on part II.



Worst-case analysis in Data Structures and Algorithms

	Design	Run time					Space	Collision
		Cost	Search	Insert	Delete	Find_min		
Unsorted array	Det.		$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	
Sorted array by key			$O(\log n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	
Singly Linked list			$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	
Hashing	Rnd.		$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(p)$ $p < n$	YES
Binary Search Tree	Det.		$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	NO
?	Det.		$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	NO

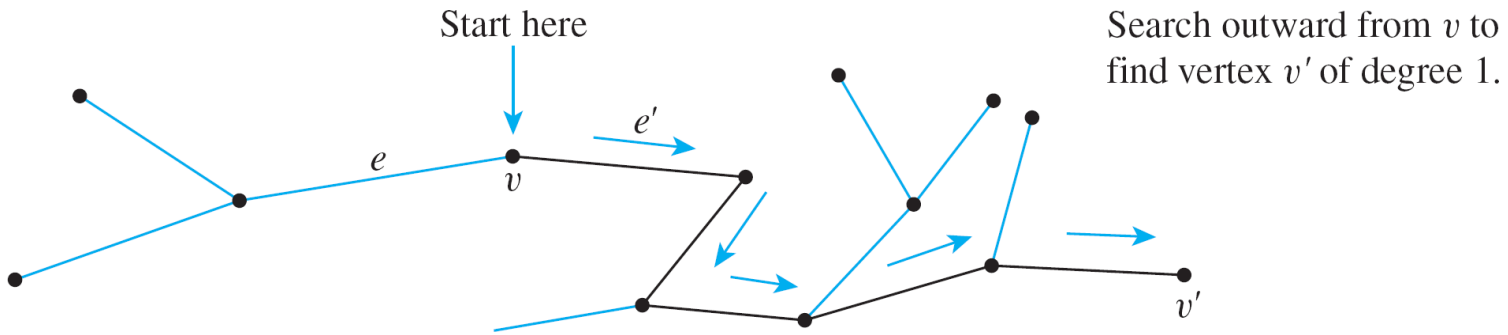
Find a data structure that can ALWAYS achieve these complexities!

Theorem 1. Any non-trivial tree has at least one vertex of degree 1.

- Let T be a particular but arbitrarily chosen tree that has more than one vertex, and consider the following algorithm:
- Step 1: Pick a vertex v of T and let e be an edge incident on v.

[If there were no edge incident on v, then v would be an isolated vertex. But this would contradict the assumption that T is connected (since it is a tree) and has at least two vertices.]

- Step 2: While $\deg(v) > 1$, repeat Steps 2a, 2b, and 2c:
- Step 2a: Choose e' to be an edge incident on v such that $e' \neq e$. [Such an edge exists because $\deg(v) > 1$ and so there are at least two edges incident on v.]
- Step 2b: Let v' be the vertex at the other end of e' from v. [Since T is a tree, e' cannot be a loop and therefore e' has two distinct endpoints.]
- Step 2c: Let $e = e'$ and $v = v'$. [This is just a renaming process in preparation for repeating Step 2.]
- The algorithm just described must eventually terminate because the set of vertices of the tree T is finite and T is circuit-free. When it does, a vertex v of degree 1 will have been found.

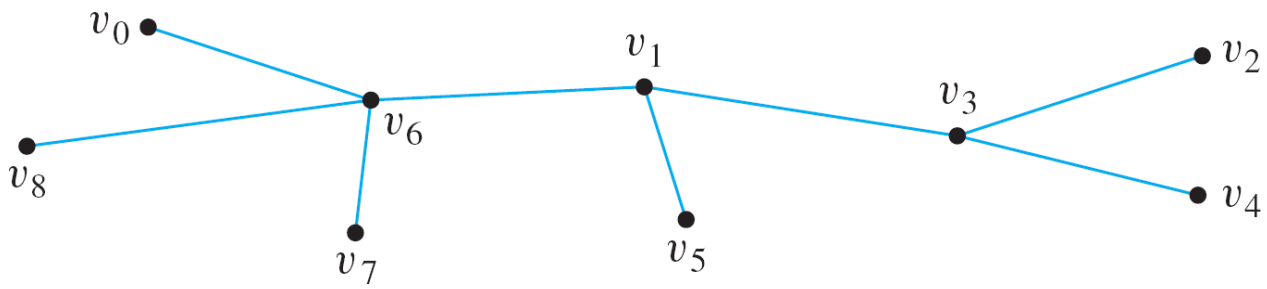


- Using this theorem, it is not difficult to show that, in fact, any tree that has more than one vertex has at least *two* vertices of degree 1.

Leaf and internal vertex

Definition 2. Let T be a tree. If T has at least two vertices, then a vertex of degree 1 in T is called a leaf (or a terminal vertex), and a vertex of degree greater than 1 in T is called an internal vertex (or a branch vertex). The unique vertex in a trivial tree is also called a leaf or terminal vertex.

Example: Find all terminal vertices and all internal vertices in the following tree:



Theorem 2. Any tree with n vertices ($n > 0$) has $n - 1$ edges.

The proof is by mathematical induction.

- Let the property $P(n)$ be the sentence:

  Any tree with n vertices has $n - 1$ edges. ← $P(n)$

- We use mathematical induction to show that this property is true for every integer $n \geq 1$.

- Show that $P(1)$ is true: Let T be any tree with one vertex. Then T has zero edges (since it contains no loops). Since $0 = 1 - 1$, then $P(1)$ is true.

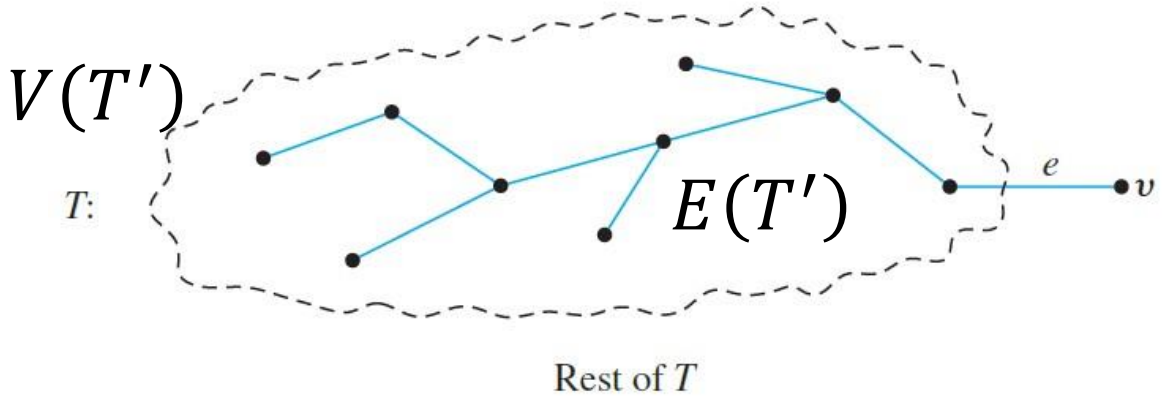
- Show that for every integer $k \geq 1$, if $P(k)$ is true then $P(k+1)$ is true:
- Suppose k is any positive integer for which P(k) is true. In other words, suppose that

  Any tree with k vertices has $k-1$ edges. ← $P(k)$ inductive hypothesis
- We must show that $P(k+1)$ is true. In other words, we must show that

  Any tree with $k+1$ vertices has $(k+1)-1=k$ edges. ← $P(k+1)$
- Let T be a particular but arbitrarily chosen tree with $k+1$ vertices. [We must show that T has k edges.] Since k is a positive integer, $k+1 \geq 2$, and so T has more than one vertex. Hence by Lemma 1.1, T has a vertex v of degree 1. Also, since T has more than one vertex, there is at least one other vertex in T besides v. Thus there is an edge e connecting v to the rest of T.

- Define a subgraph T' of T so that

$V(T') = V(T) - \{v\}$ and $E(T') = E(T) - \{e\}$.



- Then
  1. The number of vertices of T' is $(k+1) - 1 = k$.
  2. T' is circuit-free (since T is circuit-free, and removing an edge and a vertex cannot create a circuit).
  3. T' is connected.
- Hence, by the definition of tree, T' is a tree. Since T' has k vertices, by inductive hypothesis

$$\text{the number of edges of } T' = (\text{the number of vertices of } T') - 1 = k - 1.$$

- It follows that

$$\text{the number of edges of } T = (\text{the number of edges of } T') - 1 = (k+1) - 1 = k.$$

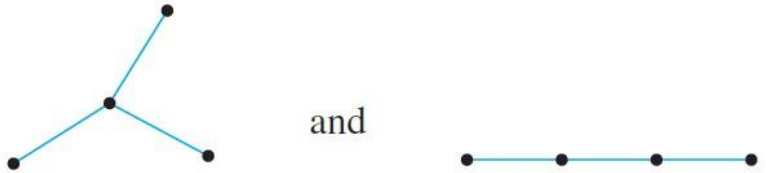
Example

1. A graph G has ten vertices and twelve edges. Is it a tree?
2. Find all non-isomorphic trees with four vertices.

1. No. By Theorem 1., any tree with ten vertices has nine edges, not twelve.
2. By Theorem 10.4.2, any tree with four vertices has three edges. Thus the total degree of a tree with four vertices must be 6. Also, every tree with more than one vertex has at least two vertices of degree 1. Thus the following combinations of degrees for the vertices are the only ones possible:

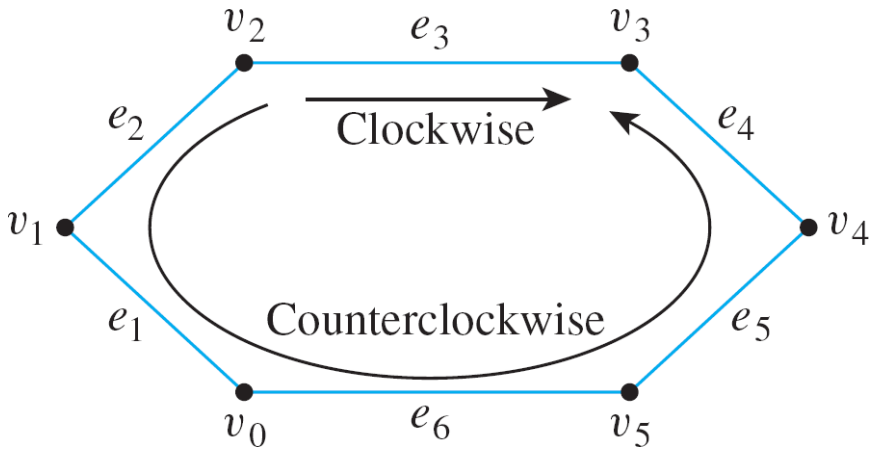
1, 1, 1, 3 and 1, 1, 2, 2.

There are two non-isomorphic trees corresponding to both of these possibilities, as shown below.



Lemma 10.4.3. If G is any connected graph, C is any circuit in G, and any one of the edges of C is removed from G, then the graph that remains is connected.

- Essentially, the reason why Lemma 10.4.3 is true is that any two vertices in a circuit are connected by two distinct paths.
- It is possible to draw the graph so that one of these goes "clockwise" and the other goes "counterclockwise" around the circuit. For example, in the circuit below, the clockwise path from v_2 to v_3 is



$$v_2 e_3 v_3,$$

and the counterclockwise path from v_2 to v_3 is

$$v_2 e_2 v_1 e_1 v_0 e_6 v_5 e_5 v_4 e_4 v_3, \text{ where } v_6 = v_0.$$

Theorem 10.4.4. For any positive integer n, if G is a connected graph with n vertices and $n-1$ edges, then G is a tree.

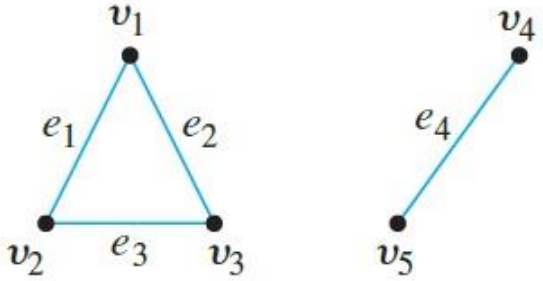
- Let n be a positive integer and suppose G is a particular but arbitrarily chosen graph that is connected and has n vertices and $n-1$ edges.
  [We must show that G is a tree. Now a tree is a connected, circuit-free graph. Since we already know G is connected, it suffices to show that G is circuit-free.]
- Suppose G is not circuit-free. That is, suppose G has a circuit C. [We must derive a contradiction.]
- By Lemma 10.4.3, an edge of C can be removed from G to obtain a graph G' that is connected. If G' has a circuit, then repeat this process: Remove an edge of the circuit from G' to form a new connected graph. Continue repeating the process of removing edges from circuits until eventually a graph G'' is obtained that is connected and is circuit-free. By definition, G'' is a tree.

- Since no vertices were removed from G to form G'', G'' has n vertices just as G does. Thus, by Theorem 10.4.2, G'' has $n - 1$ edges.
- But the supposition that G has a circuit implies that at least one edge of G is removed to form G''.
- Hence G'' has no more than $(n - 1) - 1 = n - 2$ edges.
- This contradicts its having $n - 1$ edges.
- So the supposition is false. Hence G is circuit-free, and therefore G is a tree.

Converse part

- Theorem 10.4.4 is not a full converse of Theorem 10.4.2. Although it is true that every connected graph with n vertices and $n - 1$ edges (where n is a positive integer) is a tree, it is not true that every graph with n vertices and $n - 1$ edges is a tree.
- Give an example of a graph with five vertices and four edges that is not a tree.
- By Theorem 10.4.4, such a graph cannot be connected.

  One example of such an unconnected graph is shown beside.



Corollary 10.4.5. If G is any graph with n vertices and m edges, where m and n are positive integers and $m \geq n$, then G has a circuit.

Proof (by contradiction):

- Suppose not. That is, suppose there is a graph G with n vertices and m edges, where m and n are positive integers and $m \geq n$, and suppose G does not have a circuit.
- Let $G_1, G_2, \dots, G_k$ be the connected components of G, and let $n_1, n_2, \dots, n_k$ be the number of vertices of $G_1, G_2, \dots, G_k$, respectively. Because $G_1, G_2, \dots, G_k$ are the connected components of G,

$$\sum_{i=1}^{k} n_i = n.$$

- Since G does not have a circuit, none of $G_1, G_2, \ldots, G_k$ have circuits either. So, since each is connected, each is a tree. By Theorem 10.4.4, the number of edges of each G_i is n_{i-1}.
- Now because G is composed of its connected components,

$$\text{the number of edges of } G = \sum_{i=1}^{k} \text{the number of edges of } G_i$$

$$= (n_1 - 1) + (n_2 - 1) + \cdots + (n_k - 1) = \left(\sum_{i=1}^{k} n_i\right) - k = n - k < n.$$

- Thus the number of edges of G is less than n, which contradicts the hypothesis that the number of edges of G, namely, m, is greater than or equal to n. Hence the supposition is false and G has a circuit.

Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

Rooted tree, level, height

Definition 2 ([1, p. 122]). A rooted *tree* is a tree in which there is one node, called the root, at the top is distinguished from the others.

The level/depth of a **node** is the number of edges along the unique PATH between it and the root.

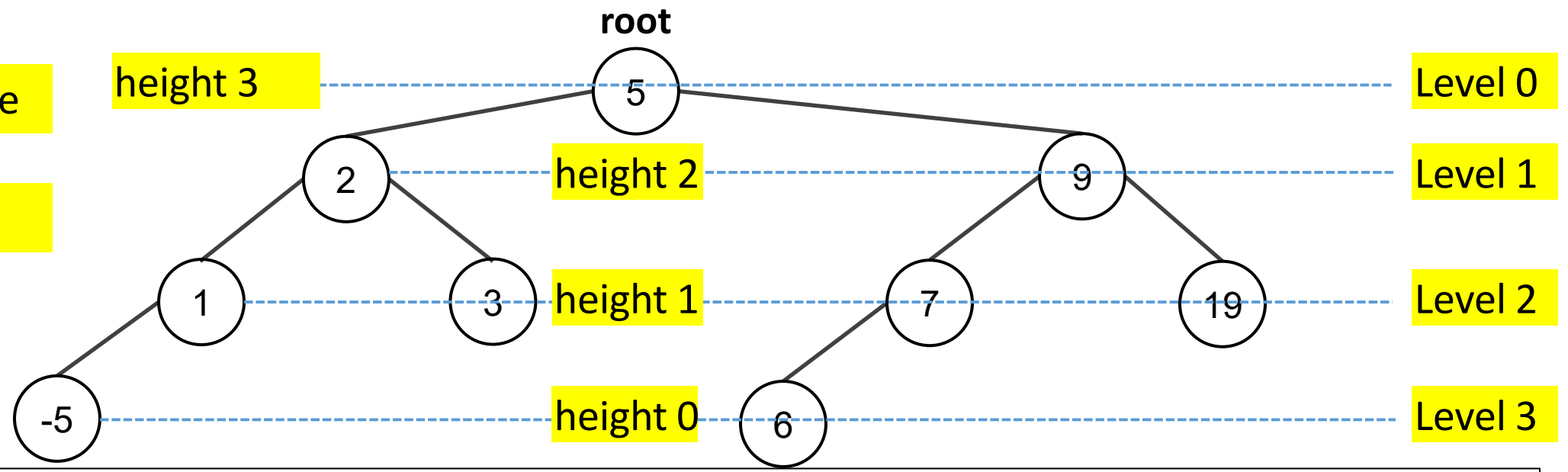
The height of a **node** is the length of the longest path from it to a leaf.

The height of a **rooted tree** is the MAXIMUM level of any node of the tree, i.e. the LONGEST PATH.

Level of a node

height(T) = 3

Empty tree has height -1



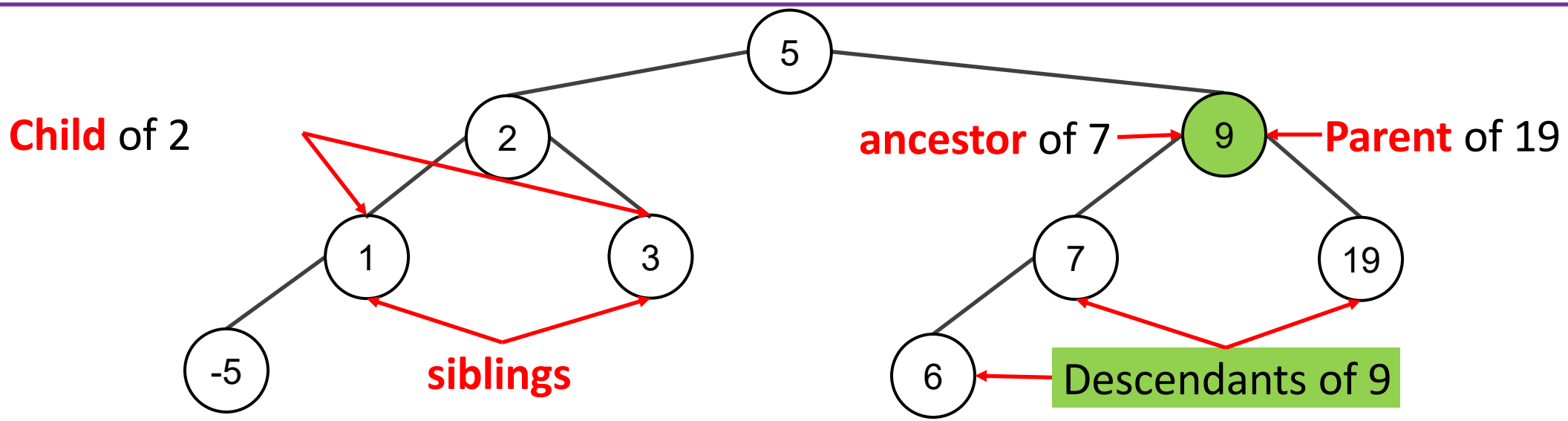
Allen, Weiss Mark. *Data structures and algorithm analysis in C++*. Pearson Education India, 2007.

Parent, Sibling, Ancestor, Descendant

Definition 3. Given the root or any internal vertex v of a rooted tree, the **children** of v are all those vertices that are adjacent to v and are one level farther away from the root than v.

If w is a **child** of v, then v is called the **parent** of w, and **two distinct vertices that are both children of the same parent are called siblings**.

Given two distinct vertices v and w, if v lies on the unique path between w and the root, then v is an **ancestor** of w, and w is a **descendant** of v.



Quiz

Consider the tree with root v_0 shown below.

a. What is the level of v_6?

b. What is the level of v_0?

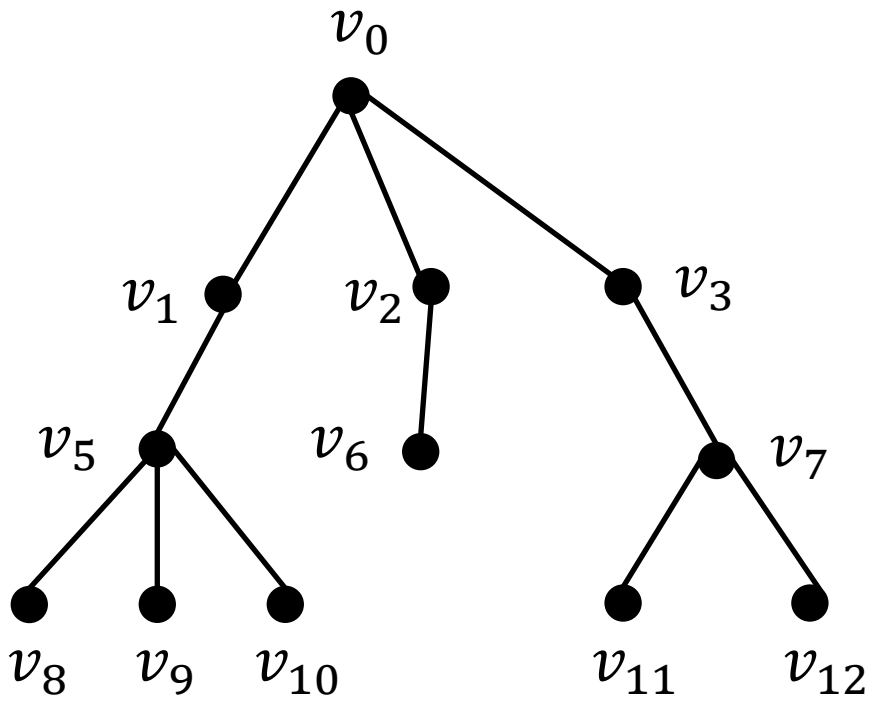
c. What is the height of this rooted tree?

d. What are the children of v_7?

e. What is the parent of v_6?

f. What are the siblings of v_5?

g. What are the descendant of v_1?

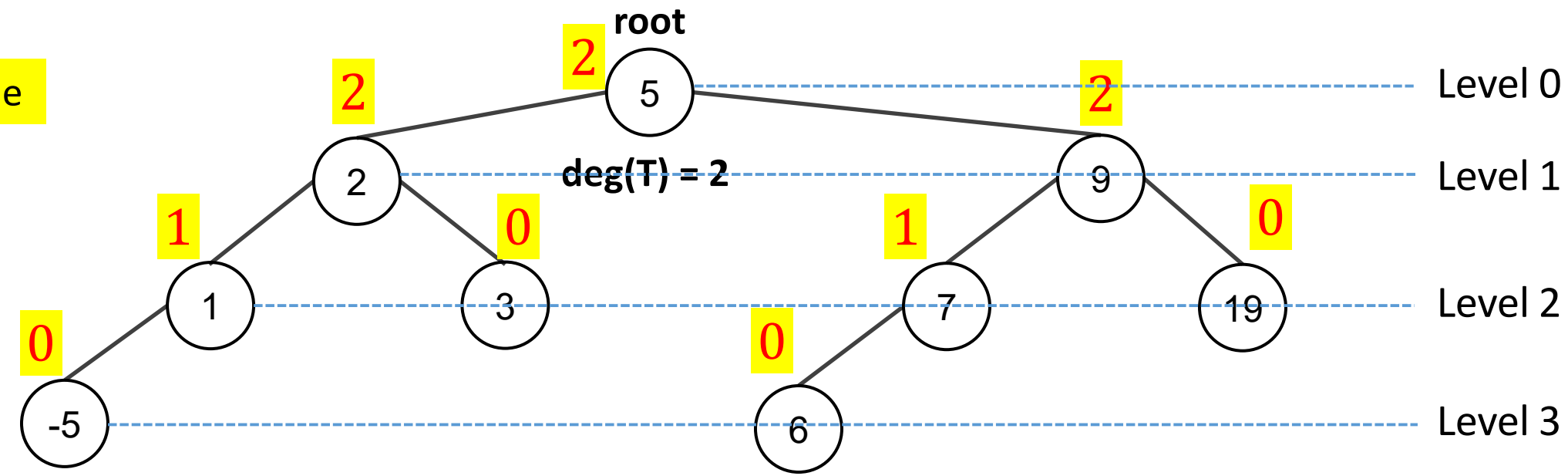


Degree

Definition 4. Degree of a node: the number of subtrees (children) it has.

- Degree of a tree: the largest degree among all nodes in the tree (i.e., the maximum number of children any single node has).
- A tree with degree n is called an n-ary tree.

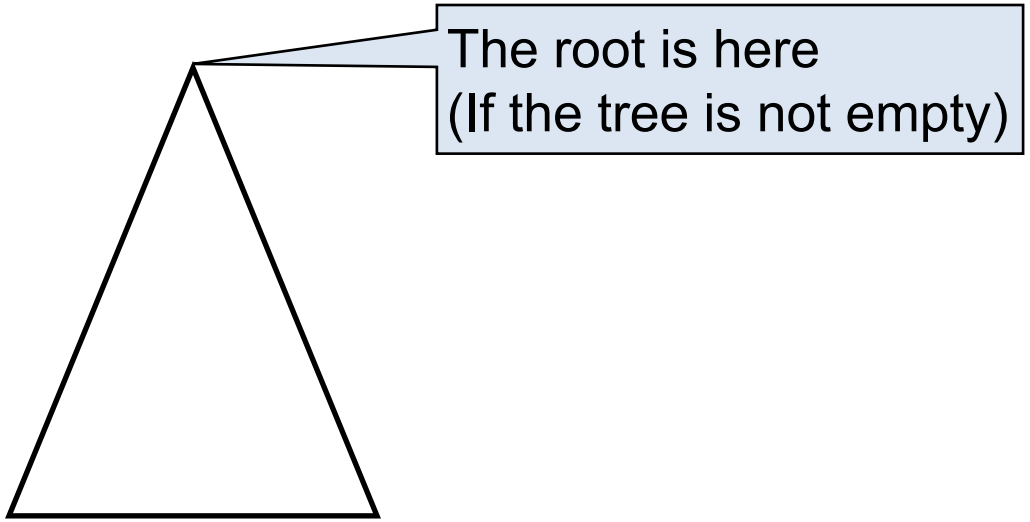
Degree of a node



Abstract illustration

We will often reason about arbitrary trees

- Their actual content is unimportant, so we abstract it away
- We draw a generic tree as a triangle

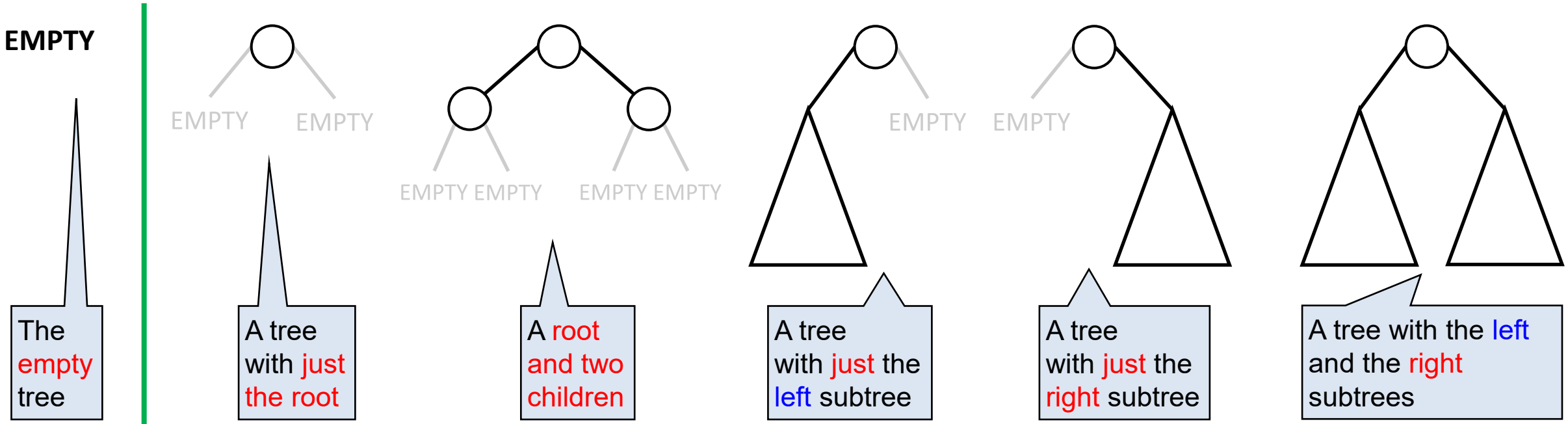


- We represent the empty tree by simply writing "**Empty**"

Empty

Instances of binary trees

Every binary tree reduces to TWO instances: either **empty** or a root with a tree on its left and a tree on its right



Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

Binary Tree Traversal

- Tree traversal (also known as tree search) is the process of visiting each node in a tree data structure exactly once in a systematic manner.

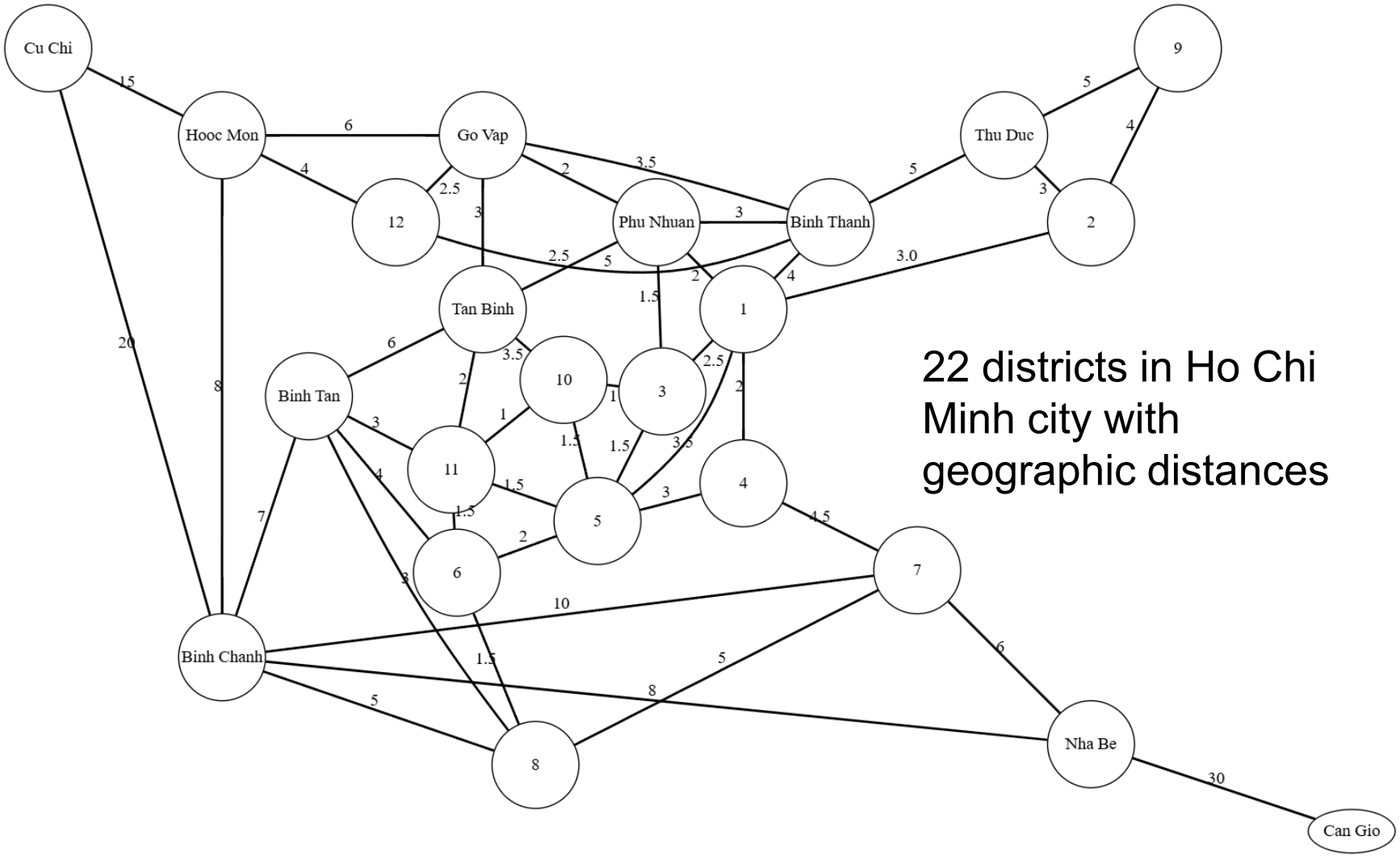
- There are two types of traversal: breadth-first search (BFS) or depth-first search (DFS).

- The following sections describe BFS and DFS on binary trees, but in general, they can be applied to any type of trees, or even graphs.

Breadth-first search (BFS)

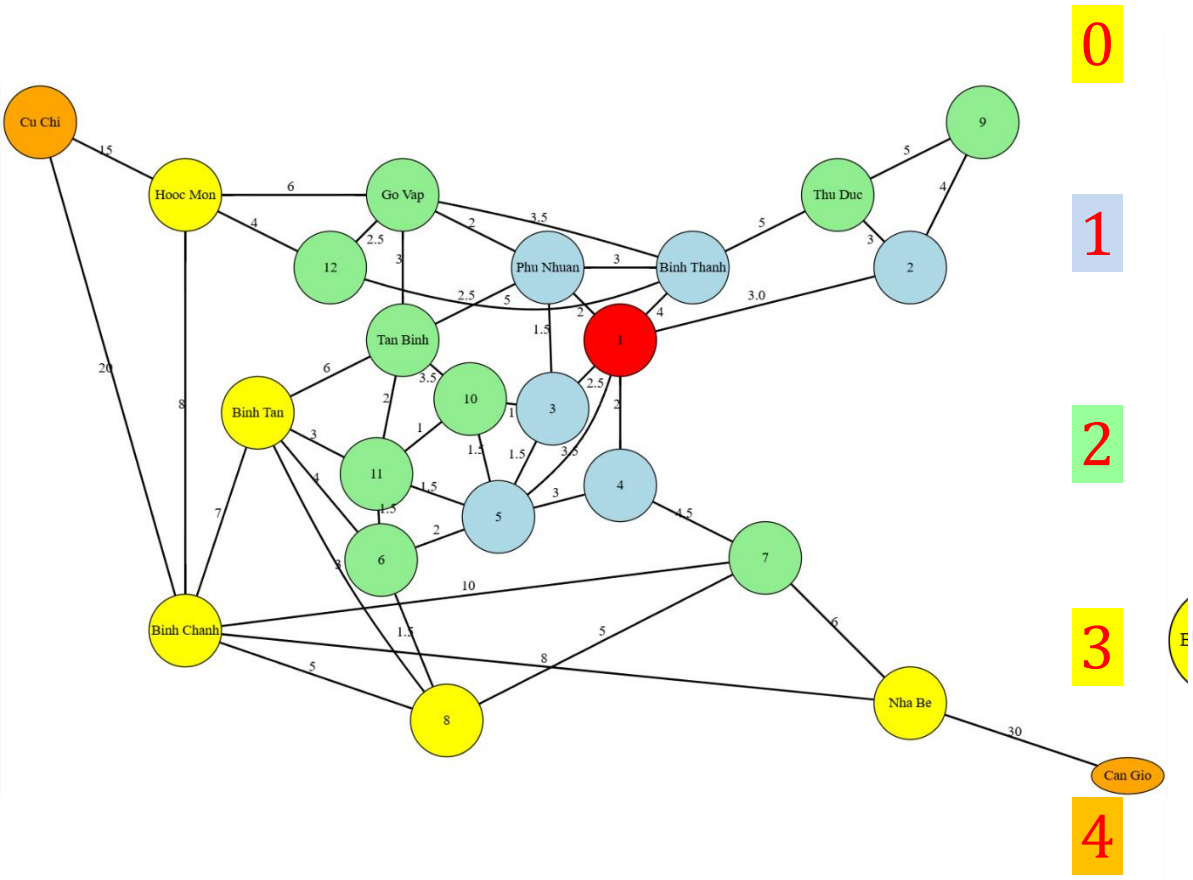
- Breadth-first search (BFS) was first invented by Zuse in 1945 [1] in his rejected PhD thesis and later reinvented by E.F. Moore in 1959 [2].

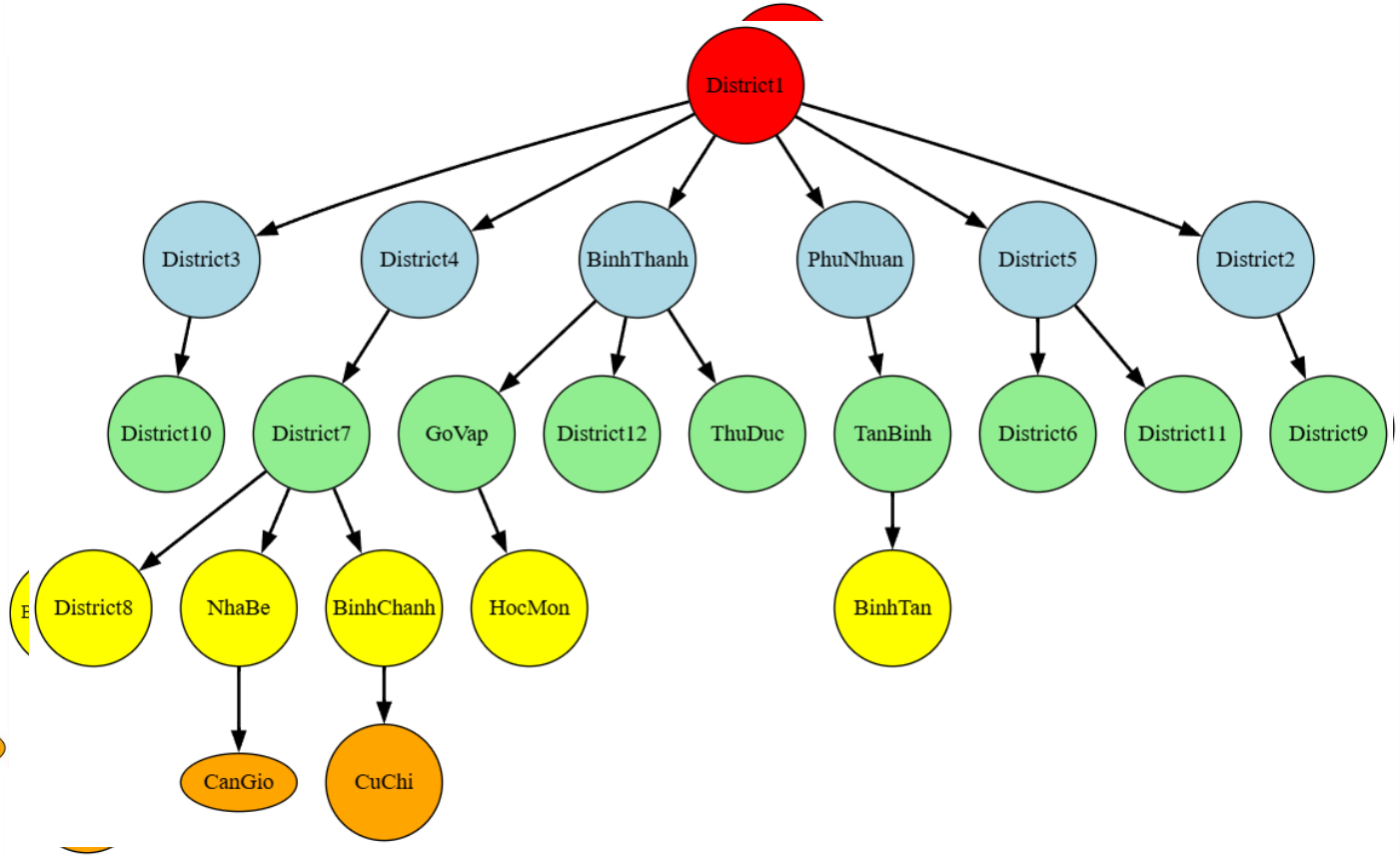
- It starts at the root and visits its adjacent nodes, and then moves to the next level.



1. Zuse, Konrad. *Der Plankalkül.* (1972).
2. Moore, Edward F. *The shortest path through a maze*. Proc. of the Int. Symp. the Theory of Switching. Harvard Univ. Press, 1959.

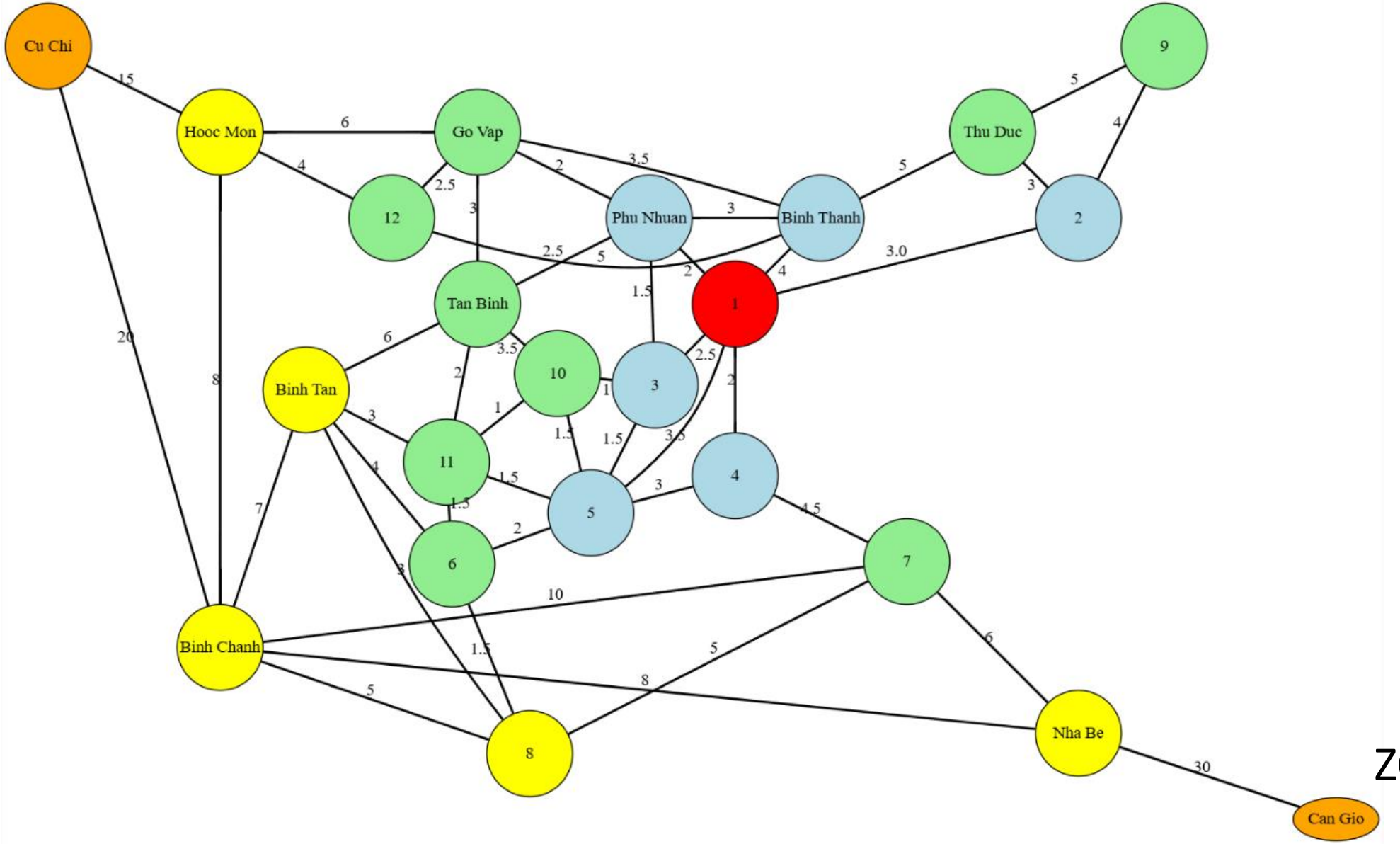
BFS for 22 districts in Ho Chi Minh city





ZONE

BFS for 22 districts in Ho Chi Minh city



BFS for 22 districts in Ho Chi Minh city

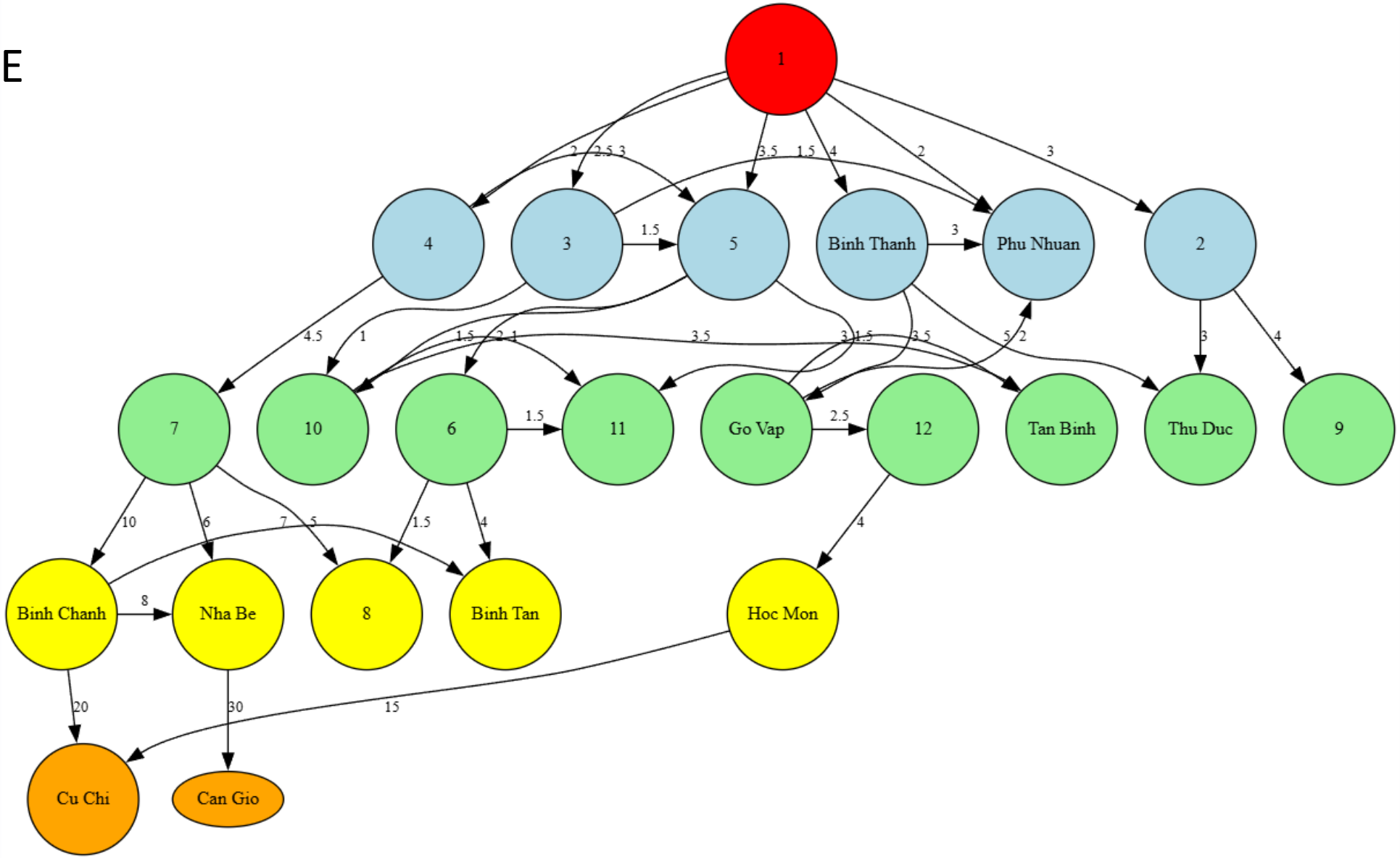
0 ZONE

1

2

3

4



Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

Depth-First Search (DFS)

There are three types of depth-first traversal:

Pre-order (Node-Left-Right)

1. Process the root.
2. Traverse EVERY subtree by recursively calling the pre-order function with the corresponding child.

```
Void NLR(TREE Root) {
  if (Root != NULL) {
    <Process Root>;
    NLR(Root->child_1);
    …
    NLR(Root->child_k);
  }
}
```

In-order (Left-Node-Right)

1. Traverse the MOST LEFT subtree by recursively calling the in-order function with the most left child.
2. Process the root.
3. Traverse the REMAINING subtrees by recursively calling the in-order function with the remaining children.

```
Void LNR(TREE Root) {
  if (Root != NULL) {
    LNR(Root->child_1);
    <Process Root>;
    LNR(Root->child_2);
    …
    LNR(Root->child_k);
  }
}
```

Post-order (Left-Right-Node)

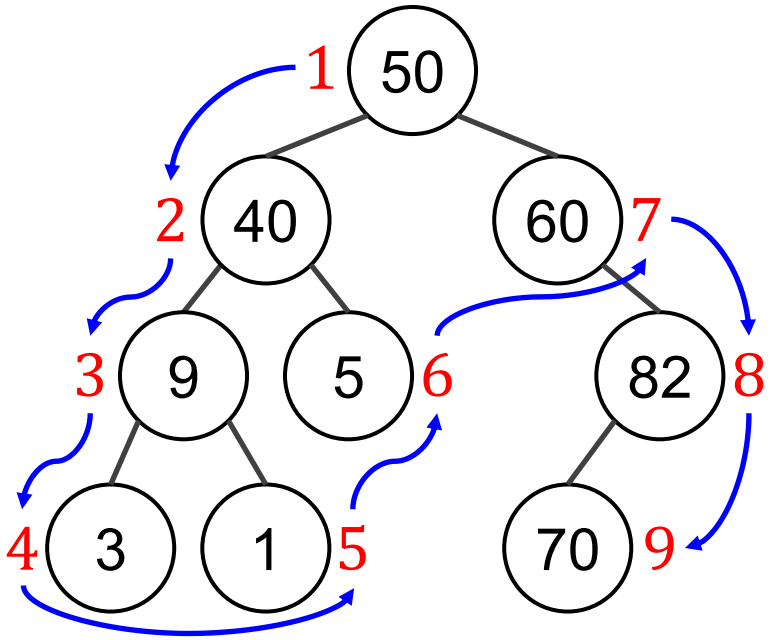
1. Traverse EVERY subtree by recursively calling the post-order function with the corresponding child.
2. Process the root.

```
Void LRN(TREE Root) {
  if (Root != NULL) {
    LRN(Root->child_1);
    …
    LRN(Root->child_k);
    <Process Root>;
  }
}
```

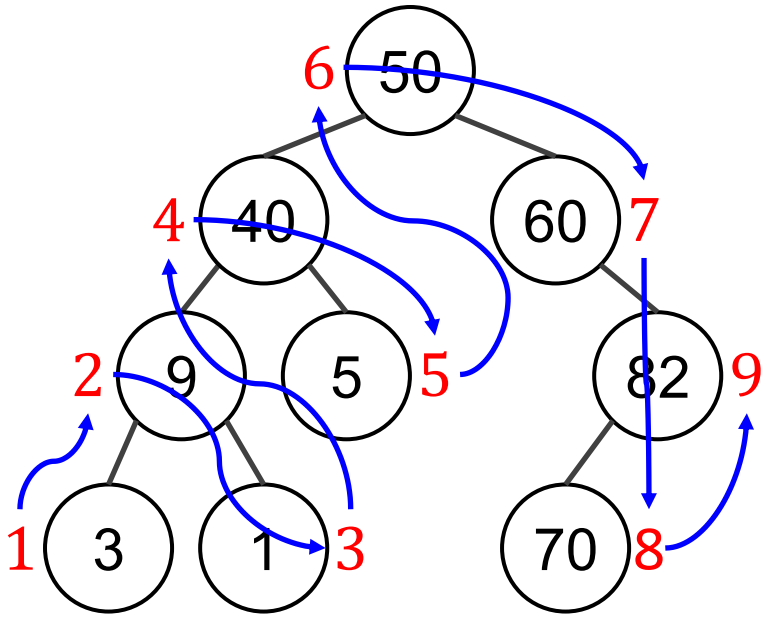
Example (1/2)

Numbers beside nodes indicate traversal order

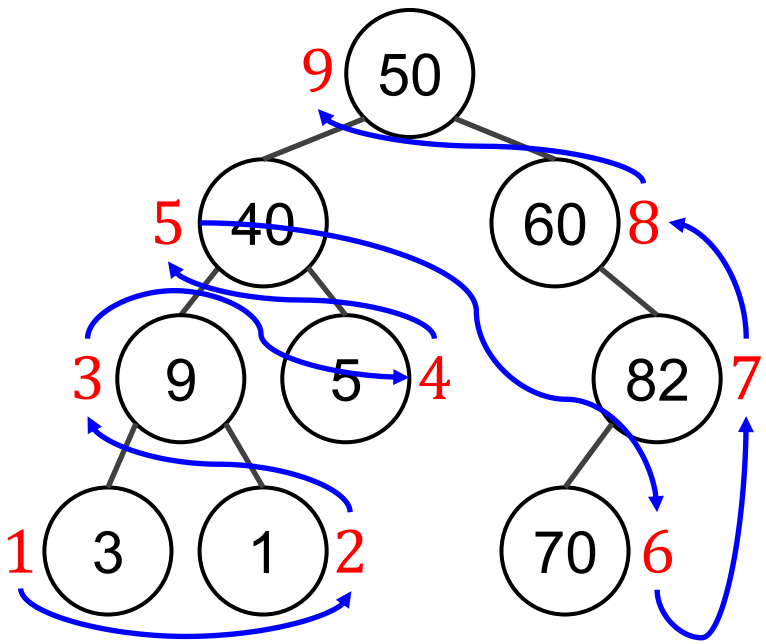
Pre-order (NLR):



In-order (LNR):



Post-order (LRN):



Pre-order (NLR):

50, 40, 9, 3, 1, 5, 60, 82, 70

In-order (LNR):

3, 9, 1, 40, 5, 50, 60, 70, 82

Post-order (LRN):

3, 1, 9, 5, 40, 70, 82, 60, 50

Example (2/2)

Pre-order

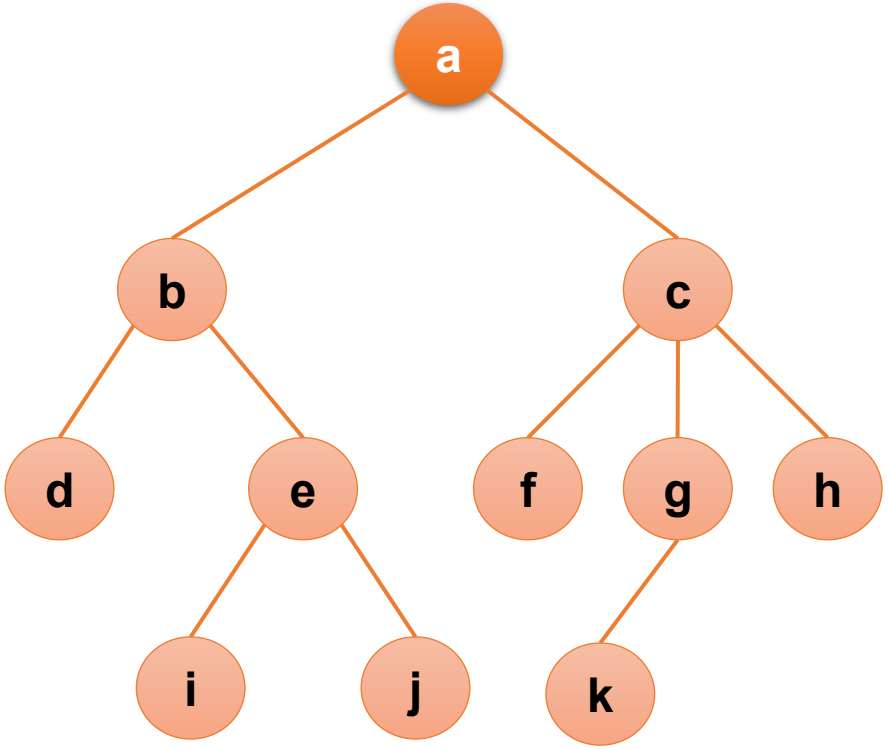
- *a b d e i j c f g k h*

In-order

- d b i e j a f c k g h

Post-order

- *d i j e b f k g h c a*

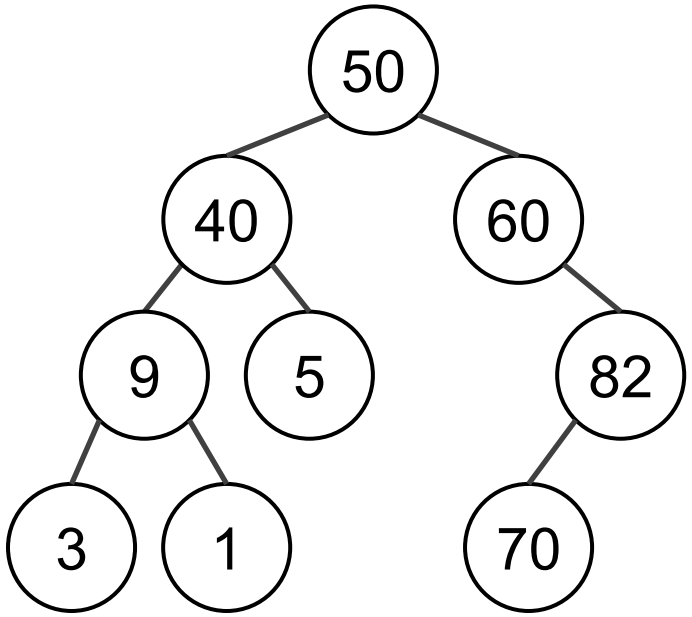


Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees

Tree representation

1. Diagram (Visual Tree Drawing)
   - Nodes are circles or boxes.
   - Lines (edges) connect parents to children.
   - Typically drawn top-down.



2. Adjacency List
   - Every node lists its direct children.

- 50: [40, 60]
- 40: [9, 5]
- 60: [82]
- 9: [3, 1]
- 82: [70]
- 3: []
- 1: []
- 5: []
- 70: []

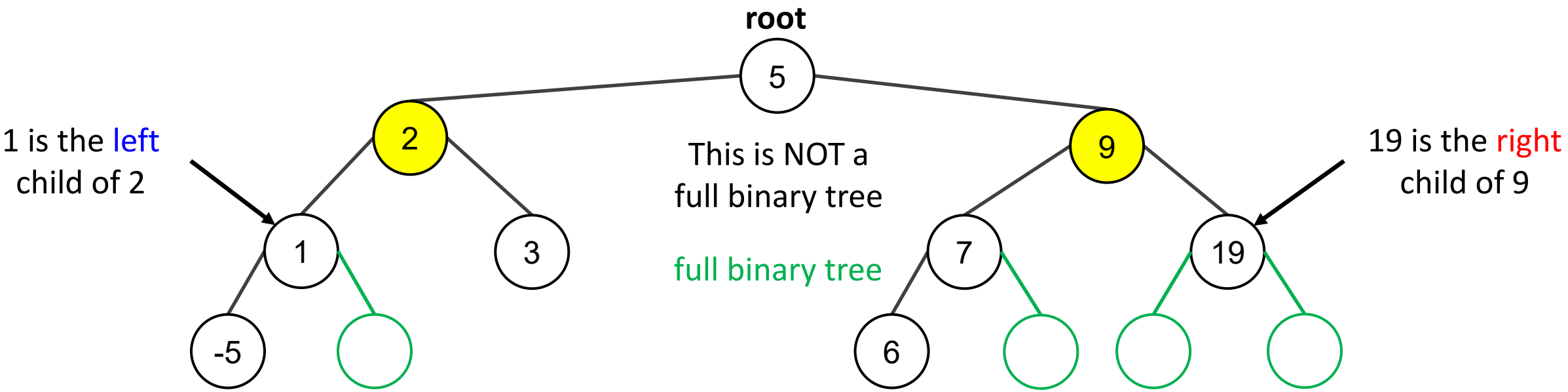
3. Parent Array
   - Store the parent of each node.

- 50: None
- 40: 50
- 60: 50
- 9: 40
- 5: 40
- 82: 60
- 3: 9
- 1: 9
- 70: 82

(Full) Binary trees

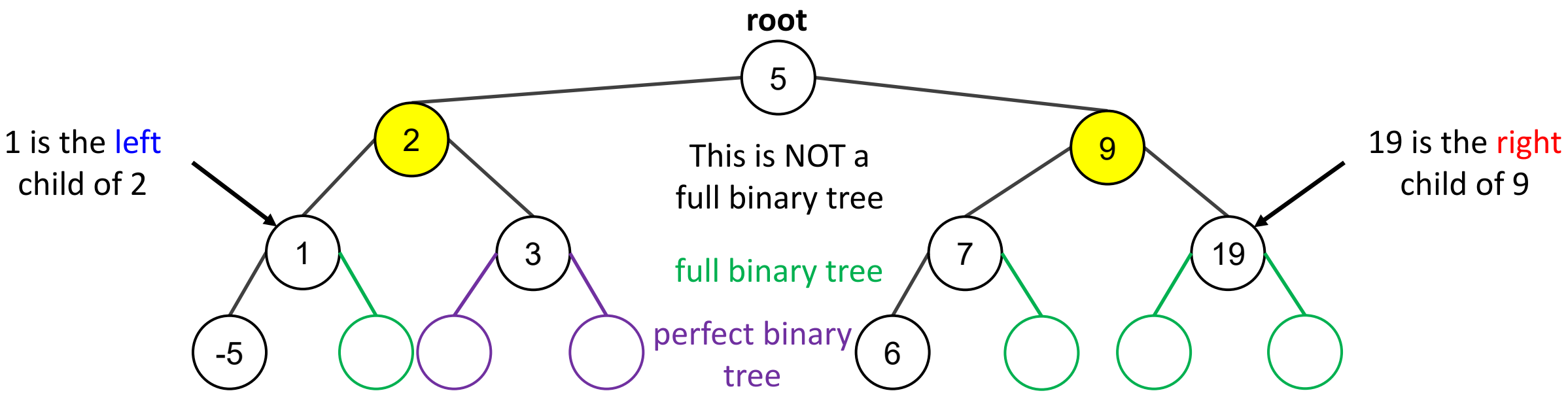
Definition 5 (A (full) binary tree). A **binary tree** is a rooted tree in which every parent has at most two children. Each child is designated either a **left child** or a **right child** (but not both), and every parent has at most one left child and one right child.

A **full binary tree** is a binary tree in which each PARENT has exactly two children.



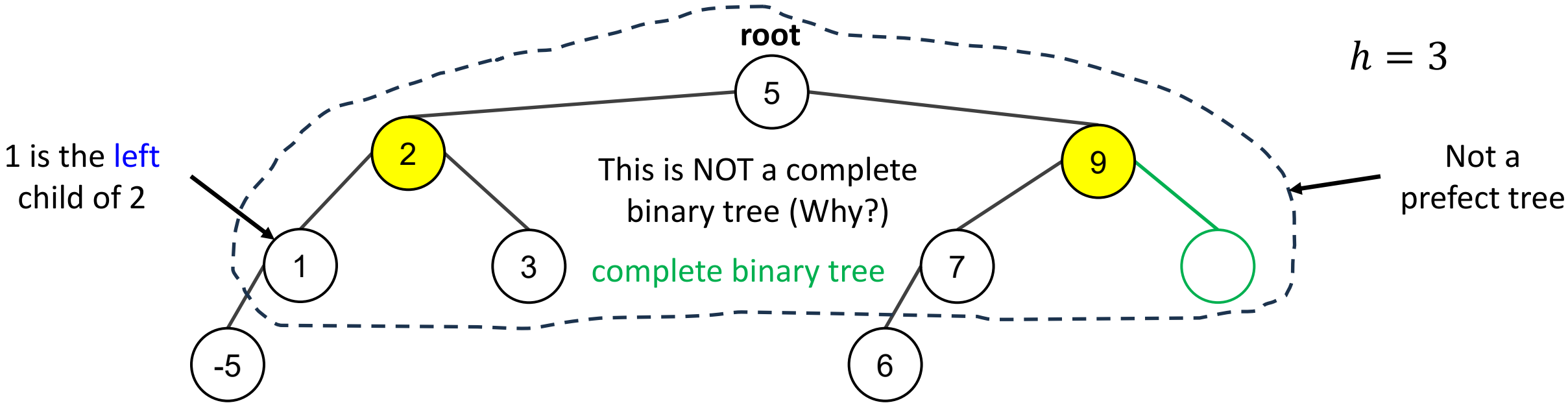
Perfect binary trees

Definition 6 (A perfect binary tree). A **binary tree** is a rooted tree in which every parent has EXACTLY two children and ALL leaves have the same level (depth).



Complete binary trees

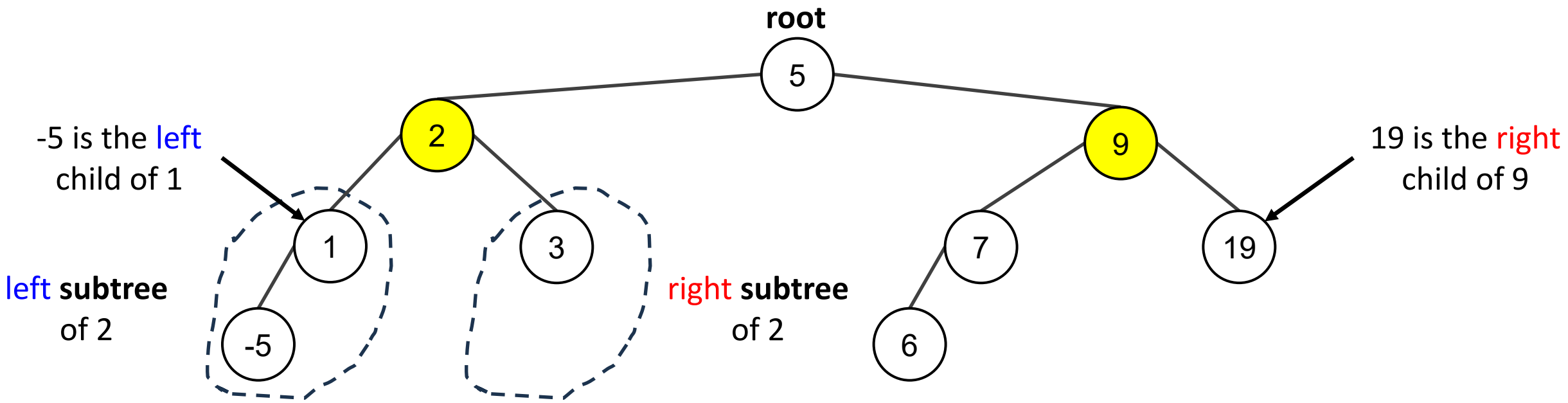
Definition 7 (A complete binary tree). A **complete binary tree** of height h is a rooted tree that is perfect down to level $h - 1$.



Left Subtree, Right Subtree

Definition 8. Given any parent v in a binary tree T, if v has a left child, then the left **subtree** of v is the binary tree whose root is the left child of v, whose nodes consist of the left child of v and all its descendants, and whose edges consist of all those edges of T that connect the nodes of the left **subtree**.

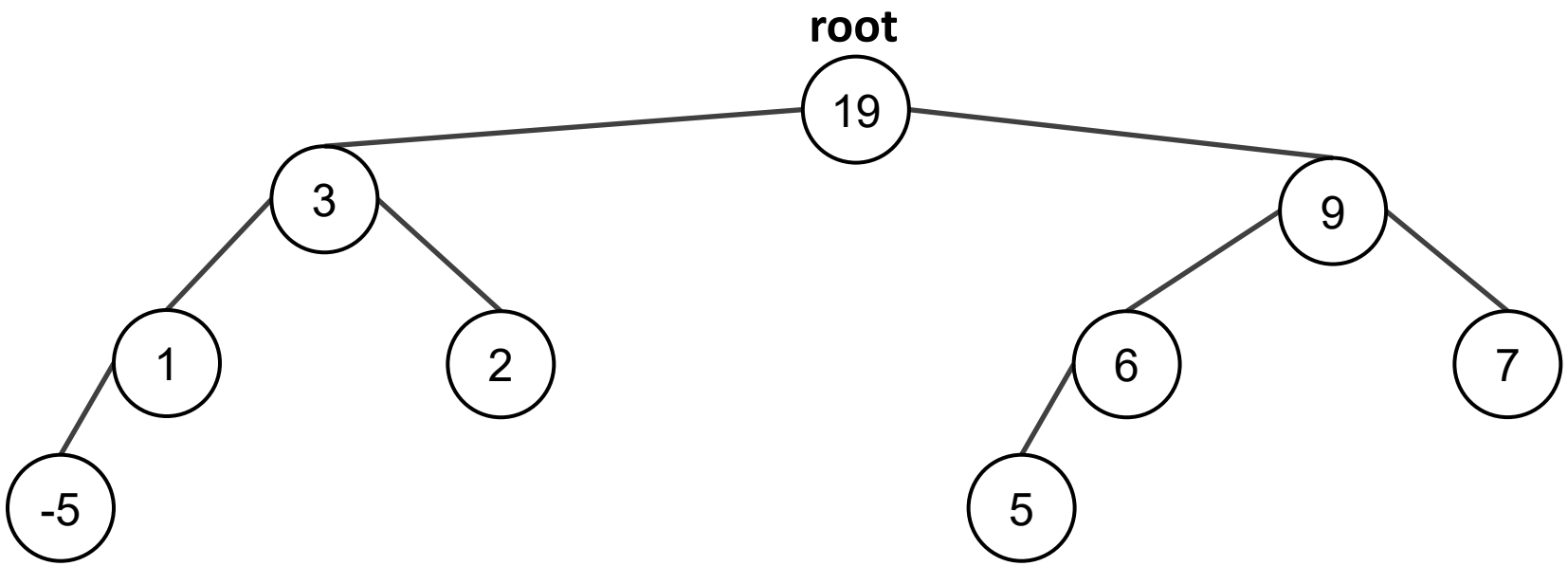
The right **subtree** of v is defined analogously.



Heap as a tree

Definition 9 (A (max) heap (tree)). A max **heap** is a complete (binary) tree that is either empty or its root:

1. Contains a value greater than or equal to the value in each of its children, and
2. Has heaps as its subtrees



Theorem 10.5.1. If k is a positive integer and T is a full binary tree with k internal vertices, then (1) T has a total of $2k + 1$ vertices, and (2) T has $k + 1$ leaves.

- Suppose k is a positive integer and T is a full binary tree with k internal vertices.

1. Observe that the set of all vertices of T can be partitioned into two disjoint subsets: the set of all vertices that have a parent and the set of all vertices that do not have a parent. Now there is just one vertex that does not have a parent, namely the root. Also, since every internal vertex of a full binary tree has exactly two children, the number of vertices that have a parent is twice the number of parents, or $2k$, since each parent is an internal vertex. Hence

$$\#\text{vertices of } T = \#\text{vertices that have a parent} + \#\text{vertices that do not have a parent} = 2k + 1$$

2. Because it is also true that the total number of vertices of T equals the number of internal vertices plus the number of leaves,

$$\#\text{vertices of } T = \#\text{internal vertices} + \#\text{leaves} = k + \#\text{leaves}.$$

- Now equate the two expressions for the total number of vertices of T:

$$2k + 1 = k + \#\text{leaves}.$$

- Thus the total number of vertices is $2k + 1$ and the number of leaves is $k + 1$.

Exercise

Q: Is there a full binary tree that has 10 internal vertices and 13 terminal vertices?

No, by Theorem 10.6.1, a full binary tree with 10 internal vertices has 10 + 1 = 11 terminal vertices.

Height and leaves of a Binary Tree

Theorem 1. For non-negative integers h, if T is any binary tree with height h and t leaves, then

$$t \leq 2^h.$$

Equivalently,

$$\log_2 t \leq h.$$

- This theorem says that the maximum number of leaves of a binary tree of height h is 2^h. Alternatively, a binary tree with t leaves has height of at least $\log_2 t$.

- **Claim**: The number of nodes at level i is at most 2^i.

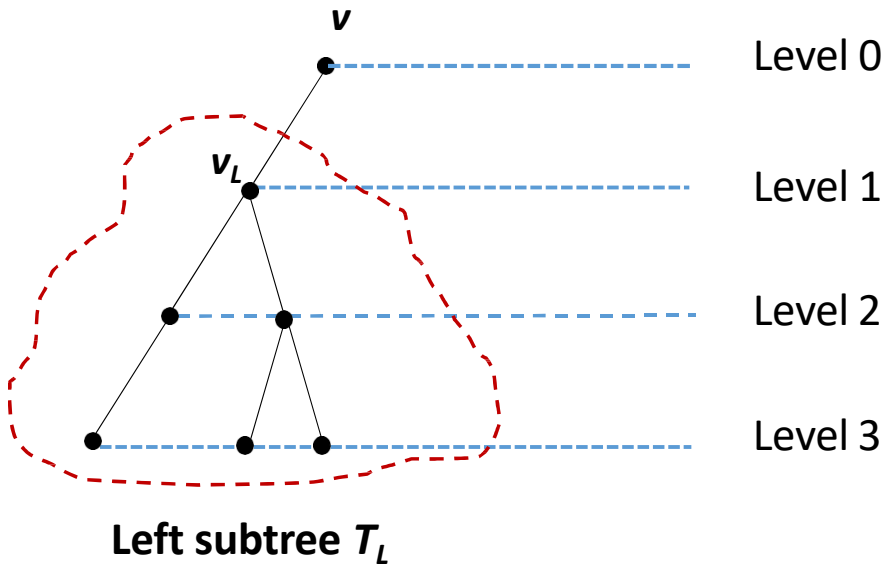
Proof (1/3)

Proof: By mathematical induction

1. Let $P(h)$ be "If T is any binary tree of height h, then the number of leaves of T is at most 2^h."
2. $P(0)$: T consists of one vertex, which is a terminal vertex. Hence $t = 1 = 2^0$.
3. Show that for all integers $k \geq 0$, if $P(i)$ is true for all integers i from 0 through k, then $P(k+1)$ is true.
4. Let T be a binary tree of height $k + 1$, root v, and t leaves.
5. Since $k \geq 0$, hence $k + 1 \geq 1$ and so v has at least one child.
6. We consider two cases: If v has only one child, or if v has two children.

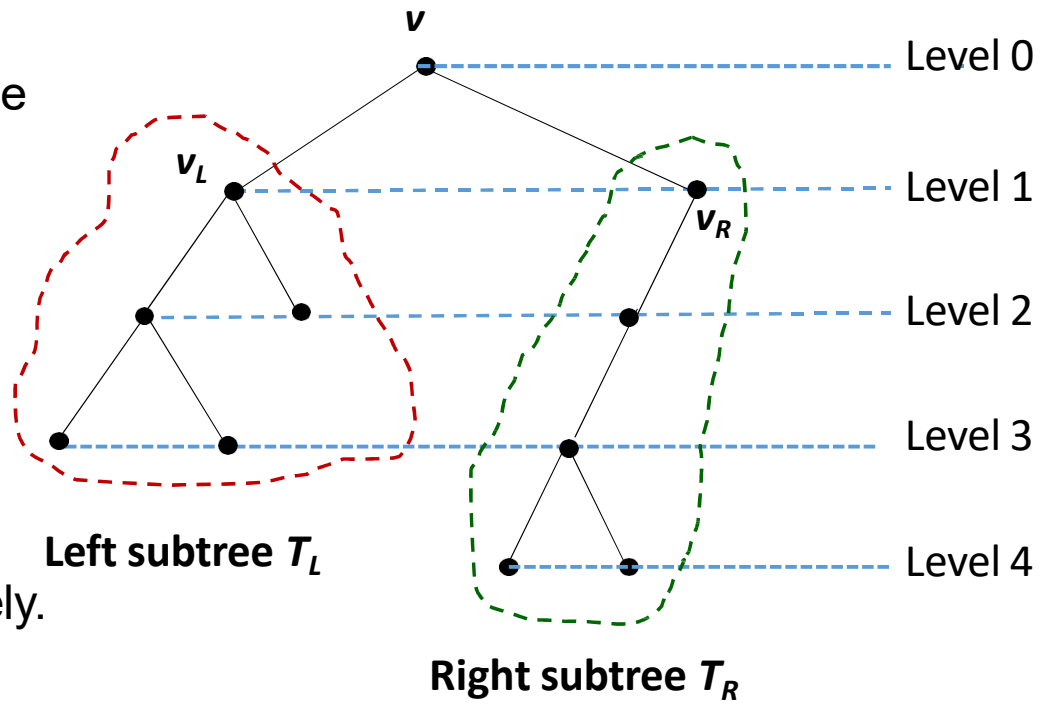
Proof (2/3): Case 1 (*v* has only one child)

1. Without loss of generality, assume that *v*'s child is a left child and denote it by v_L. Let T_L be the left subtree of *v*.
2. Because *v* has only one child, *v* is a leaf, so the total number of leaves in *T* equals the number of leaves in T_L + 1. Thus, if t_L is the number of leaves in T_L , then $t = t_L + 1$.
3. By inductive hypothesis, $t_L \leq 2^k$ because the height of T_L is *k*, one less than the height of *T*.
4. Also, because *v* has a child, $k + 1 \geq 1$ and so $2^k \geq 2^0 = 1$.
5. Therefore, $t = t_L + 1 \leq 2^k + 1 \leq 2^k + 2^k = 2^{k+1}$.



Proof (3/3): Case 2 (*v* has two children)

1. Now *v* has a left child v_L and a right child v_R , and they are the roots of a left subtree T_L and a right subtree T_R respectively.
2. Let h_L and h_R be the heights of T_L and T_R respectively.
3. Then $h_L \leq k$ and $h_R \leq k$ since *T* is obtained by joining T_L and T_R and adding a level.
4. Let t_L and t_R be the number of leaves of T_L and T_R respectively.



5. Then, since both T_L and T_R have heights less than *k* + 1, by inductive hypothesis, $t_L \leq 2^{h_L}$ and $t_R \leq 2^{h_R}$.
6. Therefore, $t = t_L + t_R \leq 2^{h_L} + 2^{h_R} \leq 2^k + 2^k = 2^{k+1}$.

- We proved both cases that *P*(*k*+1) is true.
- Hence if *T* is any binary tree with height *h* and *t* terminal vertices (leaves), then $t \leq 2^h$.

Quiz

Q: Is there a binary tree that has height 6 and 70 leaves?

No, by Theorem 1, any binary tree *T* with height 6 has at most $2^6 = 64$ leaves, so such a tree cannot have 70 leaves.

Given a binary tree *T* height of h.

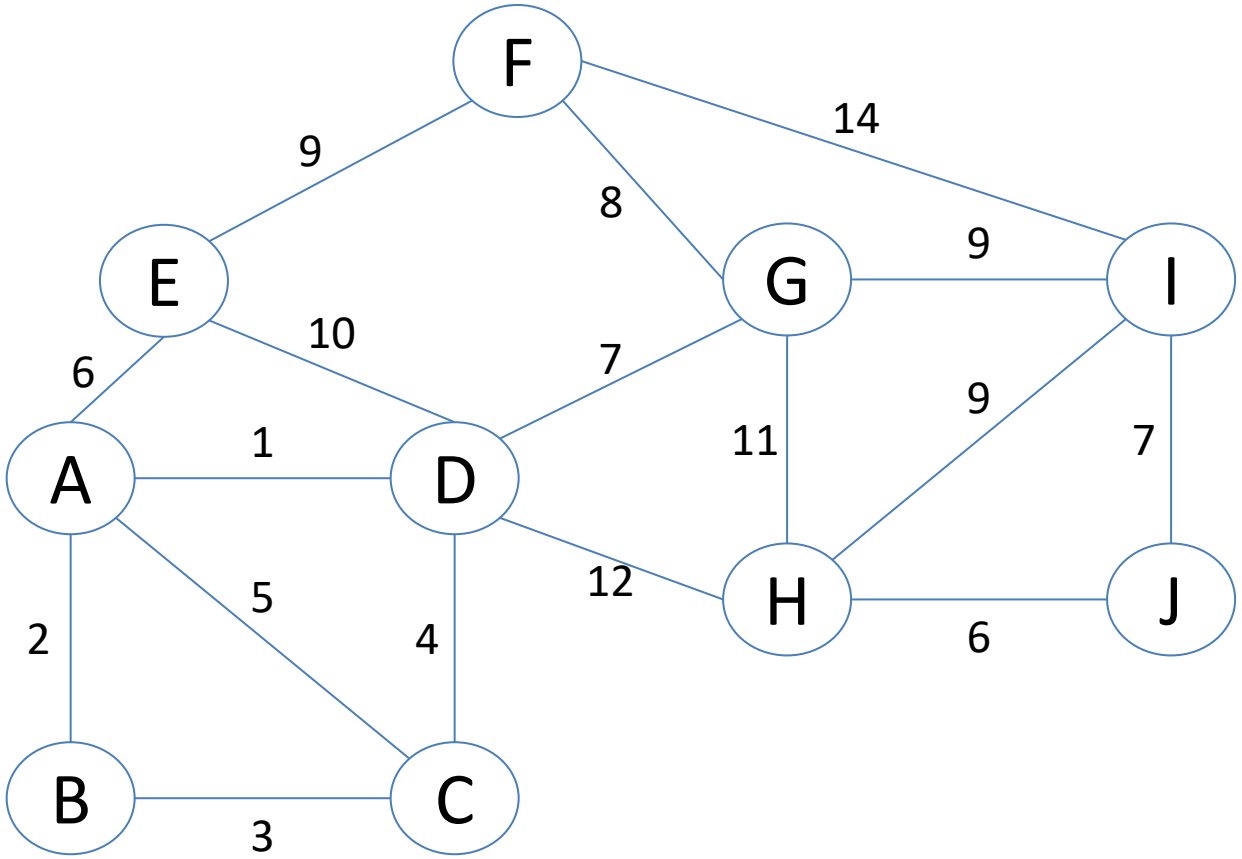
- What is the maximum number of nodes?
- What is the minimum number of nodes?

- Given a binary tree *T* with n nodes.
- What is the maximum height of that tree?
- What is the minimum height of that tree?

Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees
   - Prim's algorithm
   - Kruskal's algorithm

A Networking Problem



find a minimum (cost) spanning tree

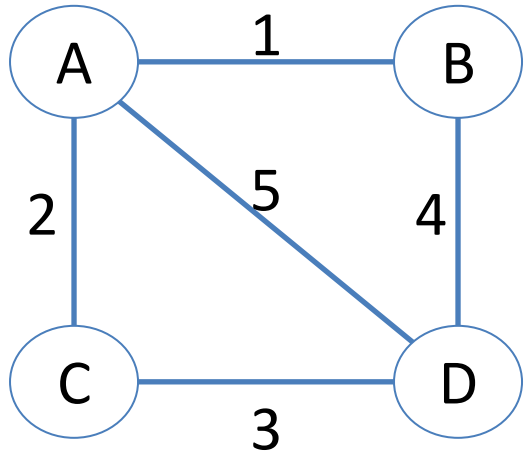
The vertices represent 10 regional data centers that need to be connected with high-speed data lines. Feasibility studies show that the links illustrated above are possible, and the cost in millions of USD is displayed next to the link. Which links should be constructed to enable full communication (with relays allowed) and keep the total cost minimal?

What is a Minimum (Cost) Spanning Tree (MST)? (1/4)

- Tree:
  - A structure with no cycles; meaning there is exactly one unique path between any two nodes.
- Spanning:
  - The tree includes all nodes in the graph.
- Minimum Cost:
  - Among all possible spanning trees, this one has the lowest total edge weight.

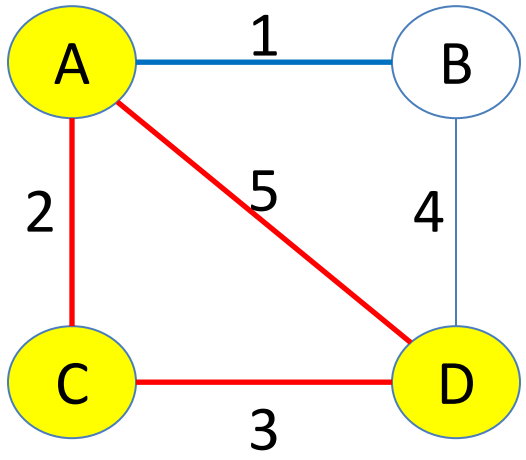
What is a Minimum (Cost) Spanning Tree (MST)? (2/4)

- Let's think about simple MSTs on this graph:

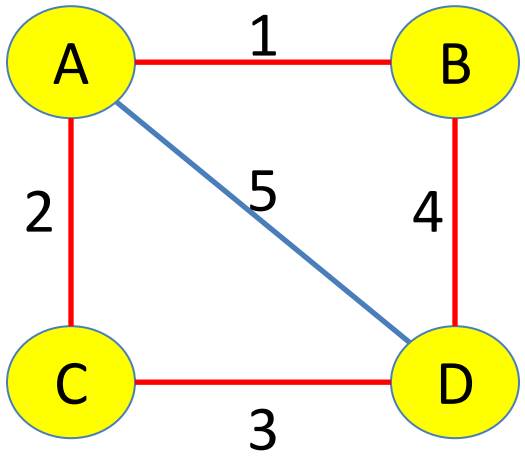


What is a Minimum (Cost) Spanning Tree (MST)? (3/4)

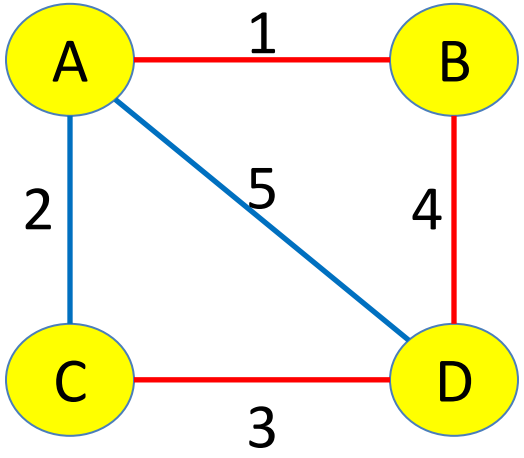
- Red edges and nodes are in T
- Is T a minimum cost spanning tree?



Not spanning; B is not in T.



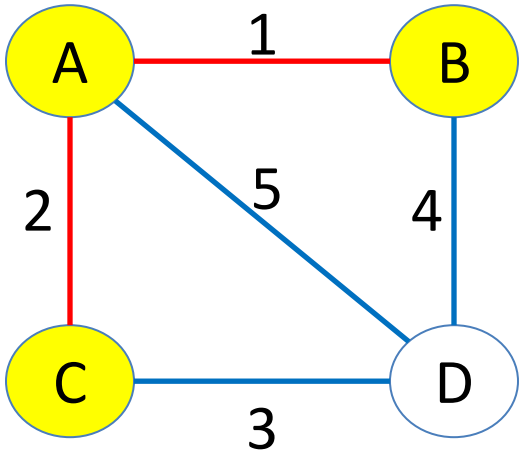
Not a tree; has a cycle.

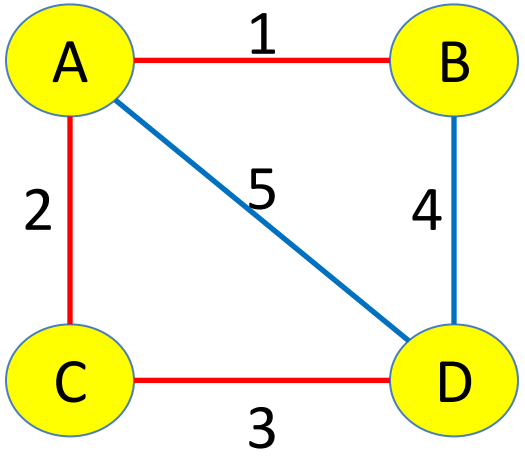


Not minimum cost; can swap edges weighted 4 and 2.

What is a Minimum (Cost) Spanning Tree (MST)? (4/4)

- Which edges form an MST?





Quiz

- If we build an MST from a graph G = (V, E), how many edges does the MST have?

- When can we find an MST for a graph?

An application of MSTs

- Electronic circuit designs [1]
  - Circuits often need to wire together the pins of several components to make them electrically equivalent.
  - To connect n pins, we can use $n - 1$ wires, each connecting two pins.
  - Want to use the minimum amount of wire.
  - Model a problem with a graph where each pin is a node, and every possible wire between a pair of pins is an edge.

[1] Cormen, Thomas H., et al. *Introduction to algorithms*. MIT press, 2022.

A few other applications of MSTs

- Clustering Algorithms:
  - MSTs can be used as a basis for some clustering algorithms. By removing the longest edges in an MST, the tree can be broken into disconnected components, which can represent clusters.
- Network Design (Telecommunications, Computer Networks, Utilities):
  - Designing power grids or water distribution networks with the minimum cost of laying down the infrastructure.
  - Optimizing the path data as it travels across routers to its destination on the internet.
- Generating a 2D maze design such as the Pacman game.

Prim's vs. Kruskal's algorithms

- Two famous algorithms for finding an MST: Prim's and Kruskal's algorithms.

Feature	Kruskal's	Prim's
Approach	Edge-based	Vertex-based
Best for	Sparse graphs	Dense graphs
Data structure needed	Union-Find	Priority Queue (Heap)
Time Complexity	$O(\|E\| \log \|E\|)$	$O\big((\|V\| + \|E\|) \log \|V\|\big)$ with Binary Heap
Works well with	Edge list	Adjacency list or matrix

Outline

1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees
   - Prim's algorithm
   - Kruskal's algorithm

History

Vojtěch Jarník
(1897–1970)
(dml.cz/jarnik-en)

Robert Clay Prim
(1921–2021)
do.ithistory.org/honor-roll/dr-robert-clay-prim

Edsger Wybe Dijkstra
(1930–2002)
amturing.acm.org/award_winners/dijkstra_1053701.cfm

- The algorithm was developed in 1930 by the Czech mathematician Vojtěch Jarník [1].
- Later rediscovered and republished by computer scientists Robert C. Prim in 1957 [2] and Edsger W. Dijkstra in 1959 [3].

[1] Boruvka, O. "O jistém problému minimálním (about a certain minimal problem). Práce moravské prírodovedecké spolecnosti v Brne 3: 37–58." (1926).
[2] Prim, Robert Clay. "Shortest connection networks and some generalizations." *The Bell System Technical Journal* 36.6 (1957): 1389-1401.
[3] Dijkstra, Edsger W. "A note on two problems in connexion with graphs." *Edsger Wybe Dijkstra: his life, work, and legacy*. 2022. 287-290.

Building an MST

- Prim's algorithm takes a graph $G = (V, E)$ and builds an MST with a list of vertices T and its list of edges L

PrimMST(V, E)

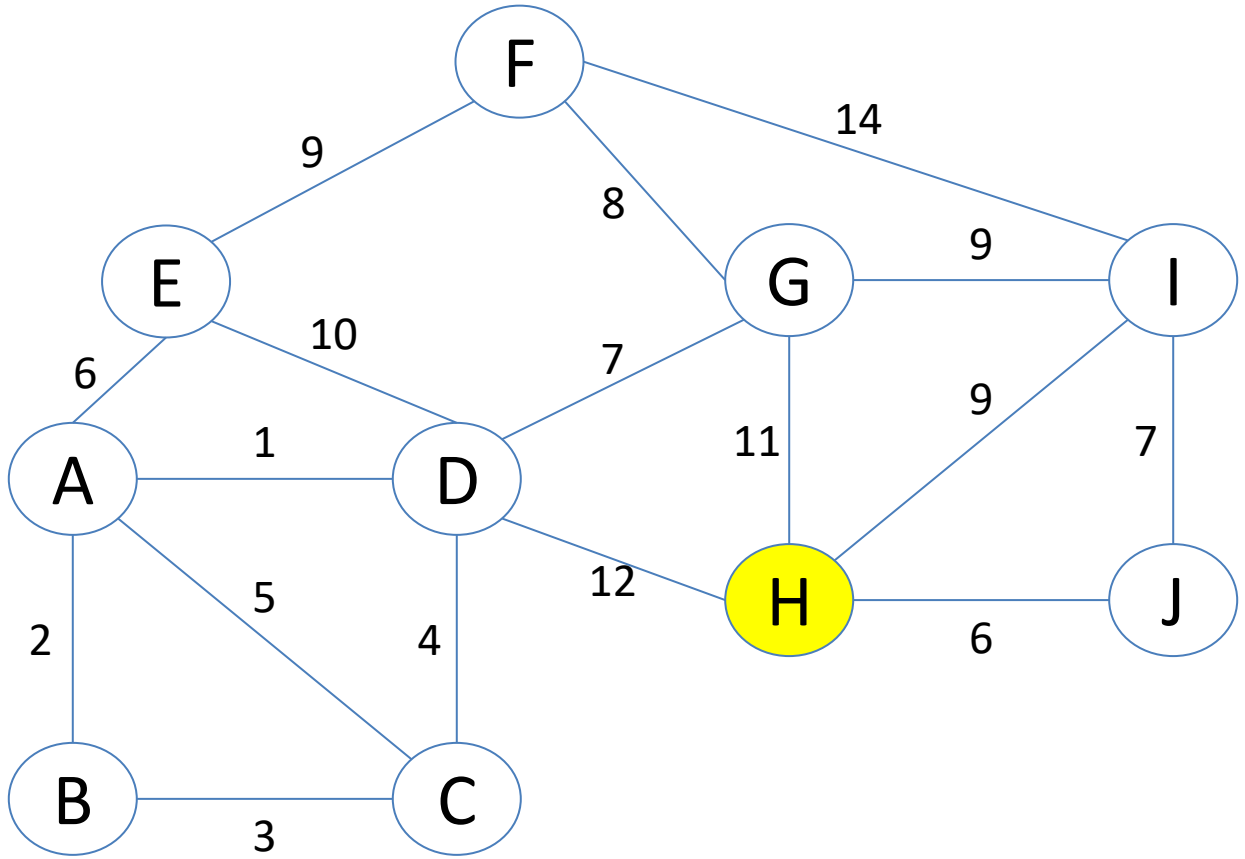
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L

$v \in \bar{T} = V \setminus T$

Example (1/11)

- Find an MST starting at vertex H by using Prim's algorithm

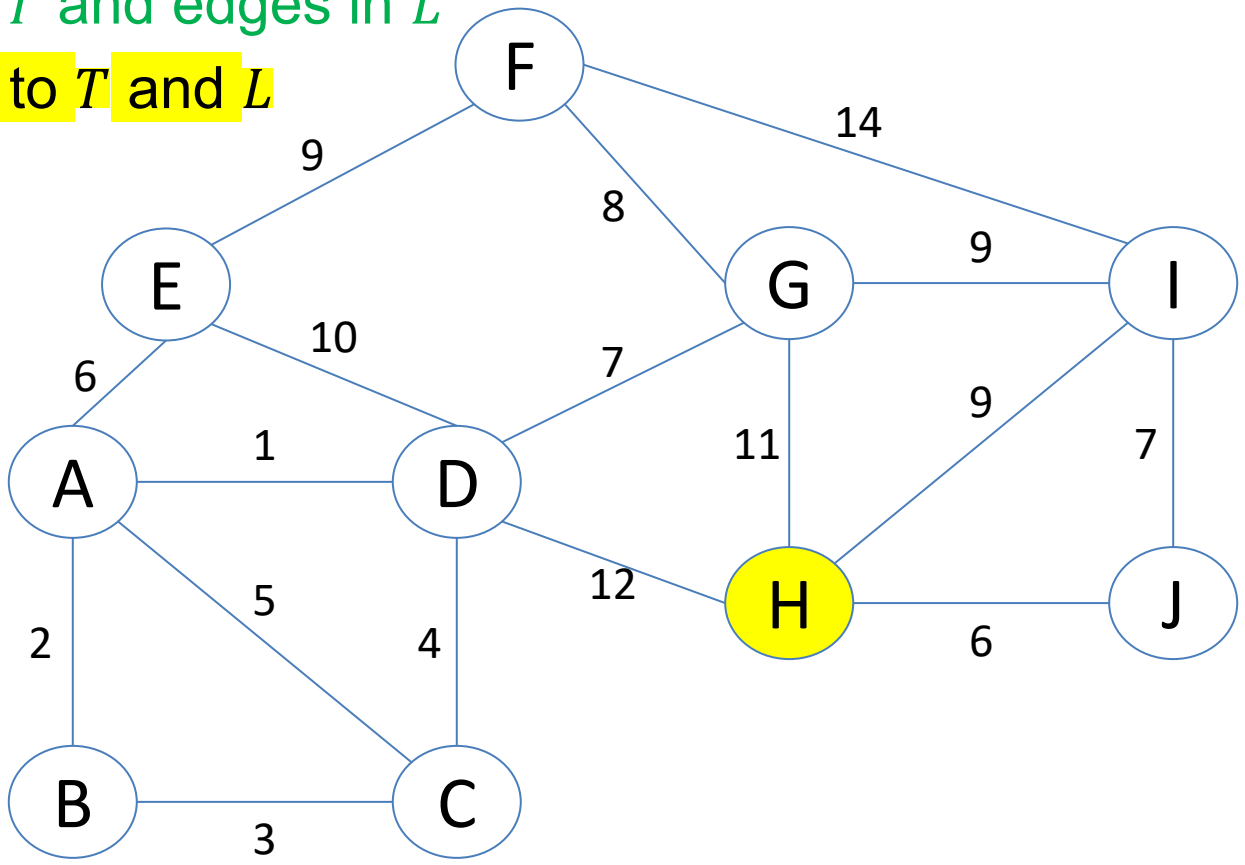
PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



Example (1/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \emptyset$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, j, h\}$$



$$L = \emptyset$$

$$\text{cost}(L) = 0$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L

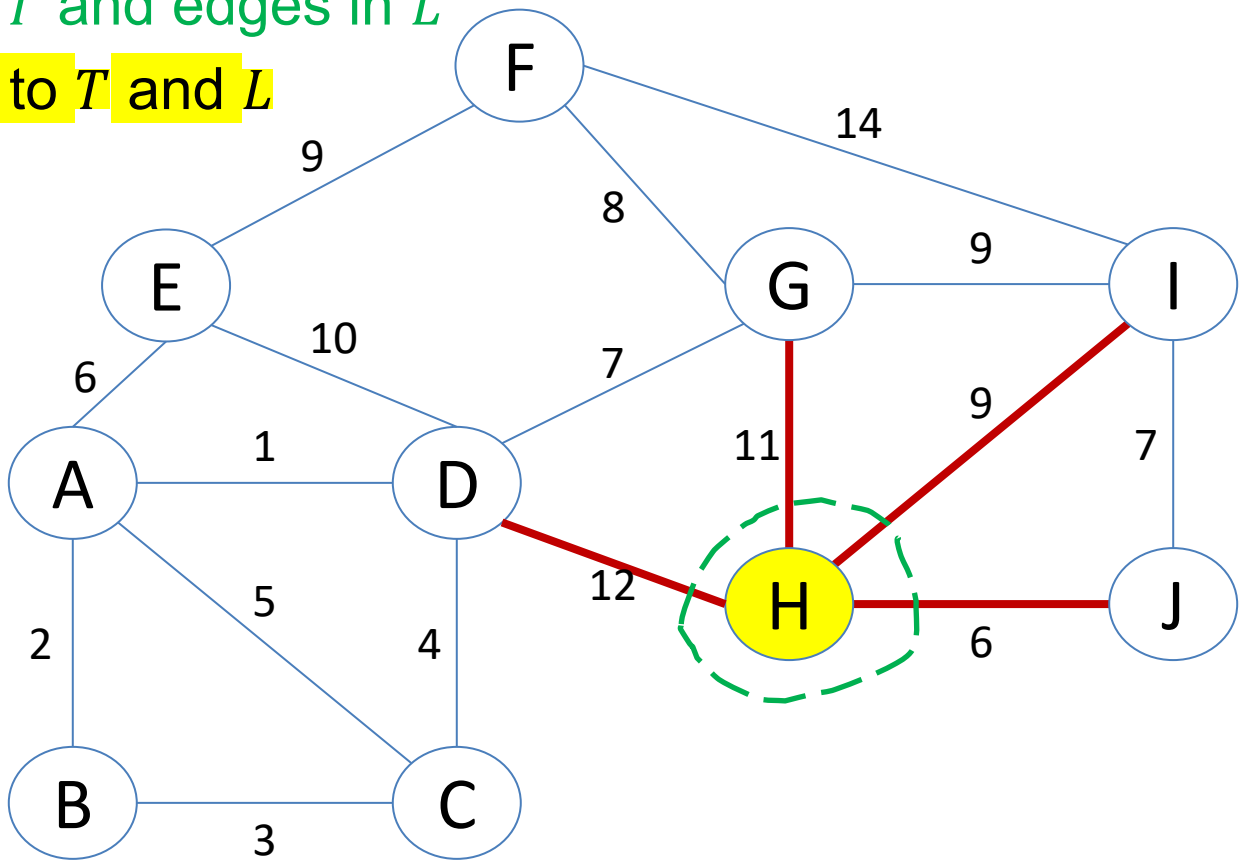
Example (2/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, j, \cancel{h}\}$$

$$L = \emptyset$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$\text{cost}(L) = 0$$

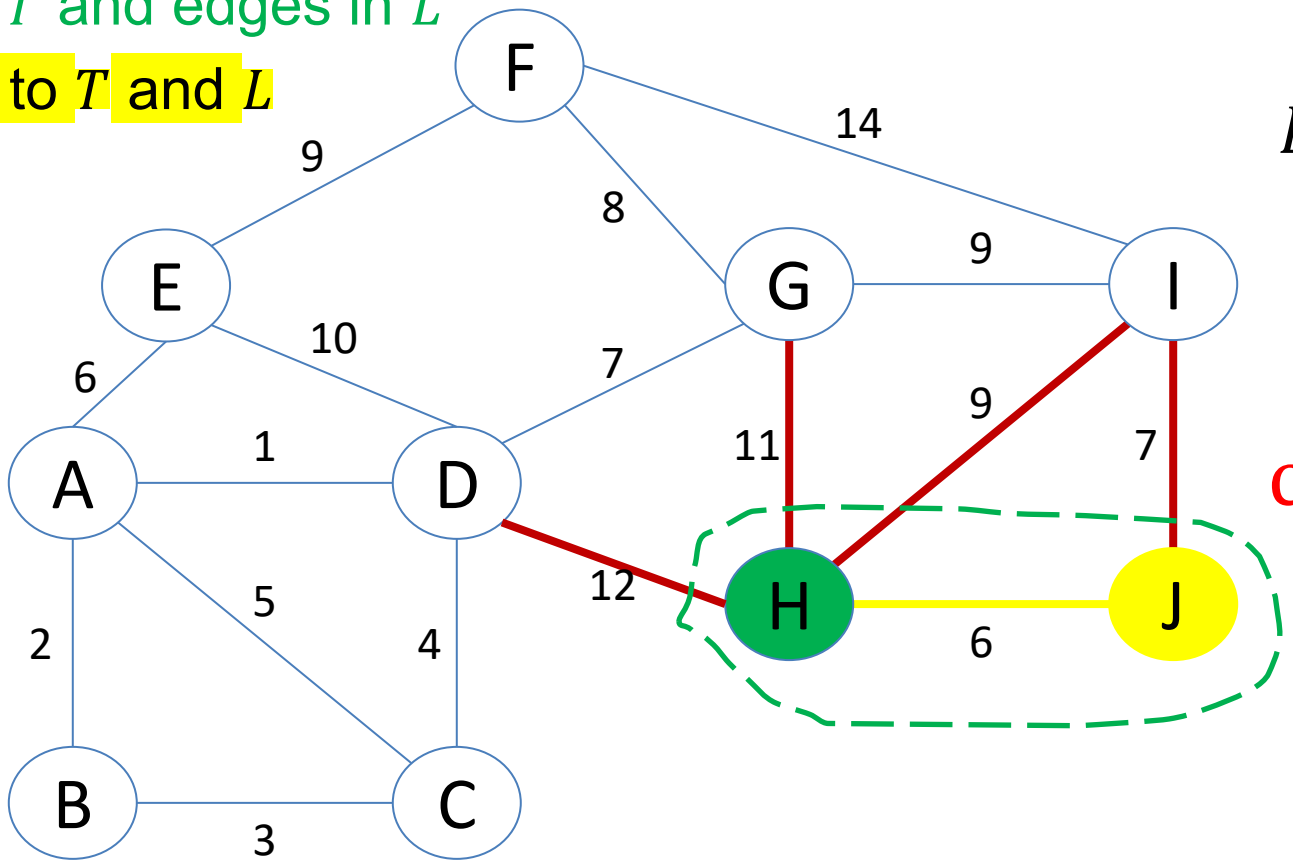
Example (3/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, i, \cancel{j}, \cancel{h}\}$$

$$L = \{(h, j)\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



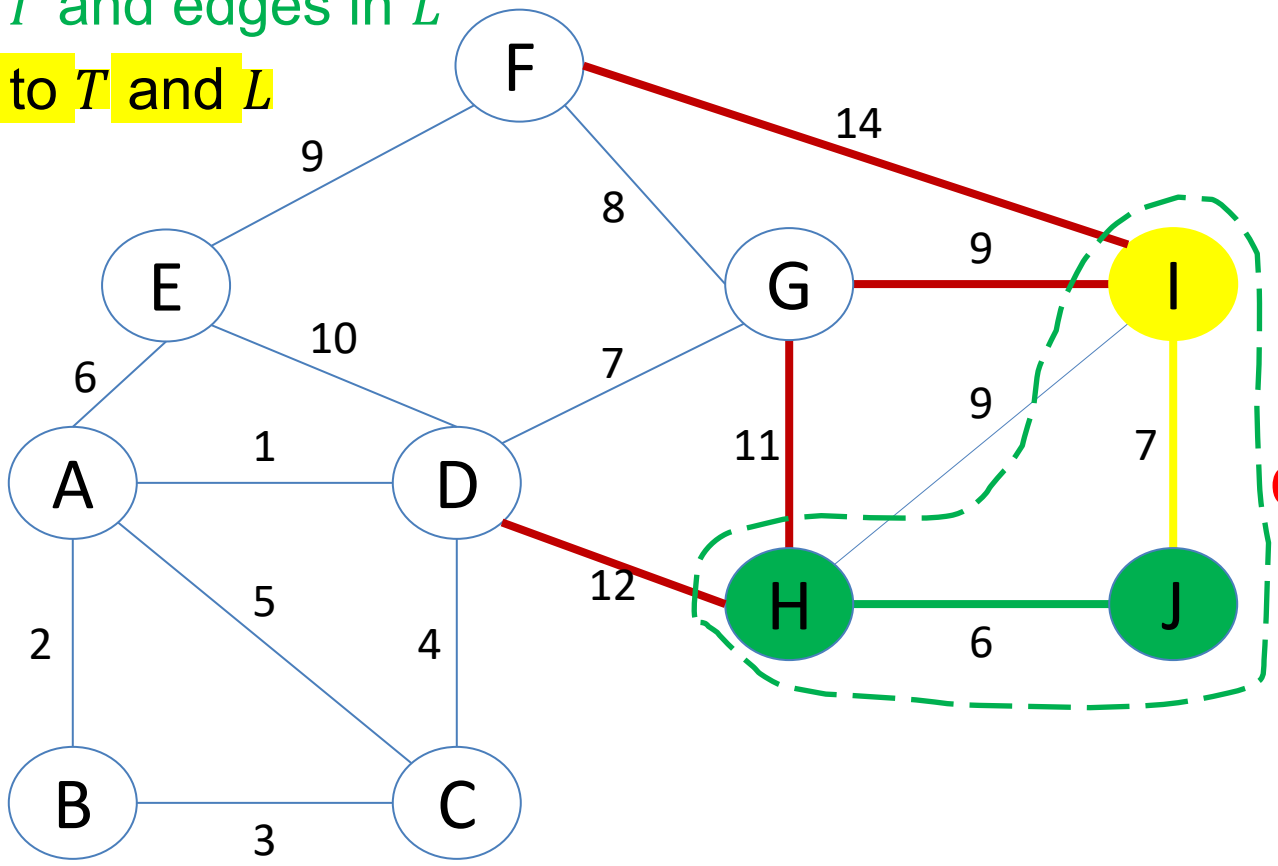
$$\text{cost}(L) = 6$$

Example (4/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i\}$$
$$\bar{T} = V \setminus T = \{a, b, c, d, e, f, g, \cancel{i}, \cancel{j}, \cancel{h}\}$$
$$L = \{(h, j), (j, i)\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L

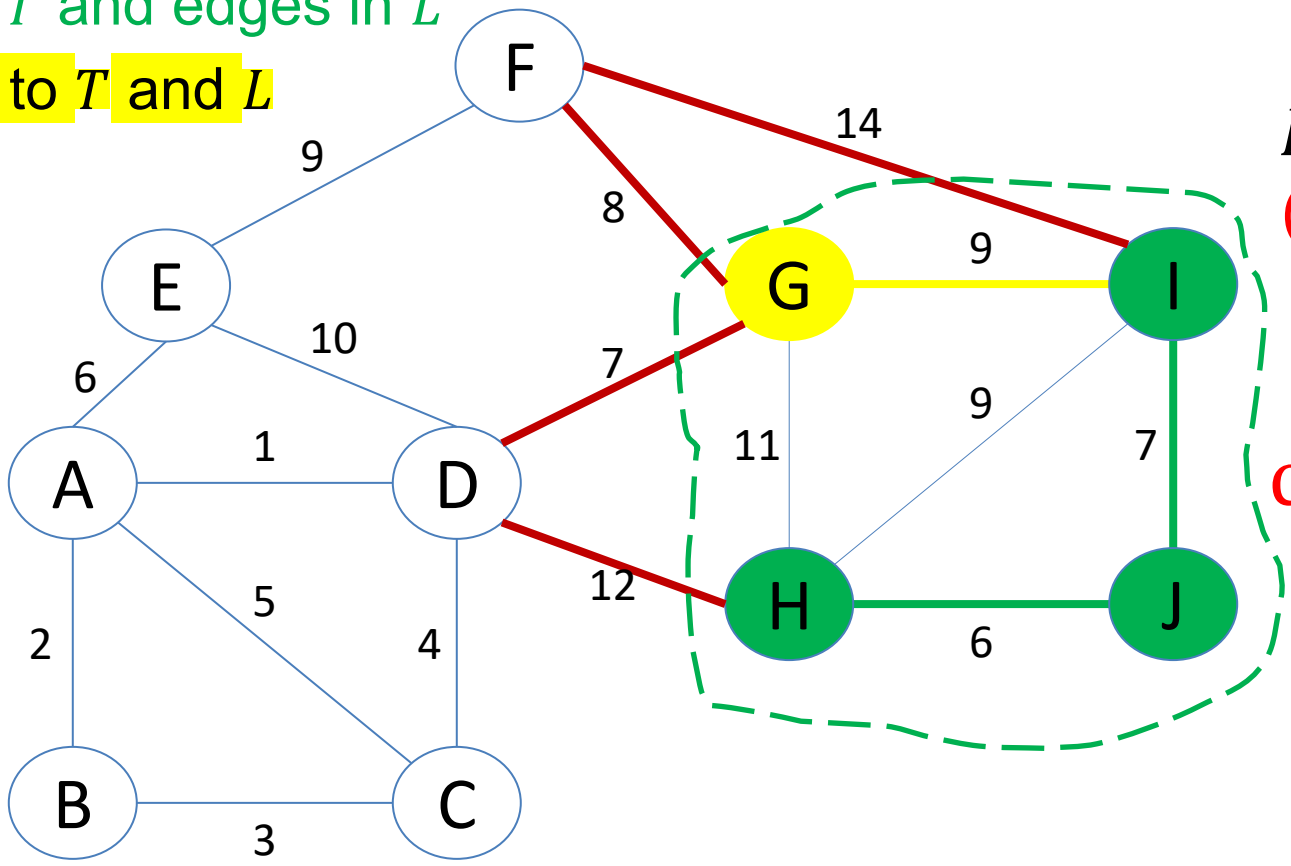


$$\text{cost}(L) = 13$$

Example (5/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$T = \{h, j, i, g\}$

$\bar{T} = V \setminus T = \{a, b, c, d, e, f, \sout{g}, \sout{i}, \sout{j}, \sout{h}\}$

$L = \{(h, j), (j, i) (i, g)\}$

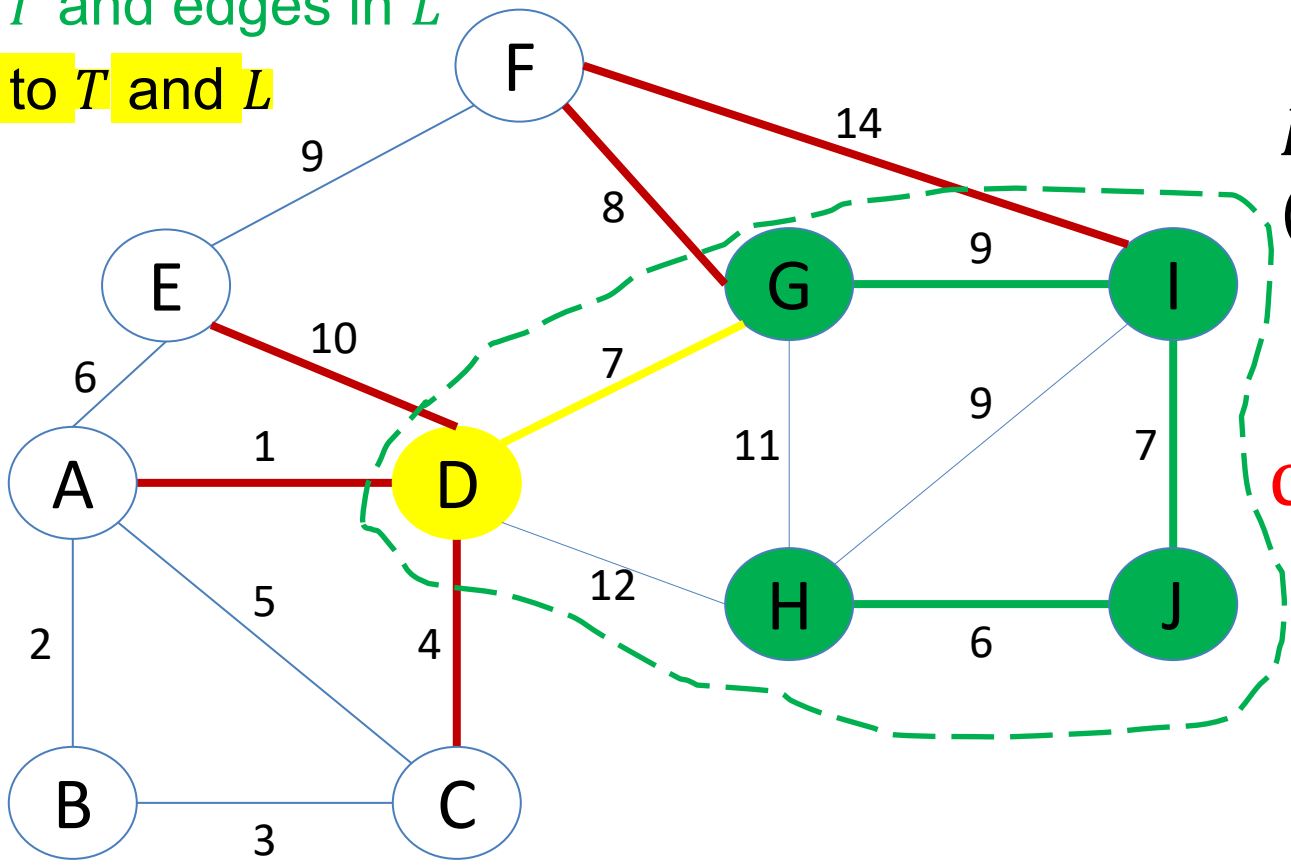
$\text{cost}(L) = 22$

Example (6/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d\}$$
$$\bar{T} = V \setminus T = \{a, b, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



$$L = \{(h, j), (j, i)$$
$$(i, g), (g, d)\}$$

$$\text{cost}(L) = 29$$

Example (7/11)

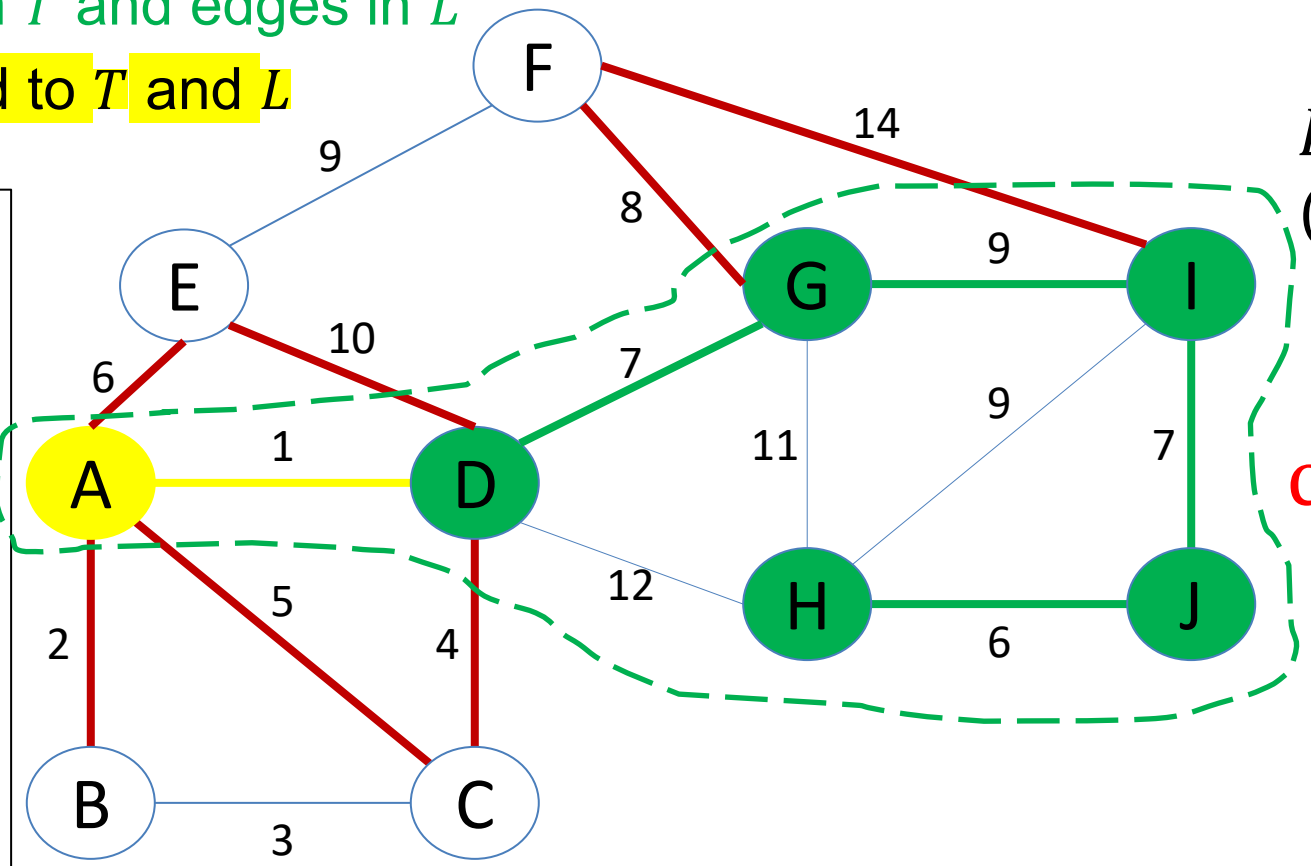
- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, b, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

$$L = \{(h, j), (j, i)$$
$$(i, g), (g, d), (d, a)\}$$

$$\text{cost}(L) = 30$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✓
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L

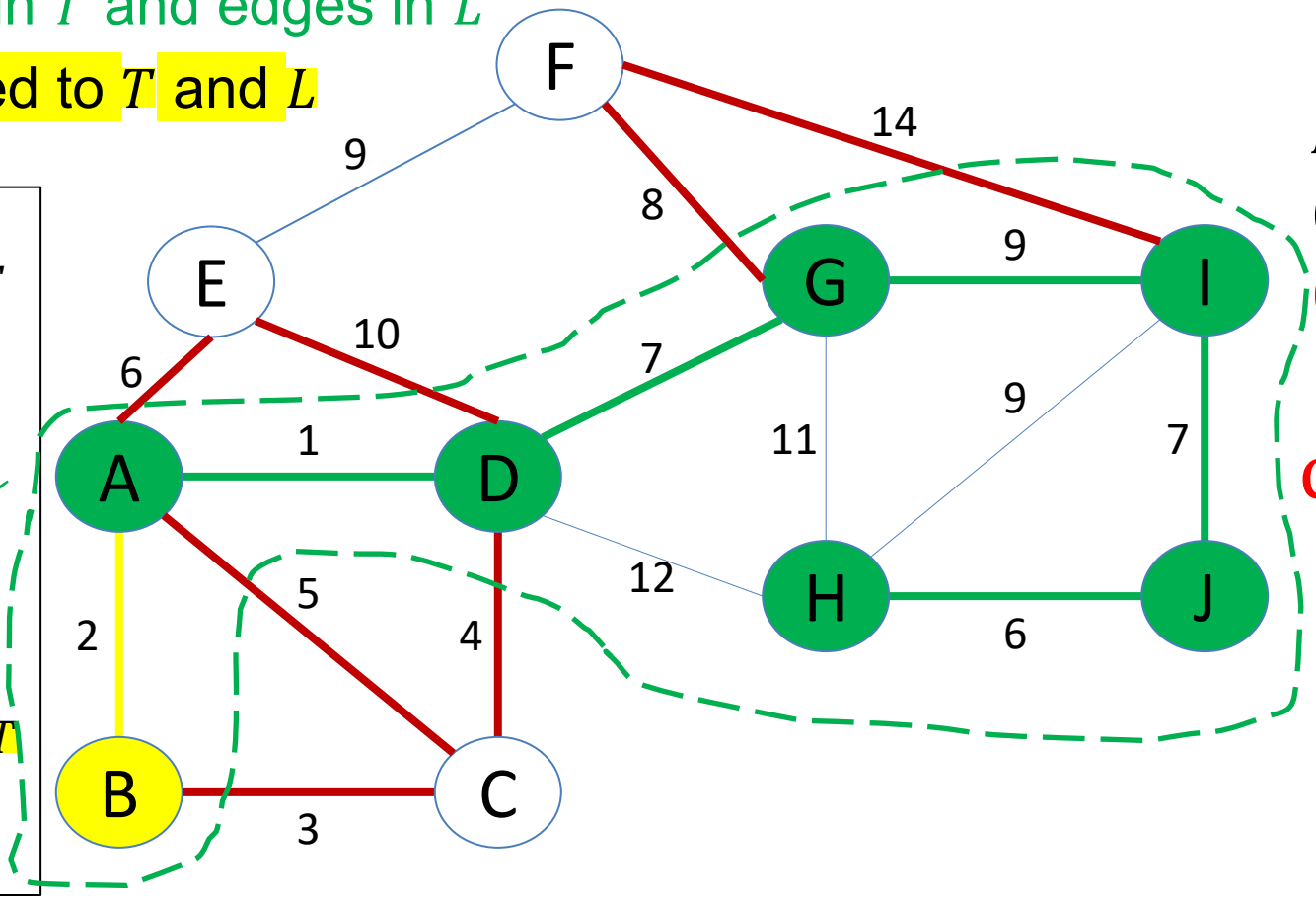


Example (8/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, c, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b)\}$$

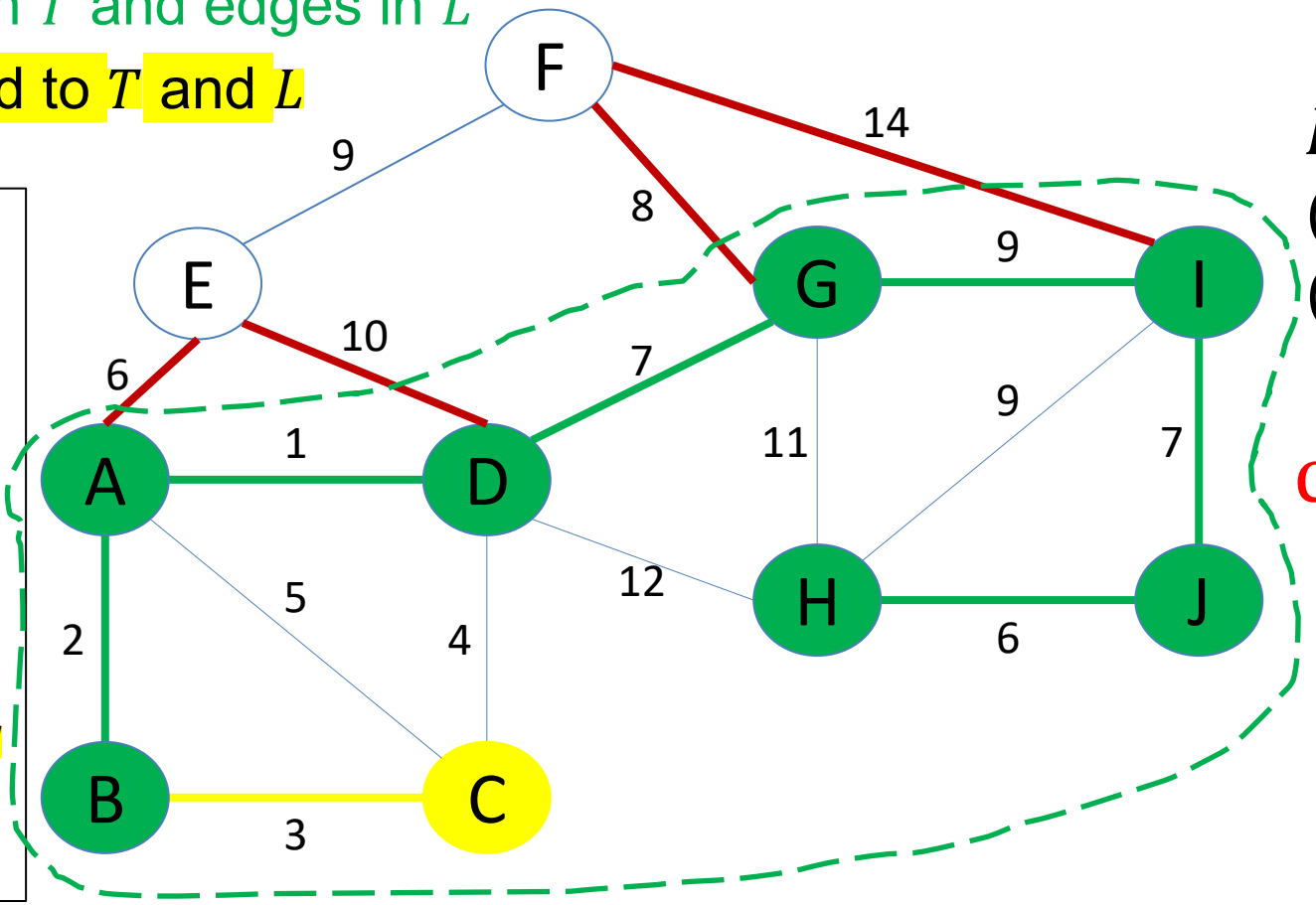
$$\text{cost}(L) = 32$$

Example (9/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, e, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✓
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) to L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c)\}$$

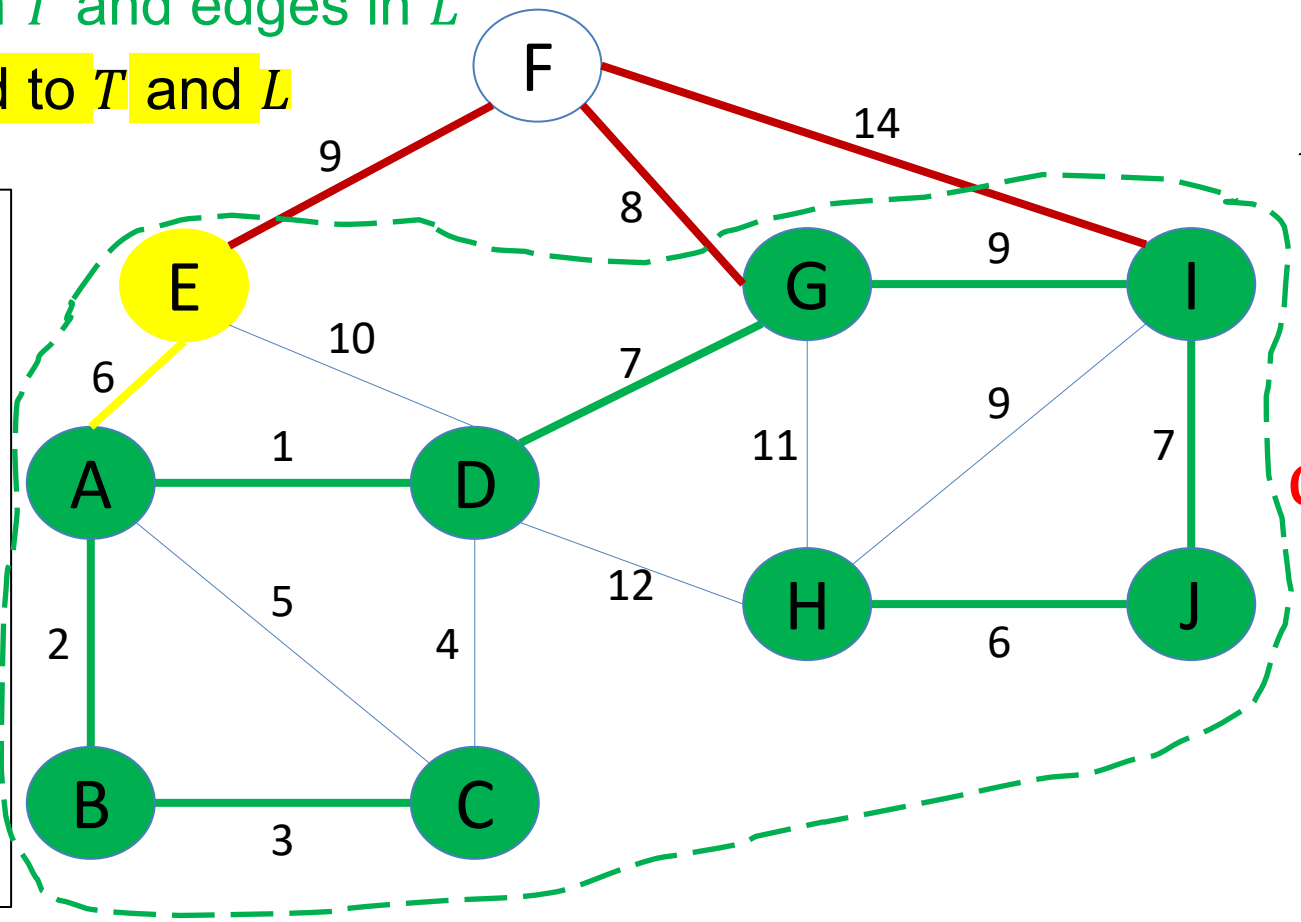
$$\text{cost}(L) = 35$$

Example (10/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, f, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



$$L = \{(h, j), (j, i)$$
$$(i, g), (g, d), (d, a),$$
$$(a, b), (b, c), (a, e)\}$$

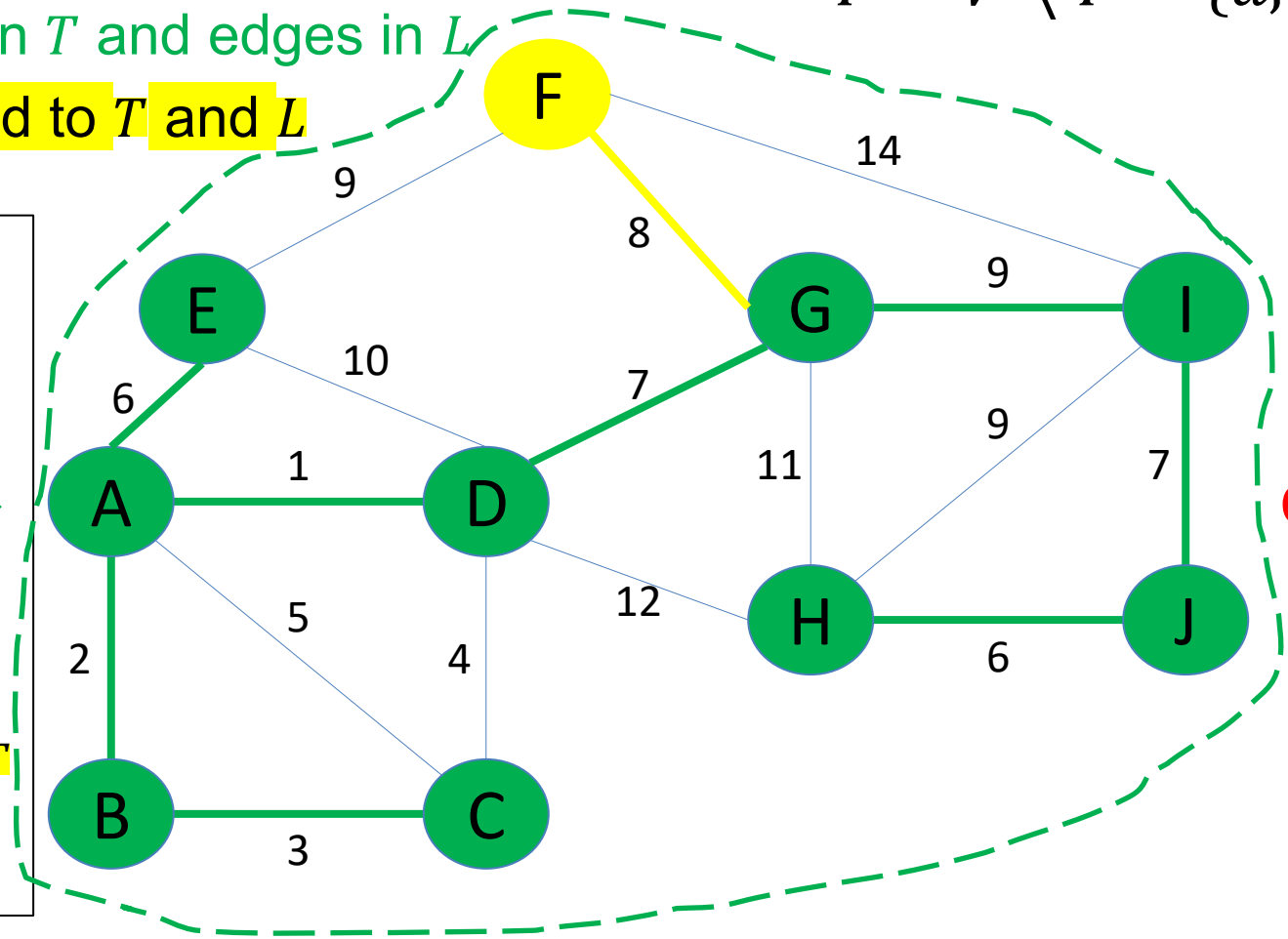
$$\text{cost}(L) = 41$$

Example (11/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e, f\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, \cancel{f}, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ ✓ vertices
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



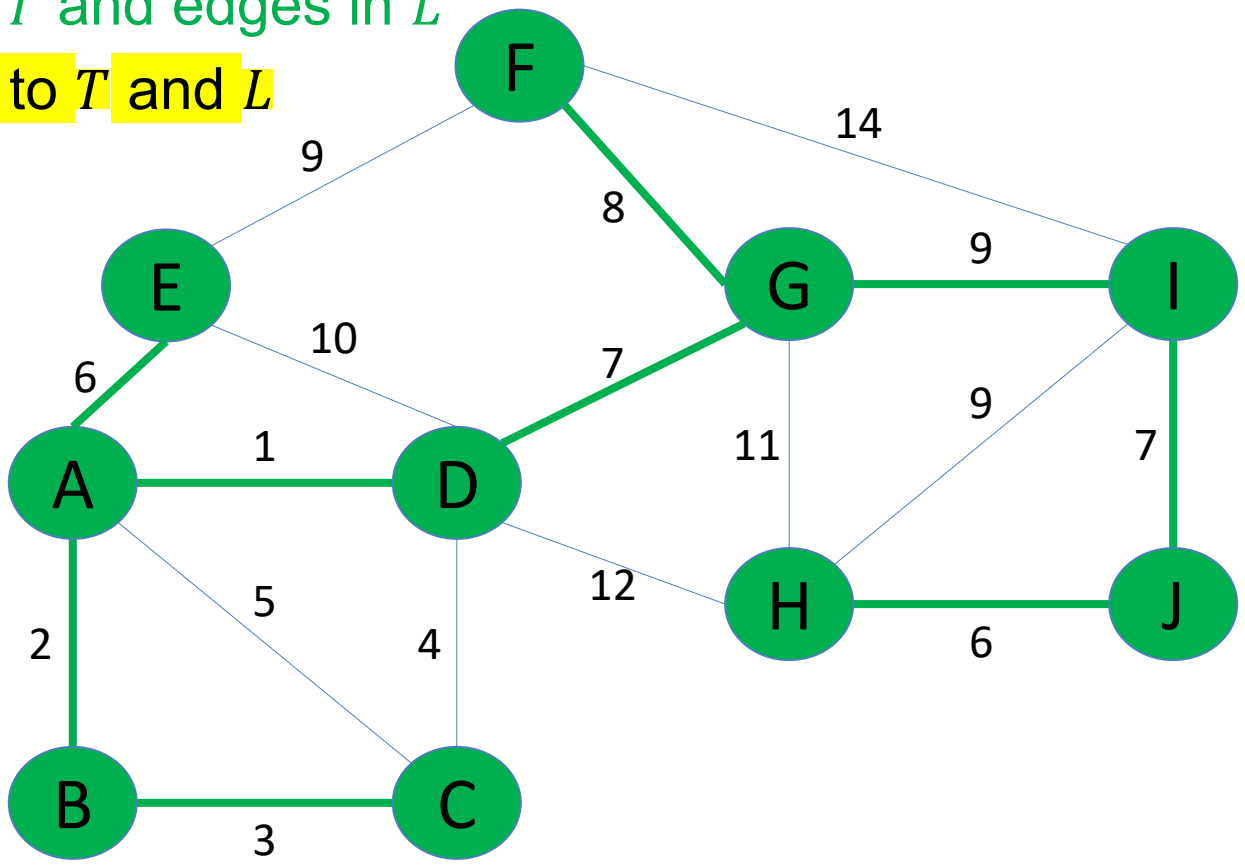
$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c), (a, e), (g, f)\}$$
$$\text{cost}(L) = 49$$

Example (11/11)

- **Red:** edges from vertices $\in T$ to vertices $\notin T$, i.e., $\in \bar{T} = V \setminus T$
- **Green**: vertices in T and edges in L
- Yellow: just added to T and L

$$T = \{h, j, i, g, d, a, b, c, e, f\}$$
$$\bar{T} = V \setminus T = \{\cancel{a}, \cancel{b}, \cancel{c}, \cancel{d}, \cancel{e}, \cancel{f}, \cancel{g}, \cancel{i}, \cancel{j}, \cancel{h}\}$$

PrimMST(V, E)
1. Initialize two empty sets T and L
2. Add an arbitrary vertex r from V to T
3. While T contains < $|V|$ vertices ✗
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add vertex v to T and (u, v) *to* L



$$L = \{(h, j), (j, i), (i, g), (g, d), (d, a), (a, b), (b, c), (a, e), (g, f)\}$$
$$\text{cost}(L) = 49$$

Minimum Cost:

49

Implementation (2/3): adding a priority queue

- What should we store in the priority queue?
  - Edges
  - From nodes $\in T$ to nodes $\notin T$

- What should we use as the key of an edge?
  - Weight of the edge

PrimMST(V, E)
1. Pick an arbitrary node r from V
2. Add r to *T*
3. While *T* contains < |V| nodes
4. Find a **minimum weight edge** (u, v) where $u \in T$ and $v \notin T$
5. Add node v to *T*

Finding lots of minimums?
Use a priority queue!

Implementation (3/3): adding a priority queue

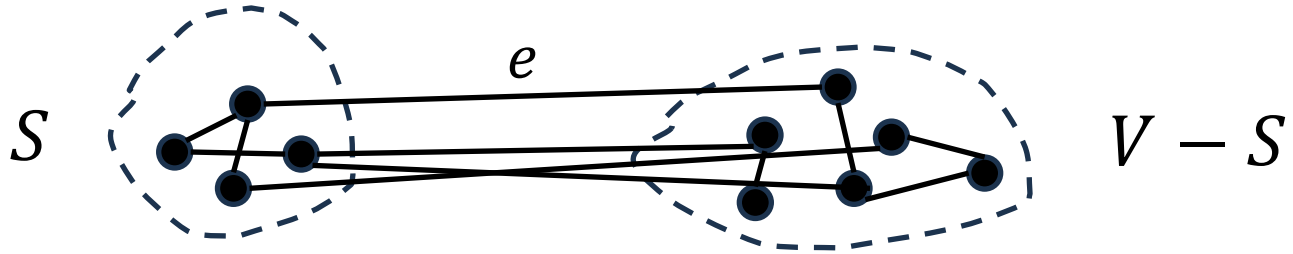
where r is any arbitrary starting node

```
PrimMST(G = (V, E), start r):
    inTree[r] := true
    parent[r] := r
    Q := min-heap priority queue of edges from r

    while Q not empty:
        (w, u, v) := Q.extractMin()
        if not inTree[v]:
            inTree[v] := true
            parent[v] := u
            for each edge (v, z):
                if not inTree[z]:
                    Q.insert((weight(v, z), v, z))

    return parent
```

Correctness (1/4): Cut Property (MST Lemma)

Lemma 1. Given a connected, weighted, undirected graph $G = (V, E)$ and a cut $(S, V - S)$ (a partition of the vertex set V into two non-empty sets S and $V - S$). Let $e = (u, v)$ be an edge crossing the cut (i.e., $u \in S$ and $v \in V - S$). Then

1. If e has a strictly minimum weight among all crossing edges, then e must be part of every Minimum Spanning Tree (MST) of G.
2. If there are multiple edges with the same minimum weight crossing the cut, then at least one of these minimum-weight crossing edges must be in some MST of G.



Part 1 (1/2)

- Assume, for the sake of contradiction, that there exists an MST, let's call it M^*, that **does not** contain the edge $e = (u, v)$, where e is the unique*minimum-weight edge crossing the cut $(S, V - S)$.
- Since M^* is an MST and $e \notin M^*$, but G is connected, M^* must still connect the two sets S and V-S. This implies that there must be at least one path in M^* from u to v.
- Consider the path in M^* connecting u to v. Since $u \in S$ and $v \in V - S$, this path must cross the cut $(S, V - S)$ at least once. Let (x, y) be any edge on this path that crosses the cut (i.e., $x \in S$ and $y \in V - S$, or vice-versa).
- By our assumption, $e = (u, v)$ is the unique minimum-weight edge crossing the cut. Therefore, the weight of e must be strictly less than the weight of (x, y), i.e., $w(e) < w(x, y)$.

Part 1 (2/2)

- Now, let's form a new spanning tree, M', from M^*. We do this by adding e to M^* and removing (x, y) from M^*.
- Adding e to M^* creates a cycle (because M^* already contains a path between u and v).
- Since (x, y) is an edge on this cycle and also crosses the cut, removing (x, y) from this cycle breaks it and maintains connectivity (it simply reroutes the connection between S and V-S through e instead of (x, y)). Thus, M' is a spanning tree.
- Let's compare the total weight of M' with M^*:

$$w(M') = w(M^*) - w(x, y) + w(e).$$

- Since we established that $w(e) < w(x, y)$, it follows that:

$$w(M') < w(M^*).$$

- This is a contradiction! We assumed M^* was an MST, meaning it had the minimum possible total weight. But we found a spanning tree M' with a strictly smaller total weight.
- Therefore, our initial assumption that M^* does not contain e $must\ be\ false$. Hence, e must be part of every MST.

Part 2 (1/2)

- Assume, for the sake of contradiction, that NO minimum-weight edge crossing the cut $(S, V - S)$ is part of an MST, let's call it M^*.
- Let $e_0 = (u_0, v_0)$ be any minimum-weight edge crossing the cut $(S, V - S)$ (i.e., $u_0 \in S, v_0 \in V - S$).
- By our assumption, $e_0 \notin M^*$.
- Similar to Part 1, since M^* is an MST and $e_0 \notin M^*$, there must be a path in M^* connecting u_0 to v_0. This path must cross the cut $(S, V - S)$ at least once. Let (x, y) be any edge on this path that crosses the cut (i.e., $x \in S, y \in V - S$, or vice-versa).
- Since e_0 is a minimum-weight edge crossing the cut, the weight of e_0 must be less than or equal to the weight of any other edge crossing the cut, including (x, y):

$$w(e_0) \leq w(x, y).$$

Part 2 (2/2)

- Let's form a new spanning tree, M', from M^*. We add e_0 to M^* and remove (x, y) from M^*.
- As before, adding e_0 to M^* creates a cycle, and removing (x, y) (which is on this cycle and crosses the cut) results in a valid spanning tree M'.
- Compare the total weights:

$$w(M') = w(M^*) - w(x, y) + w(e_0).$$

- Since $w(e_0) \leq w(x, y)$, it follows that $w(M') \leq w(M^*)$.
- As M^* is an MST, it has the minimum possible weight. Therefore, $w(M')$ cannot be strictly less than $w(M^*)$. This implies:

$$w(M') = w(M^*).$$

- This means M' is also an MST. Crucially, M' contains the edge e_0, which is a minimum-weight crossing edge. This contradicts our initial assumption that no minimum-weight edge crossing the cut is part of M^*.

Correctness (2/4)

- First, we have to realize that Prim's algorithm is greedy.
- This is because at each step, it always tries to select the next valid edge e with MINIMUM weight (greedy!)
- The greedy algorithm is usually simple to implement.

Correctness (3/4)

We prove the correctness of the Prim's algorithm by induction on the vertex set V.

- Base case: Prim's algorithm is correct for $|V| = 2,3$.
- Inductive Hypothesis: Assume that Prim's algorithm is correct for $|V| = n$.
- Inductive Step: We now prove that Prim's algorithm is also correct for $|V| = n + 1$.
- Indeed, consider a cut $(S, V - S)$ where $|S| = n$, i.e., $|V - S| = 1$. Let $u \in V - S$ and $F = \{e_1, \dots, e_k\}$ be the set of edges that has one endpoint as u.
- Let $e \in F$ be an edge such that $w(e) \leq w(e_i), \forall i \in [k] = \{1, \dots, k\}$.

Correctness (4/4)

- By using the Cut Property, the edge e must belong to some MST of $G = (V, E)$.
- For any spanning tree T^* of G, we can decompose T^* into two trees, which are T_1^* and $T_2^* = \{e^*\}$, where T_1^* is a spanning tree of G_S and $e^* \in F$.
- Let T be an MST of $G_S = \big(S, E(S)\big)$ be an induced graph of G.
- Then we have

$$w(T) + e \leq w(T_1^*) + w(e^*).$$

- Moreover, since one of the endpoint of the edge e belongs to T, the tree $T \cup \{e\}$ is an MST of G.

Complexity: analysis of running time

```
PrimMST(G = (V, E), start r):
    inTree[r] := true
    parent[r] := r
    Q := min-heap priority queue of edges from r

    while Q not empty:
        (w, u, v) := Q.extractMin()
        if not inTree[v]:
            inTree[v] := true
            parent[v] := u
            for each edge (v, z):
                if not inTree[z]:
                    Q.insert((weight(v, z), v, z))
    return parent
```

$O(|V| \log|V|)$

$O(1)$

$O(|E|)$

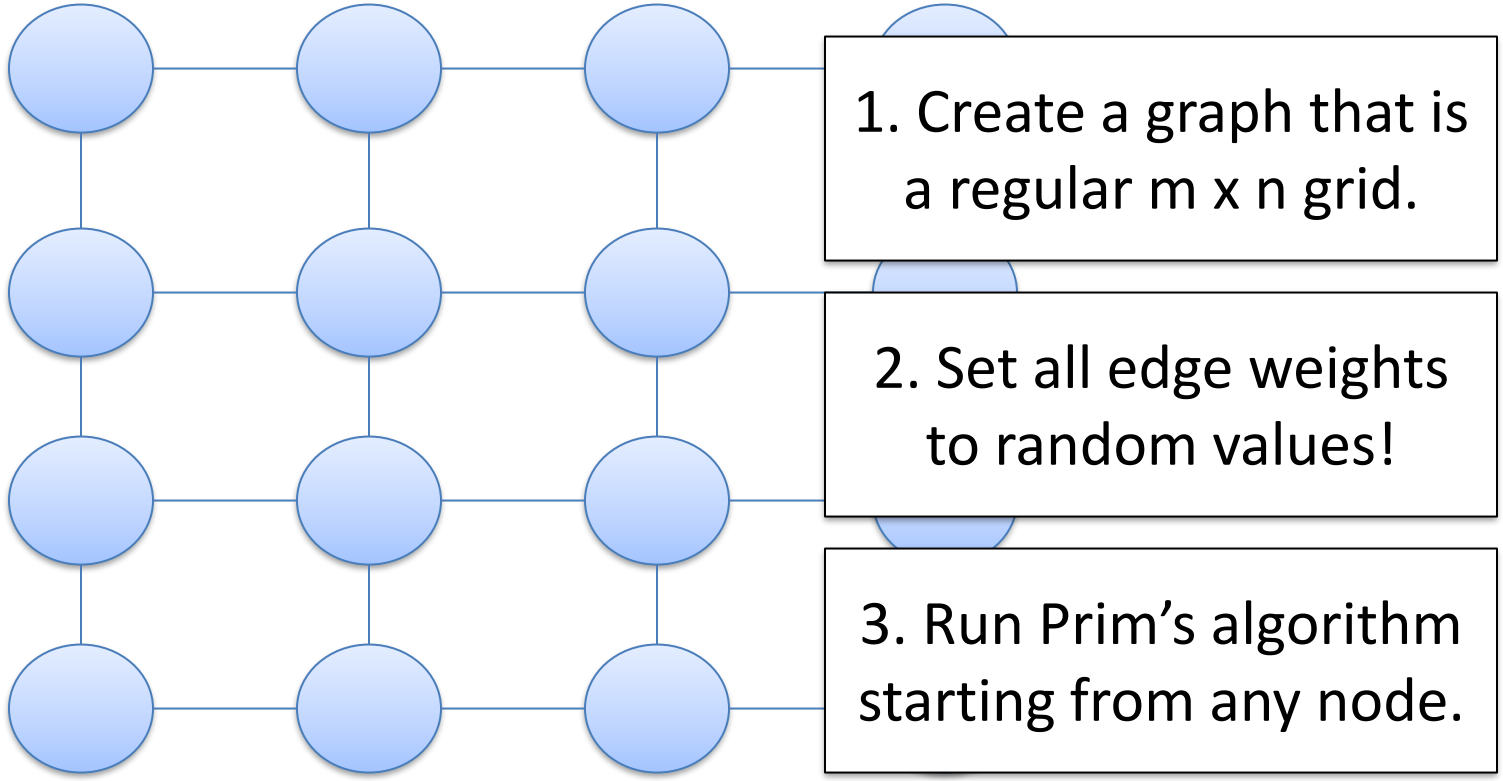
$O(|\text{adj}(v)| \log|V|)$

$O(|E| \log|V|)$

$$O\big((|V| + |E|) \log|V|\big)$$

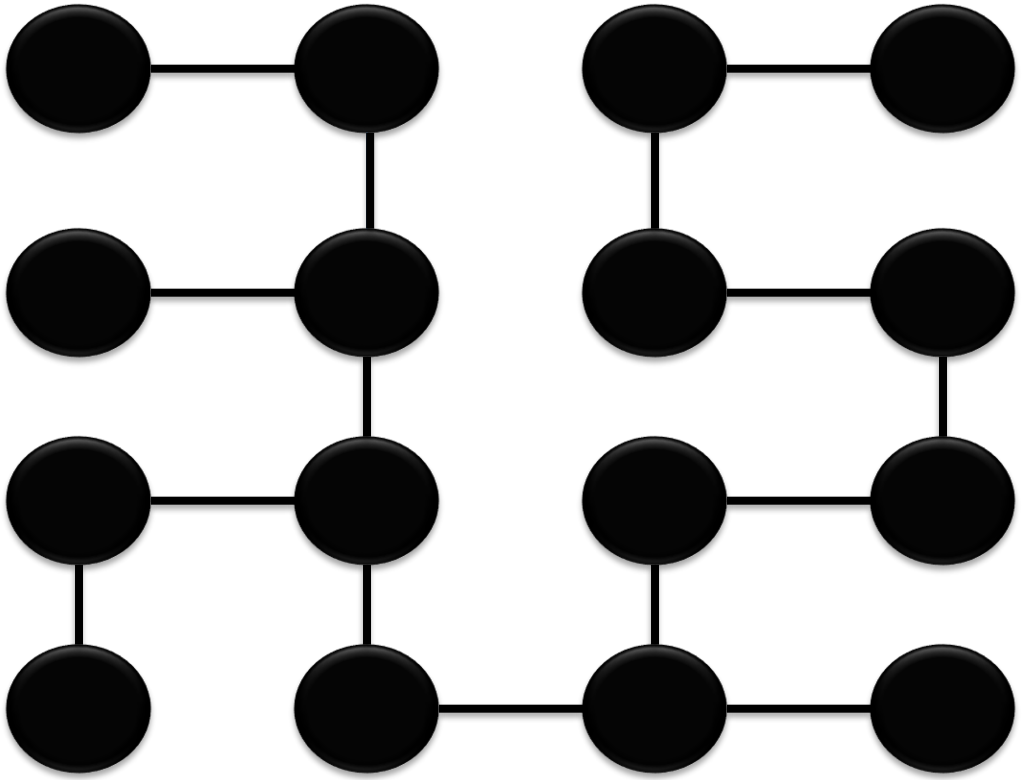
More applications: generating 2D maze (1/2)

- Prim's algorithm maze building video
- How can we use Prim's algorithm to do this?



More applications: generating 2D maze (2/2)

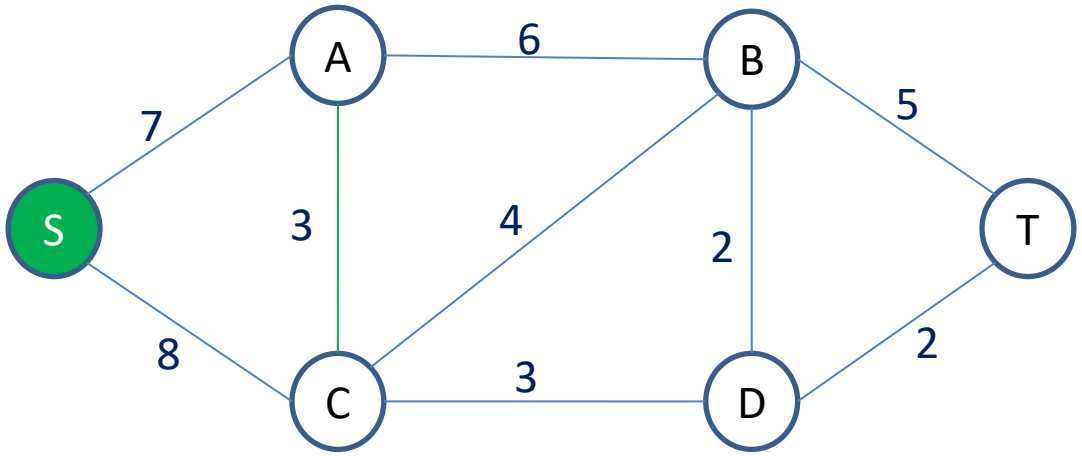
- After Prim's, we end up with something like:



- For any two adjacent vertices that are disconnected, build a wall.

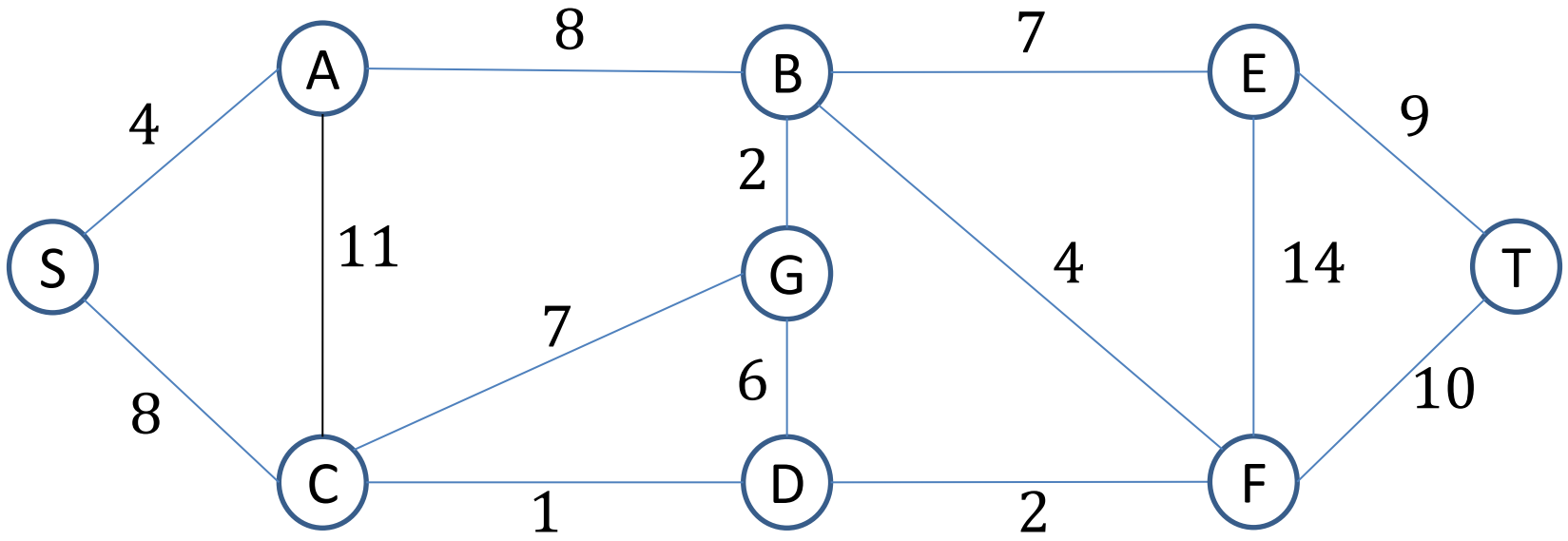
Exercise 1

- Find an MST using Prim's algorithm



Exercise 2

- Find an MST using Prim's algorithm



Outline

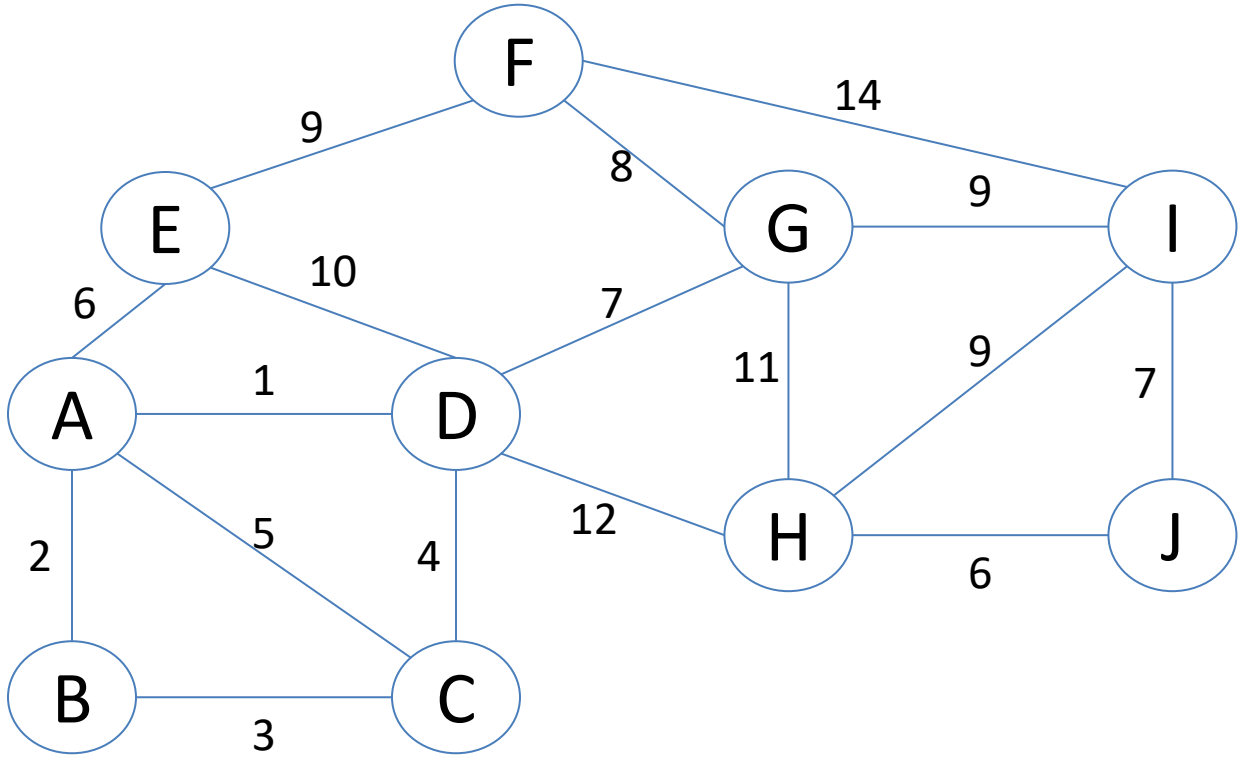
1. Trees
2. Rooted trees
3. Tree traversals
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS)
4. Binary trees
5. Spanning trees
   - Prim's algorithm
   - Kruskal's algorithm

Kruskal's algorithm

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:

   If the edge doesn't form a cycle in the MST, add it to the MST.
4. Stop when the MST contains exactly $(|V| - 1)$ edges.

A Networking Problem



Example (1/16)

Weight	2	5	1	6	3	4	10	7	12	9	8	14	11	9	9	6	7
Edge	(A, B)	(A, C)	(A, D)	(A, E)	(B, C)	(C, D)	(D, E)	(D, G)	(D, H)	(E, F)	(F, G)	(F, I)	(G, H)	(G, I)	(H, I)	(H, J)	(I, J)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

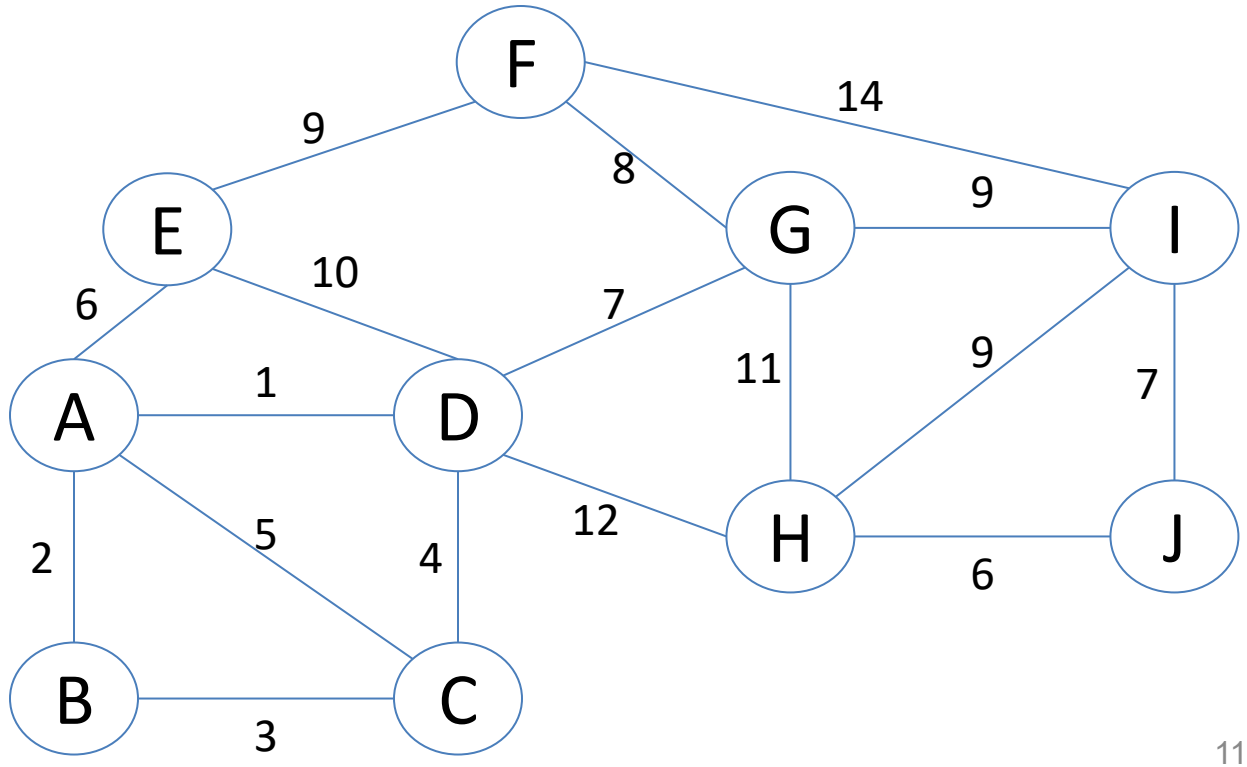
1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

Example (2/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$T = \emptyset$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

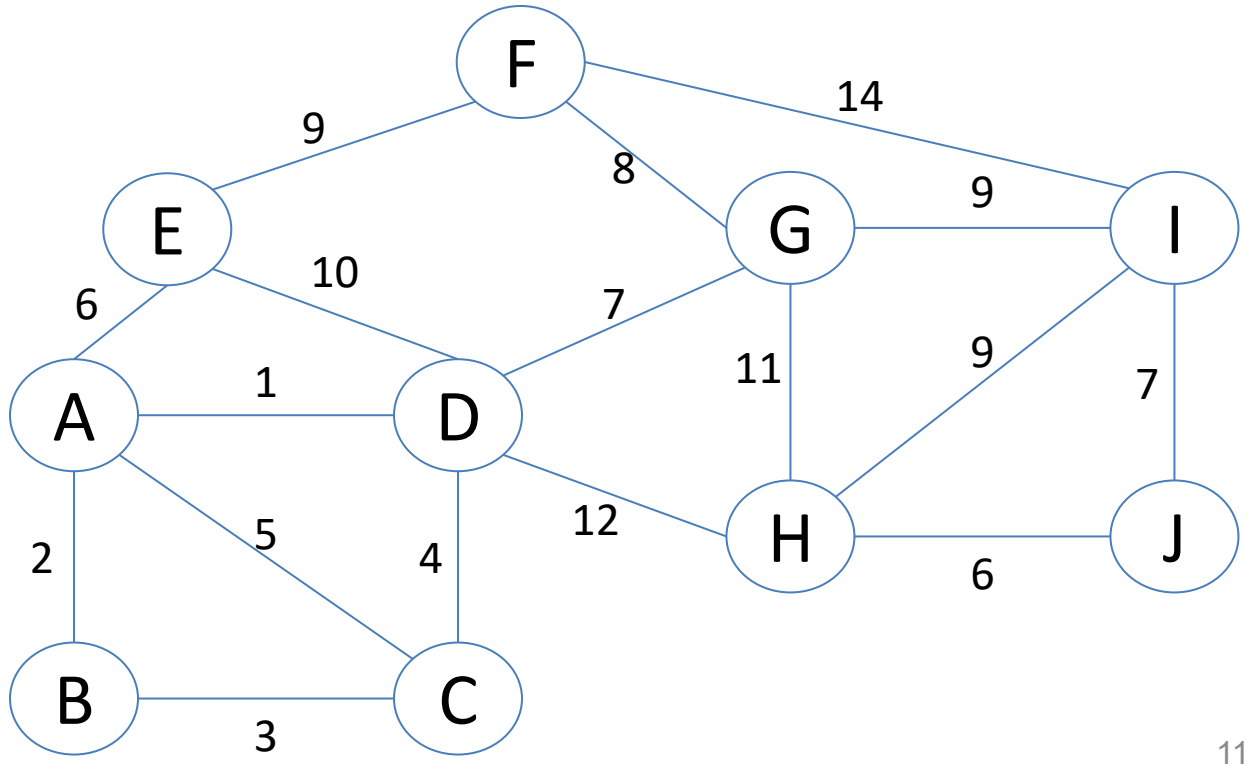


Example (3/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \emptyset, \text{cost}(T) = 0$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

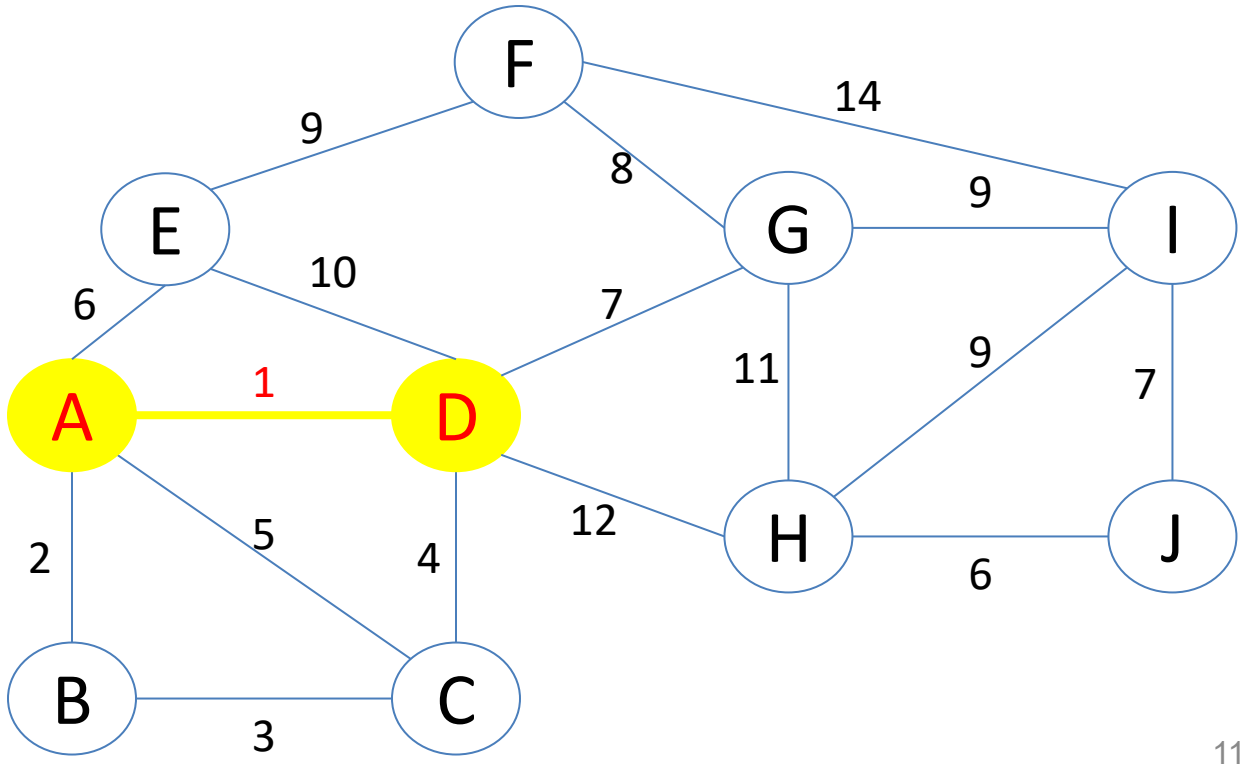


Example (4/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D)\}, \text{cost}(T) = 1$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

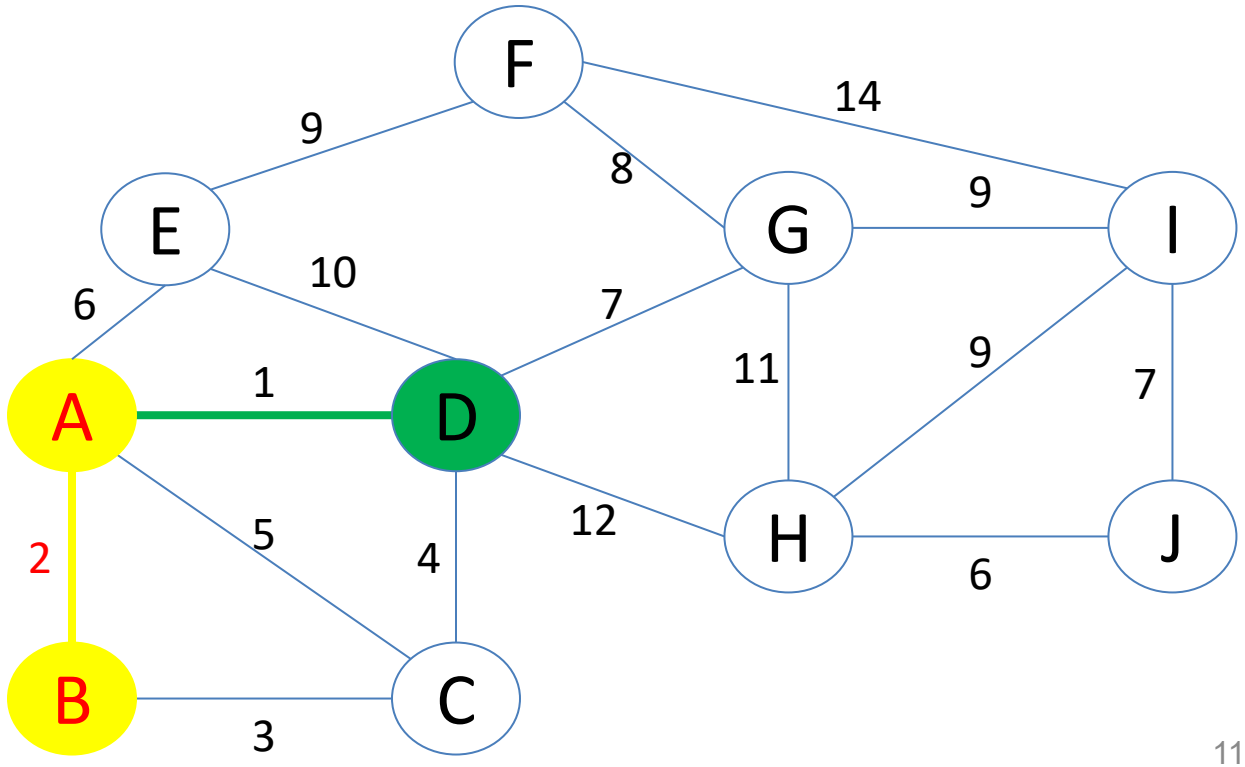


Example (5/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B)\}, \text{cost}(T) = 3$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

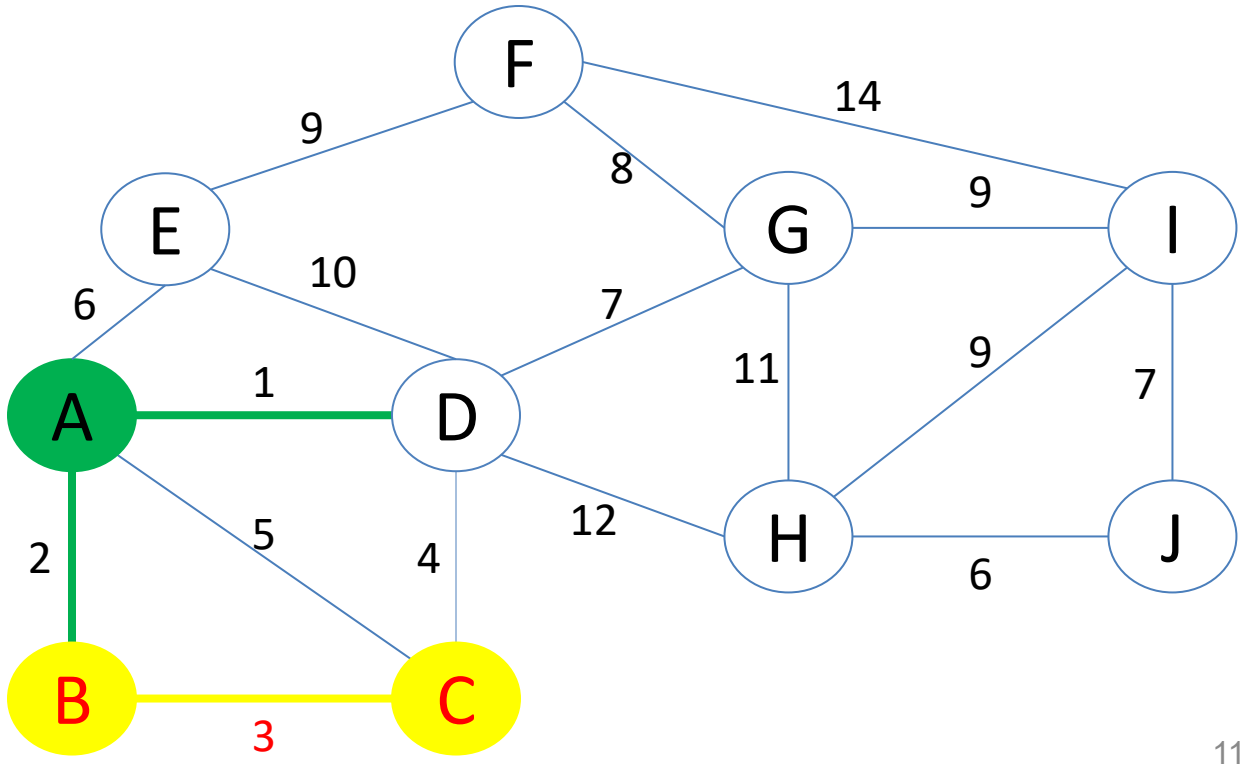


Example (6/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	(C, D)	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

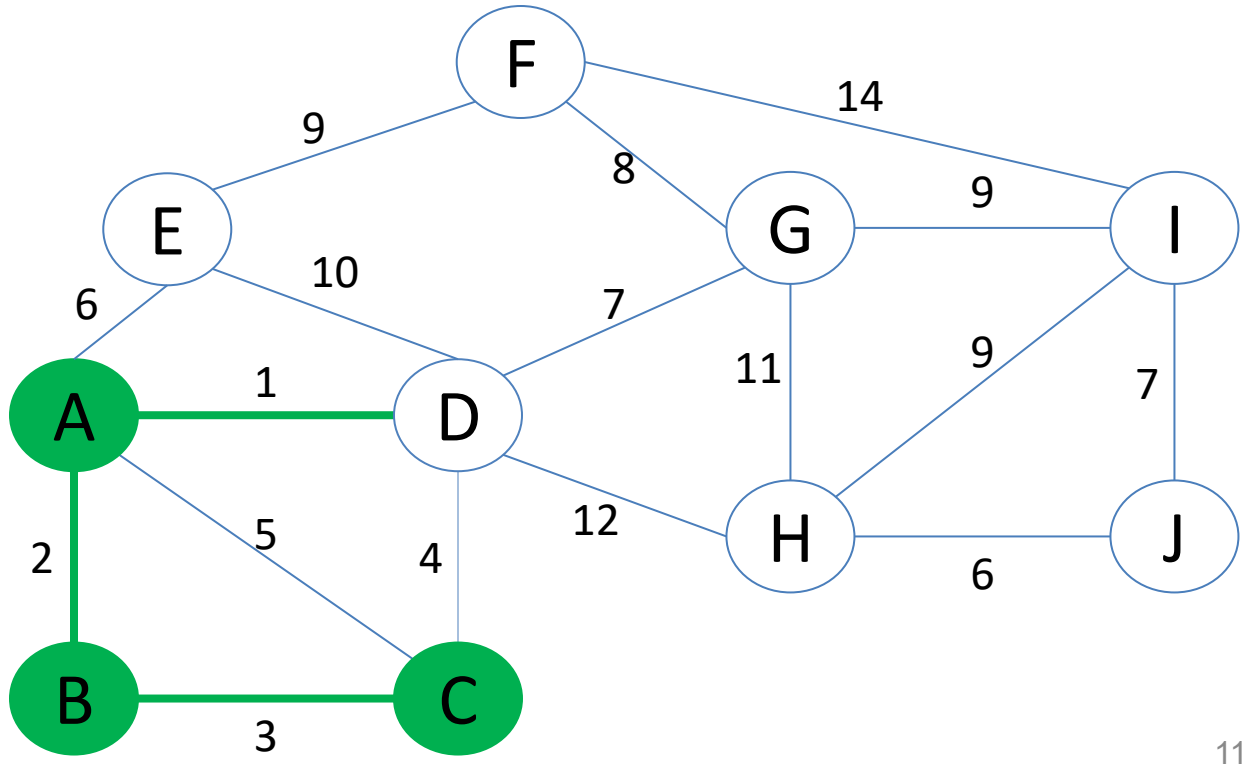


Example (7/16)

Sorted Weight	1	2	3	4	5	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	(A, C)	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

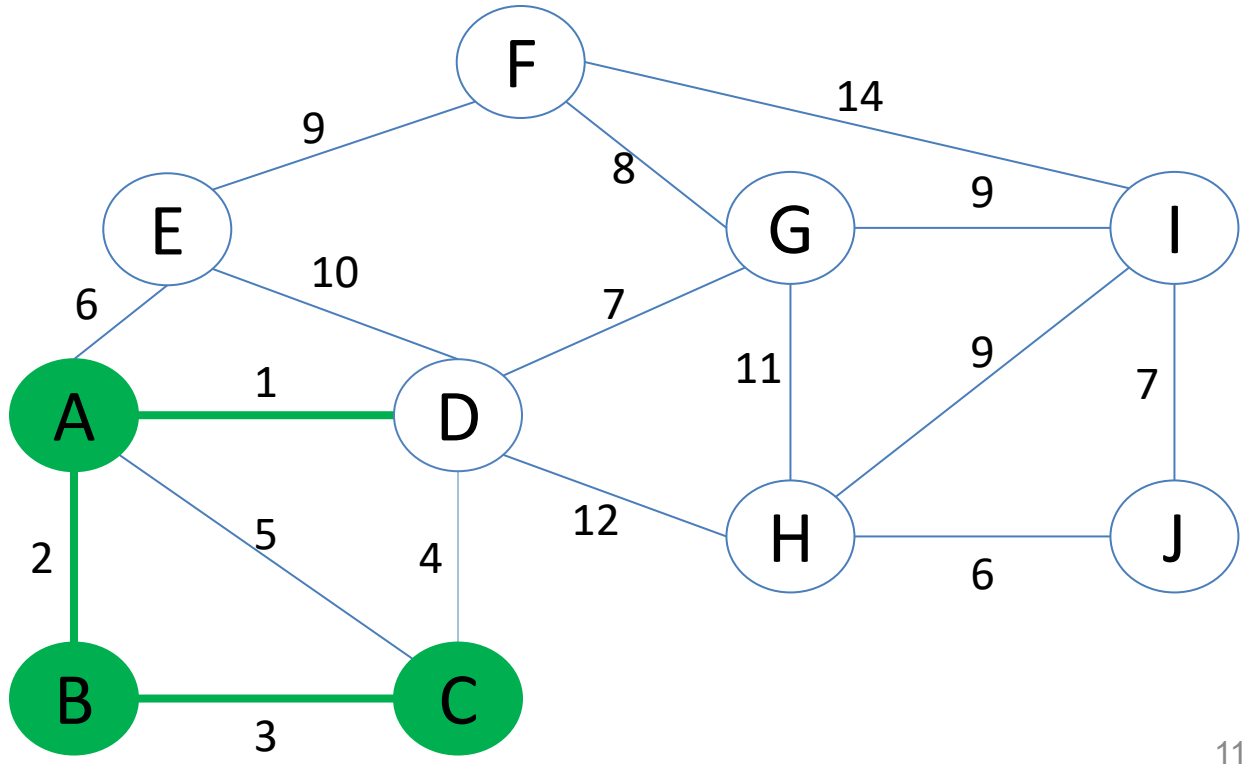


Example (8/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

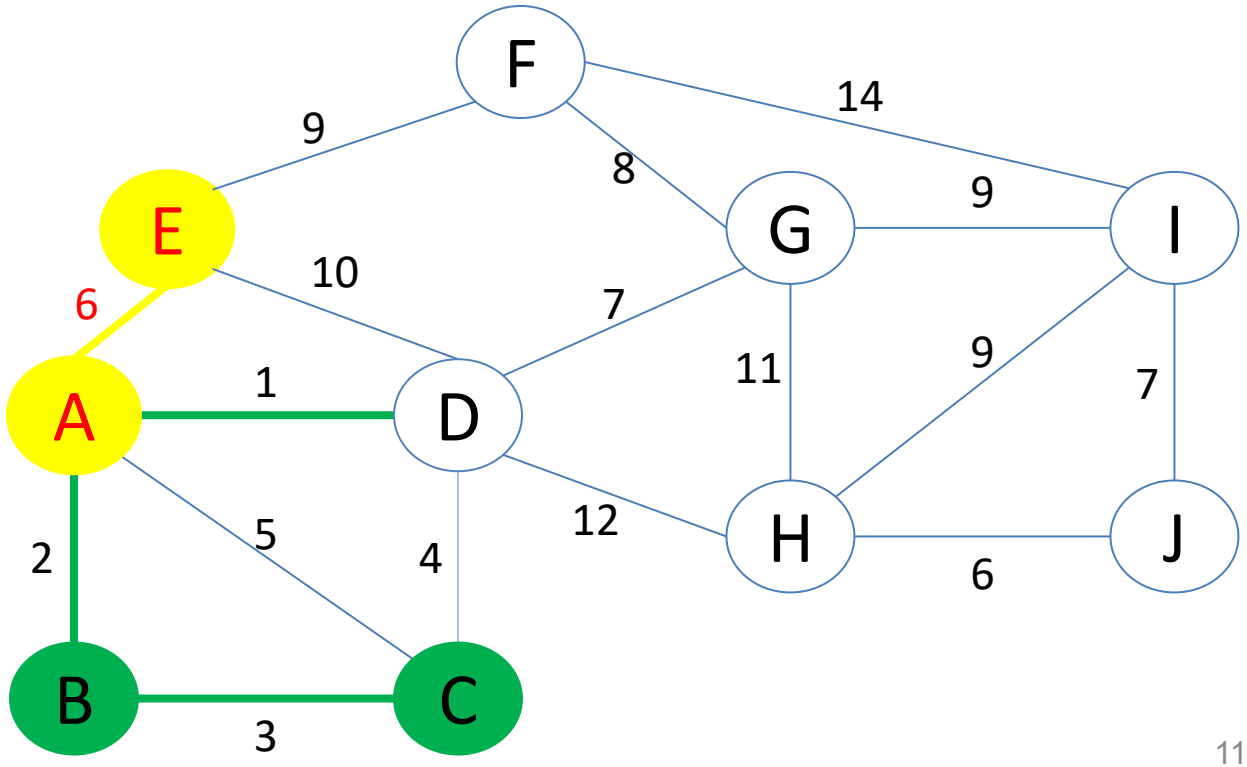


Example (9/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E)\}, \text{cost}(T) = 12$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

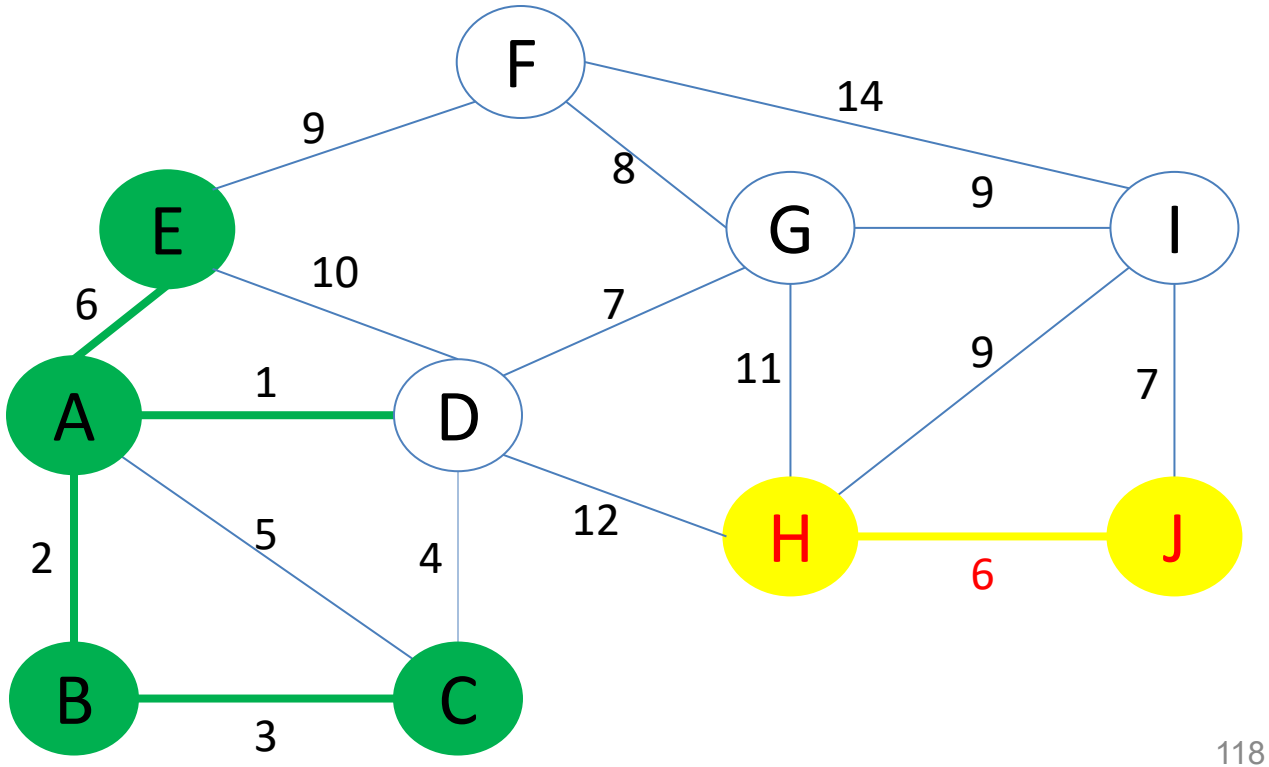


Example (10/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J)\}, \text{cost}(T) = 18$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

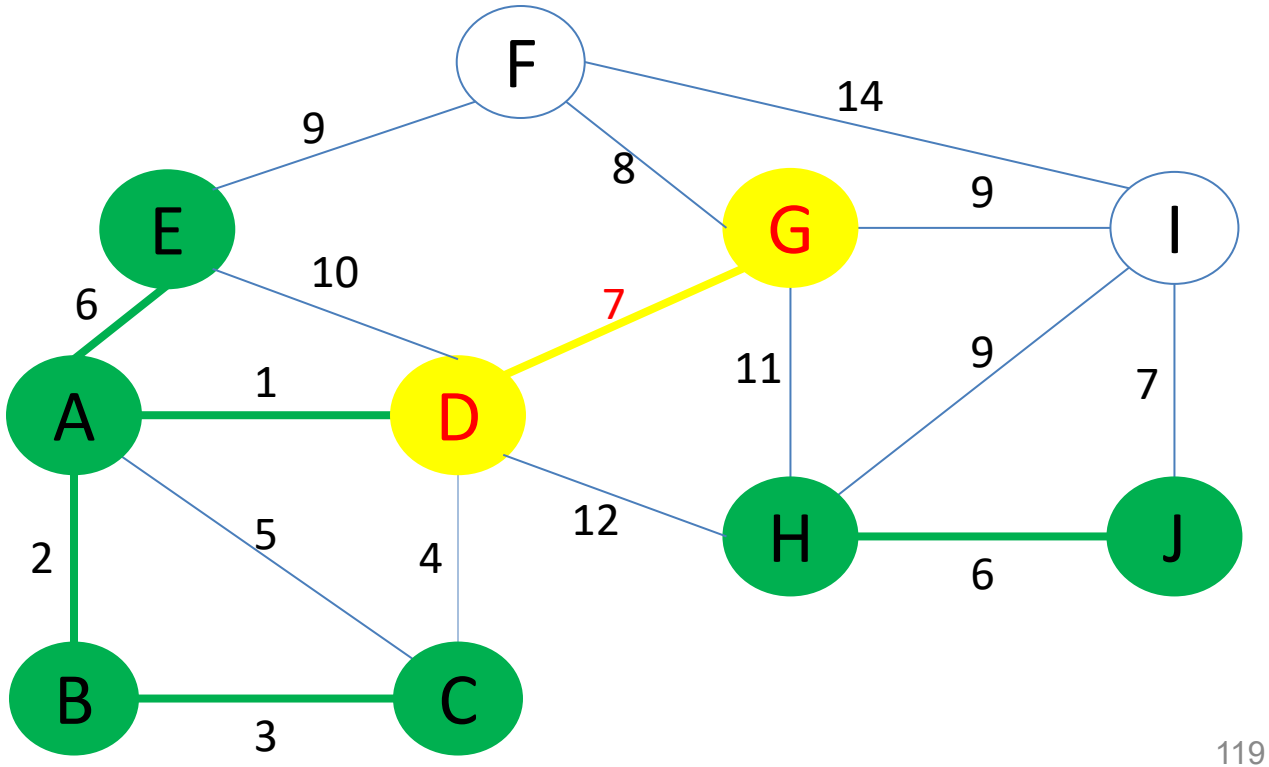


Example (11/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G)\}, \text{cost}(T) = 25$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

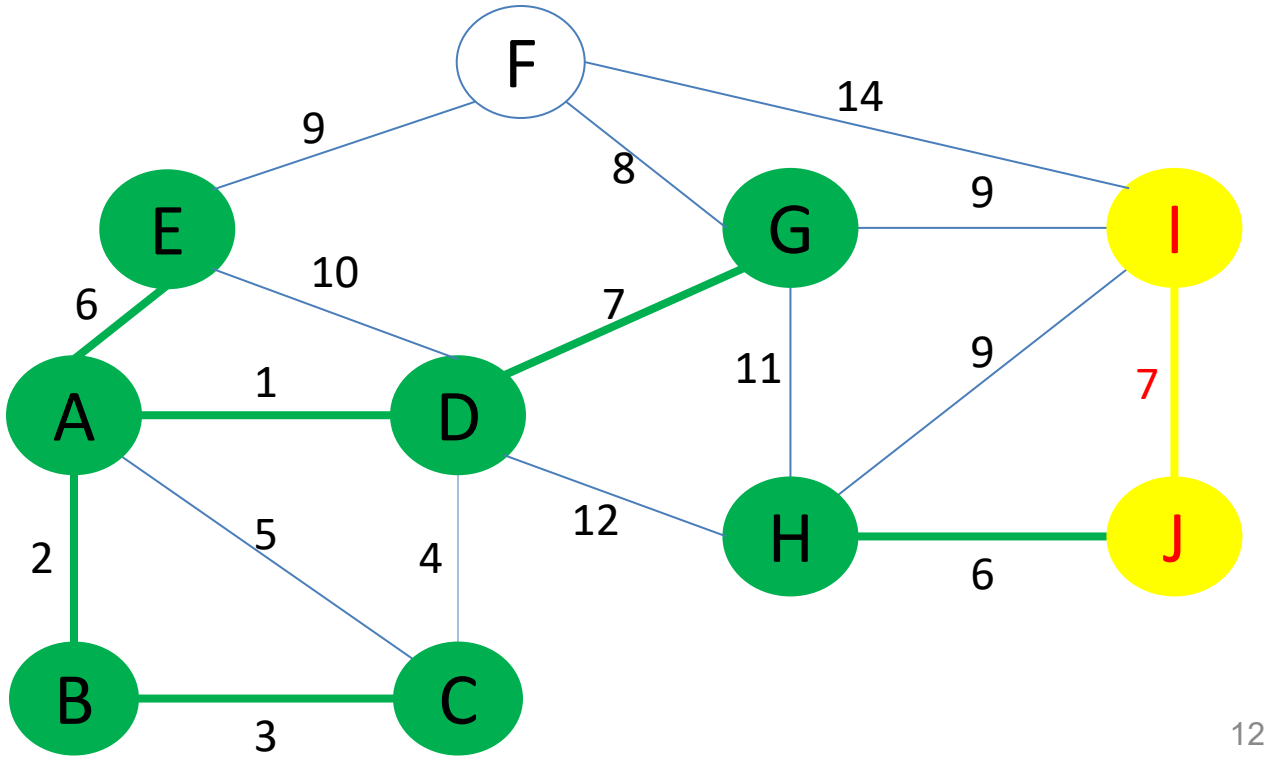


Example (12/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J)\}, \text{cost}(T) = 32$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

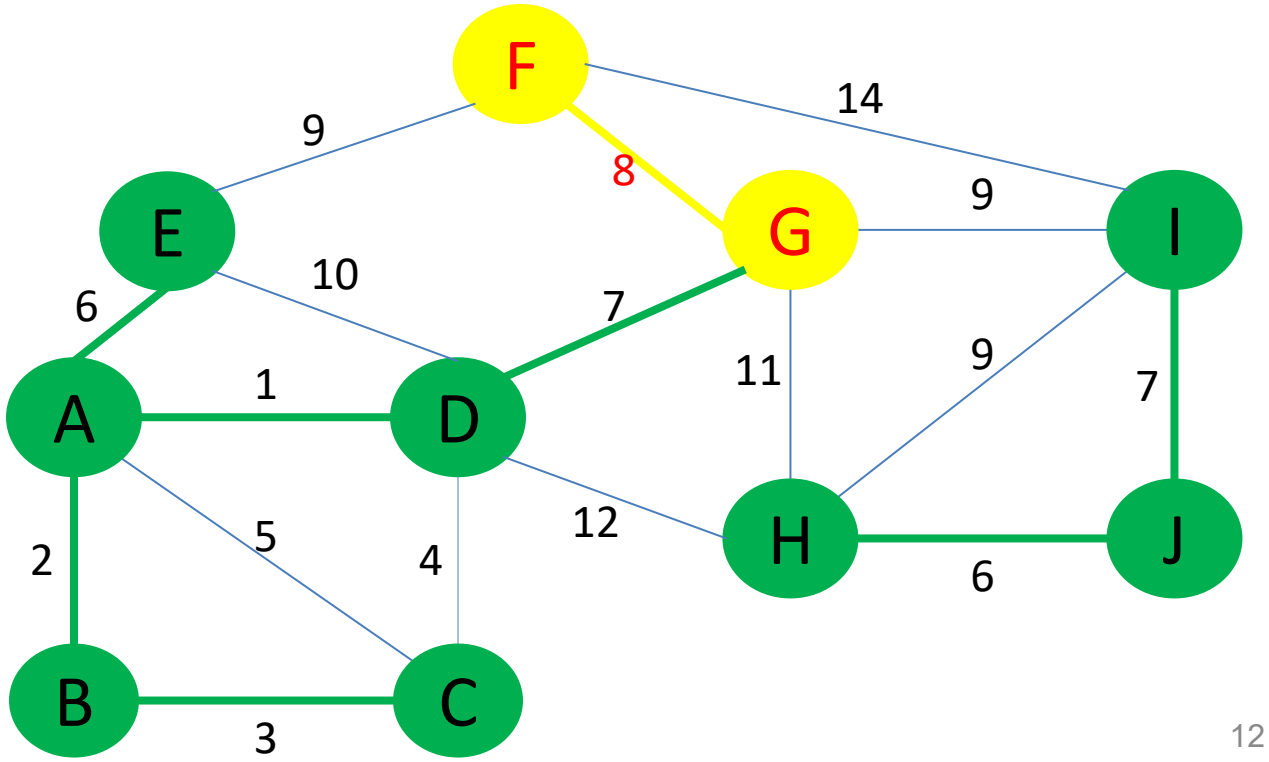


Example (13/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	9	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	(E, F)	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

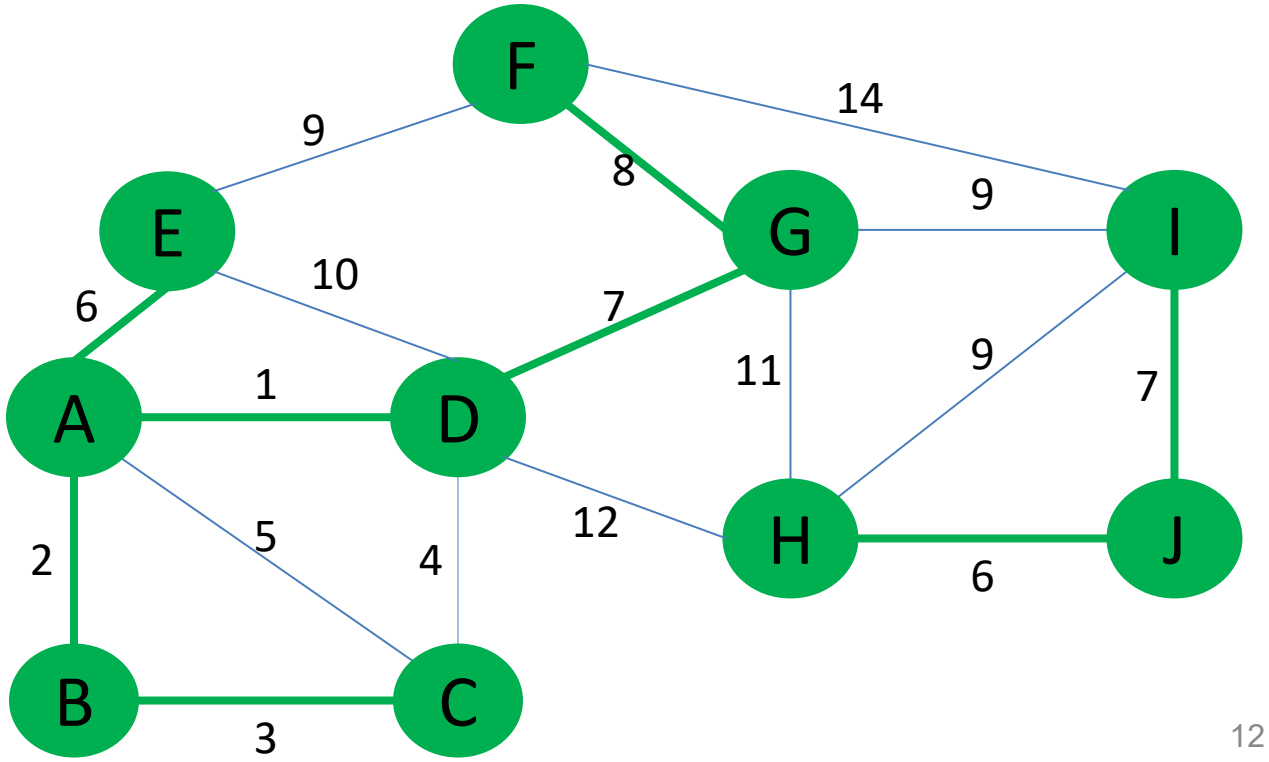


Example (14/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST. ✗
5. Stop when the MST contains exactly $(|V| - 1)$ edges. ✗

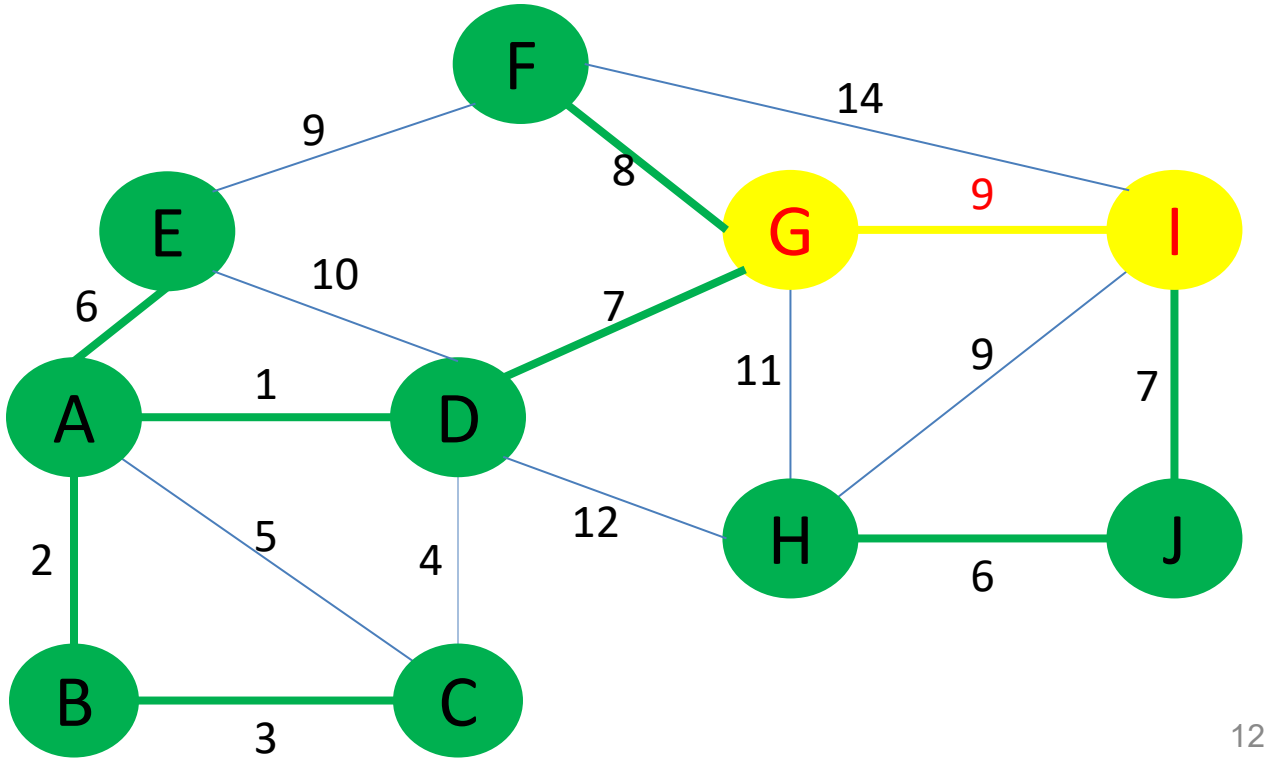


Example (15/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 49$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

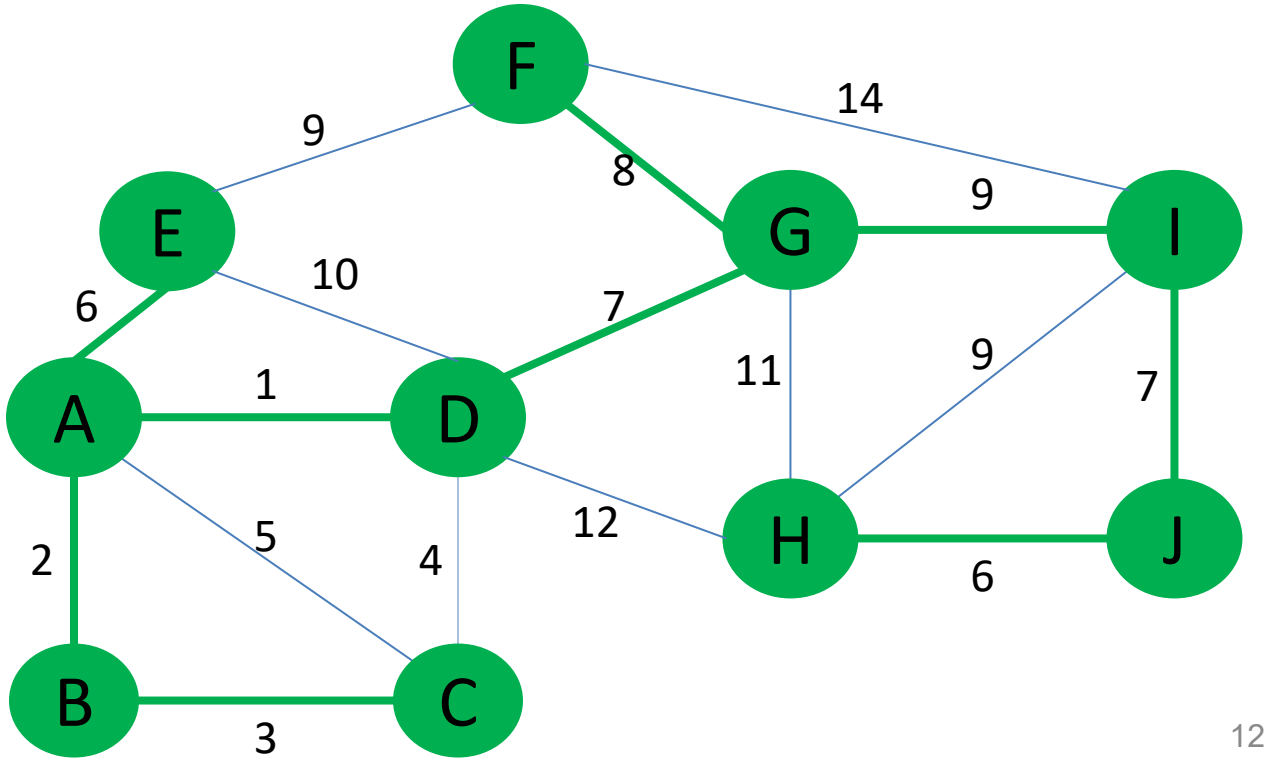


Example (16/16)

Sorted Weight	1	2	3	4	~~5~~	6	6	7	7	8	~~9~~	9	9	10	11	12	14
Sorted Edge	(A, D)	(A, B)	(B, C)	~~(C, D)~~	~~(A, C)~~	(A, E)	(H, J)	(D, G)	(I, J)	(F, G)	~~(E, F)~~	(G, I)	(H, I)	(D, E)	(G, H)	(D, H)	(F, I)

$$T = \{(A, D), (A, B), (B, C), (A, E), (H, J), (D, G), (I, J), (F, G)\}, \text{cost}(T) = 49$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.



Correctness (1/2)

- Kruskal's algorithm is also a greedy algorithm
- Because at each step, it always tries to select the next unprocessed edge e with minimum weight (greedy!).
- Simple proof on how this greedy strategy works
- Loop invariant: Every edge e that is added to T by Kruskal's algorithm is part of the MST.

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

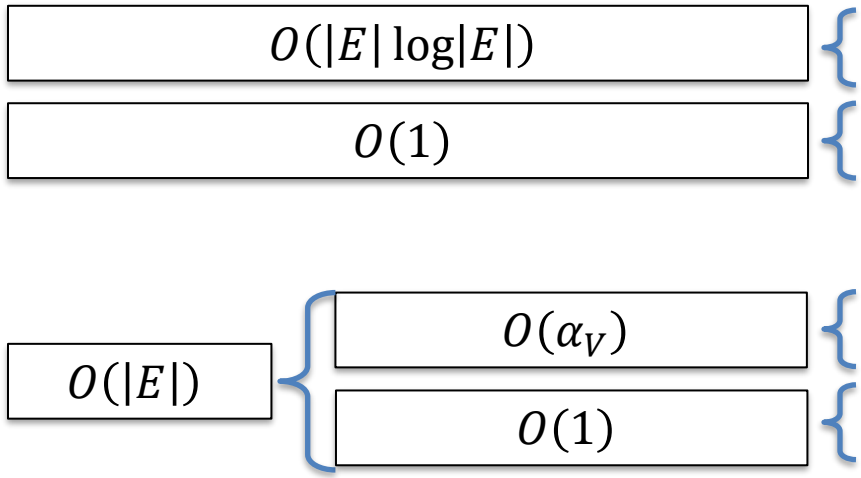
Correctness (2/2)

- Kruskal's algorithm has a special **cycle check** before adding an edge e into T. Edge e will never form a cycle.
- At the start of every loop, T is always part of MST.
- At the end of the loop, we have selected $|V| - 1$ edges from a connected weighted graph G without having any cycles. This implies that we have a Spanning Tree.
- By keep adding the next unprocessed edge e with min cost,

  $$\text{cost}(T \cup \{e\}) \leq \text{cost}(T \cup \{\text{any other unprocessed edge}\} \text{ that does not form a cycle}).$$

- At the start of every loop, T is always part of **MST**.
- At the end of the loop, the Spanning Tree T must have minimum weight $\text{cost}(T)$, so T is the final **MST**.

Complexity: analysis of running time

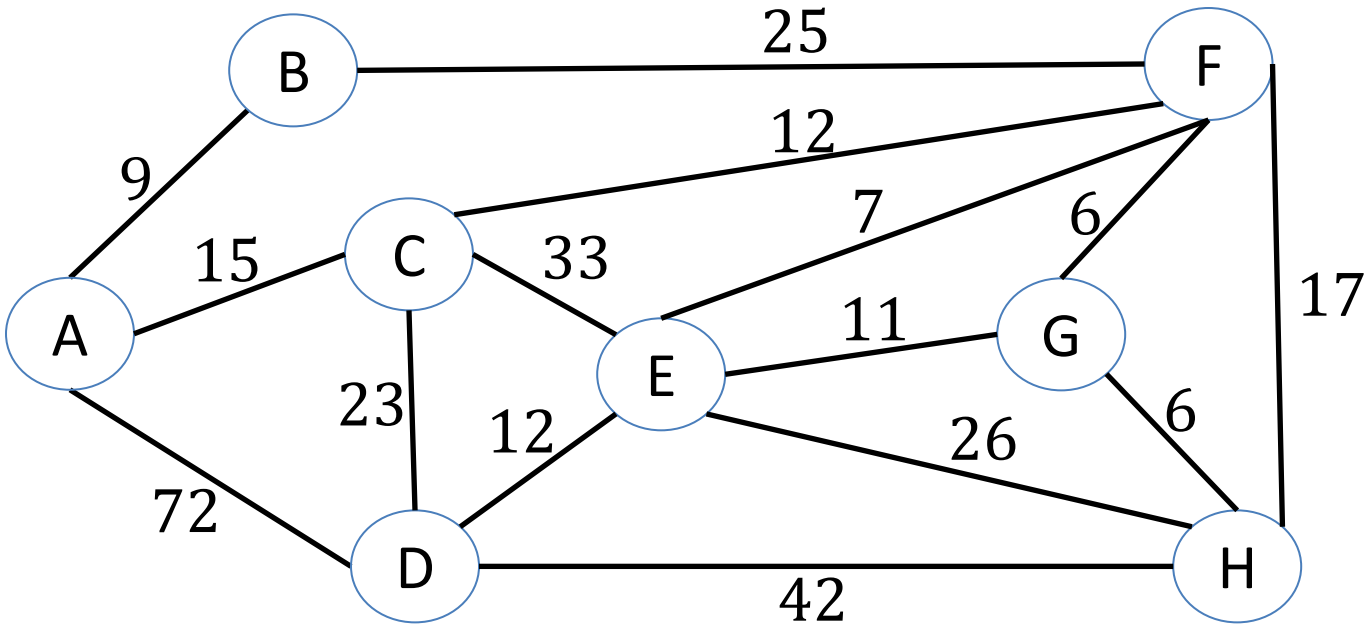


1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

$$O(|E|\log|E|) = O(|E|\log|V|^2) = O(|E|\log|V|)$$

Exercise: A Networking Problem

The vertices represent 8 regional data centers that need to be connected with high-speed data lines. Feasibility studies show that the links illustrated above are possible, and the cost in millions of dollars is displayed next to the link. Which links should be constructed to enable full communication (with relays allowed) and keep the total cost minimal?



find a minimum (cost) spanning tree

Example (1/12)

Weight	9	15	72	25	33	23	12	12	42	7	11	26	6	17	6
Edge	(A, B)	(A, C)	(A, D)	(B, F)	(C, E)	(C,D)	(C, F)	(D, E)	(D, H)	(E, F)	(E, G)	(E, H)	(F, G)	(F, H)	(G, H)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

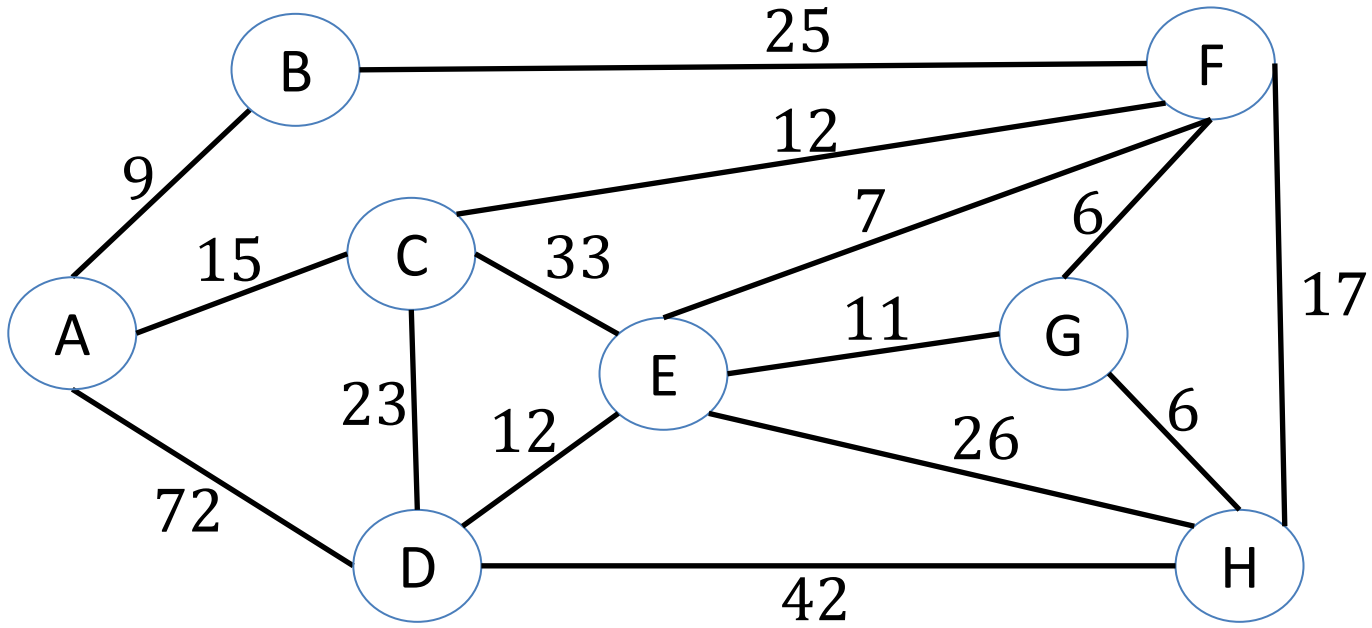
1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

Example (2/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$T = \emptyset$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

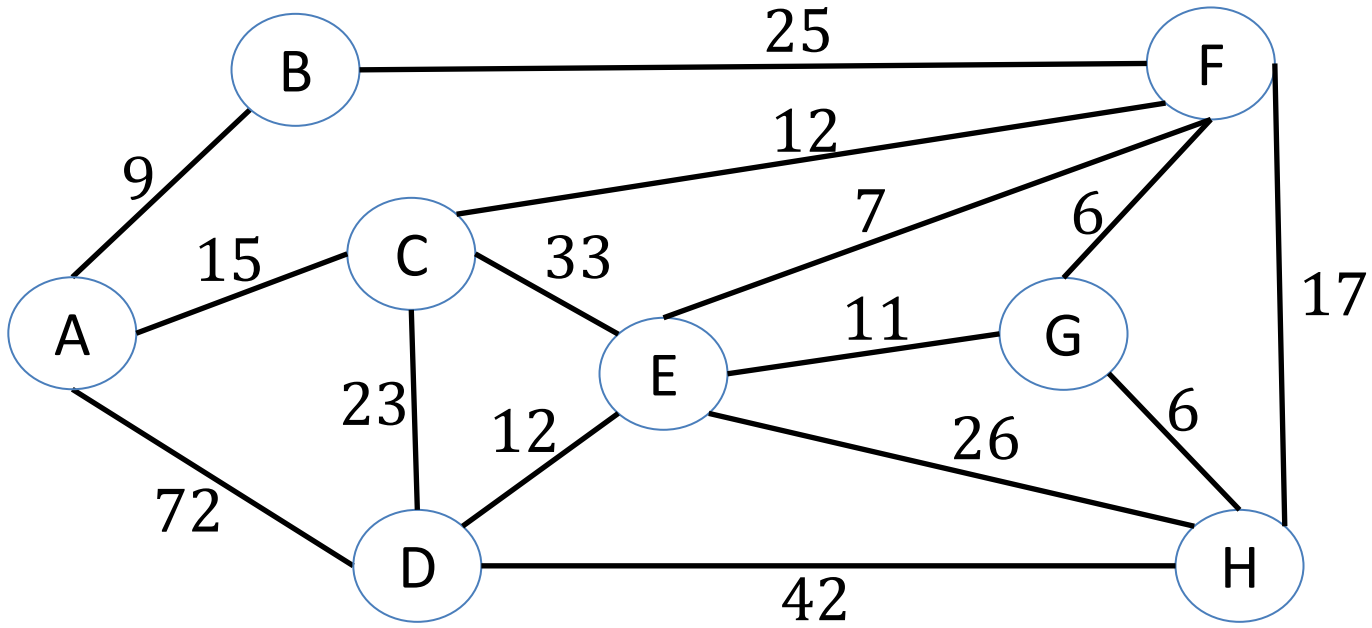


Example (3/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \emptyset, \text{cost}(T) = 0$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

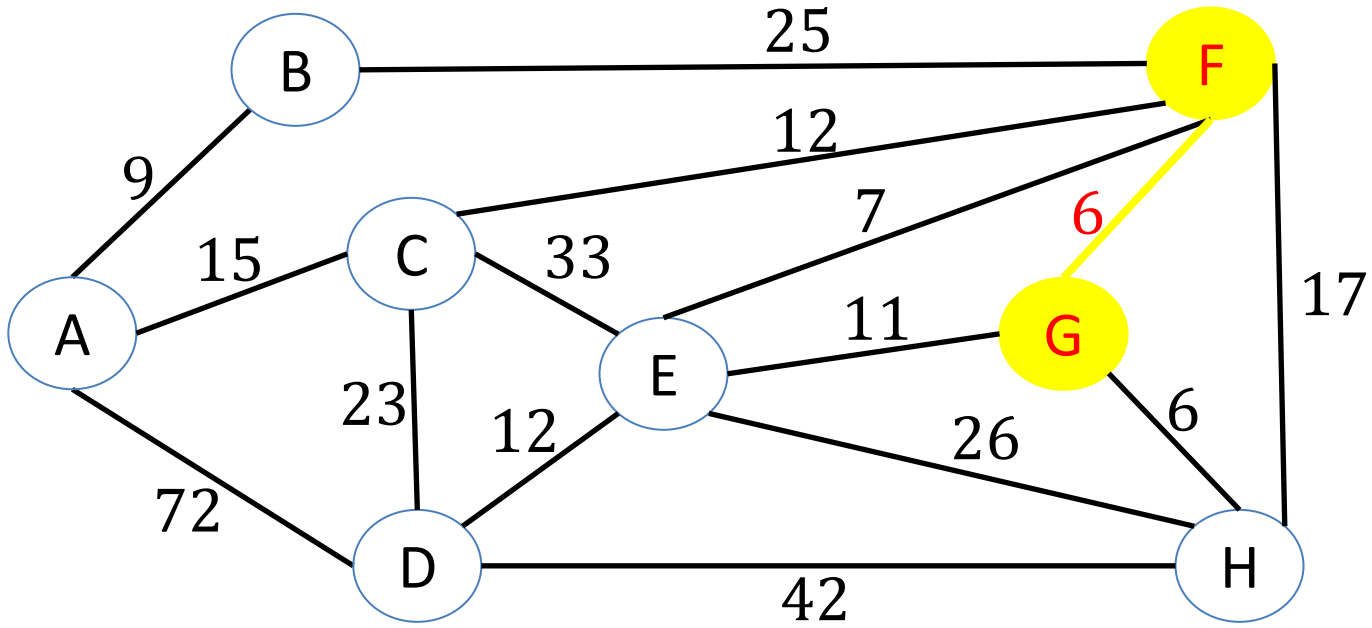


Example (4/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G)\}, \text{cost}(T) = 6$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

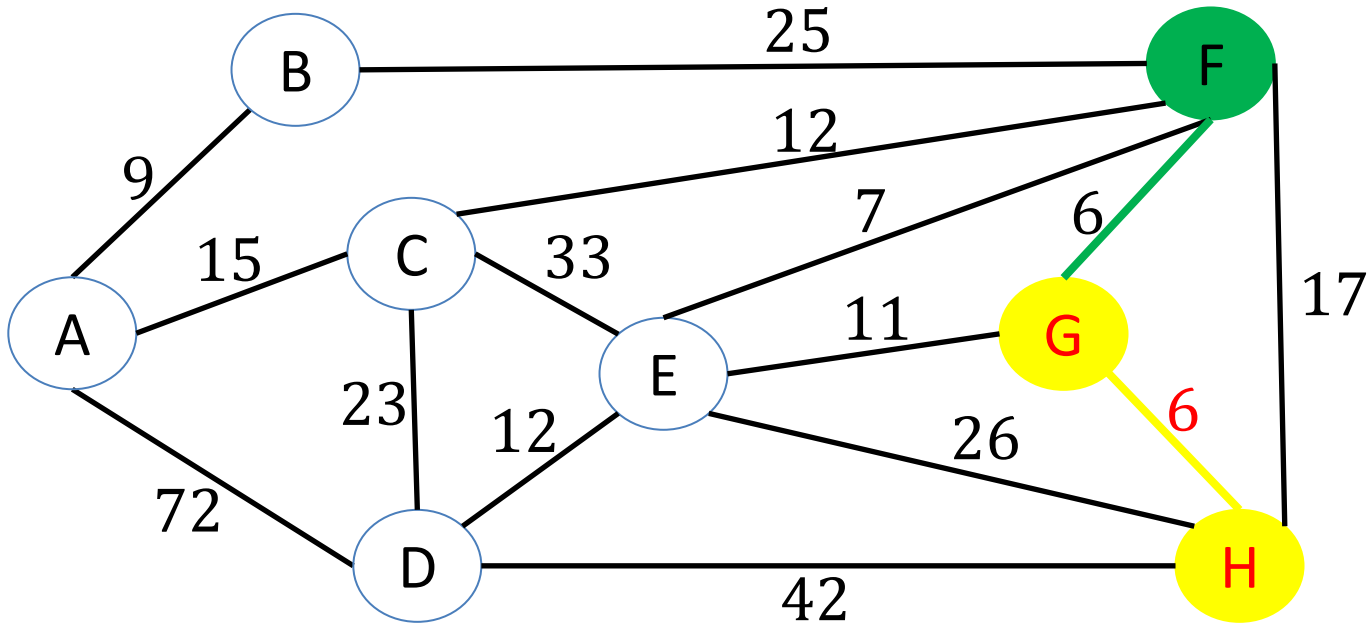


Example (5/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H)\}, \text{cost}(T) = 12$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

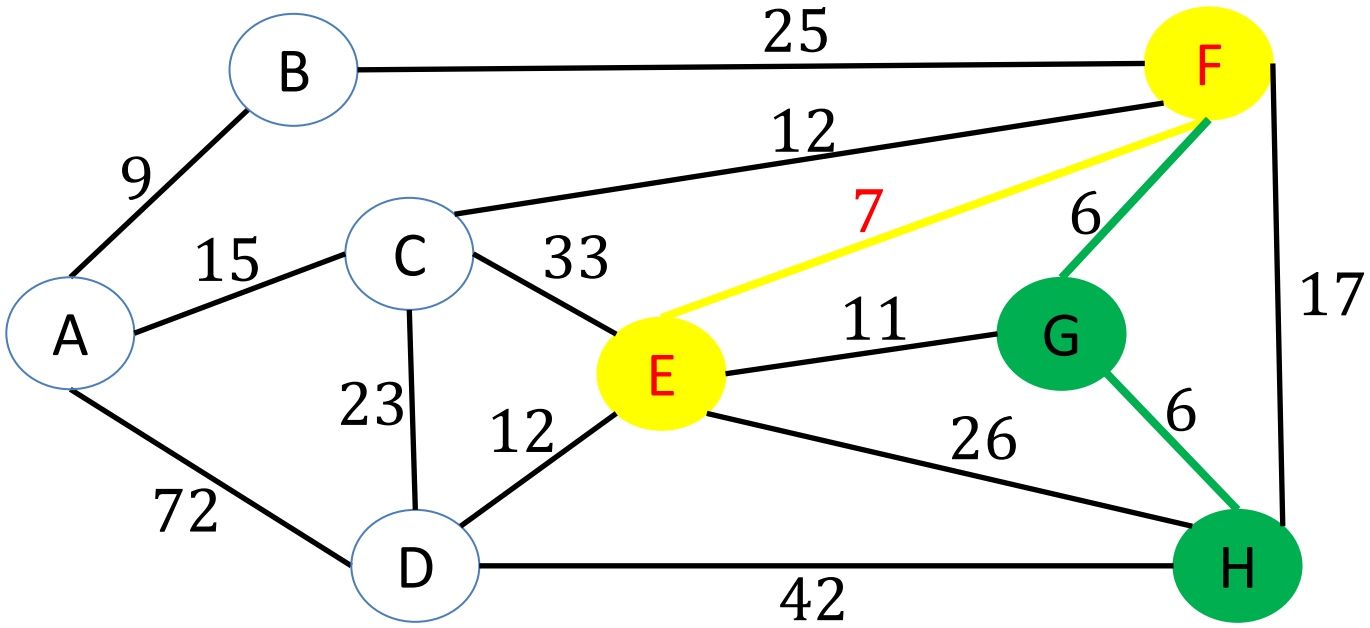


Example (6/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F)\}, \text{cost}(T) = 19$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

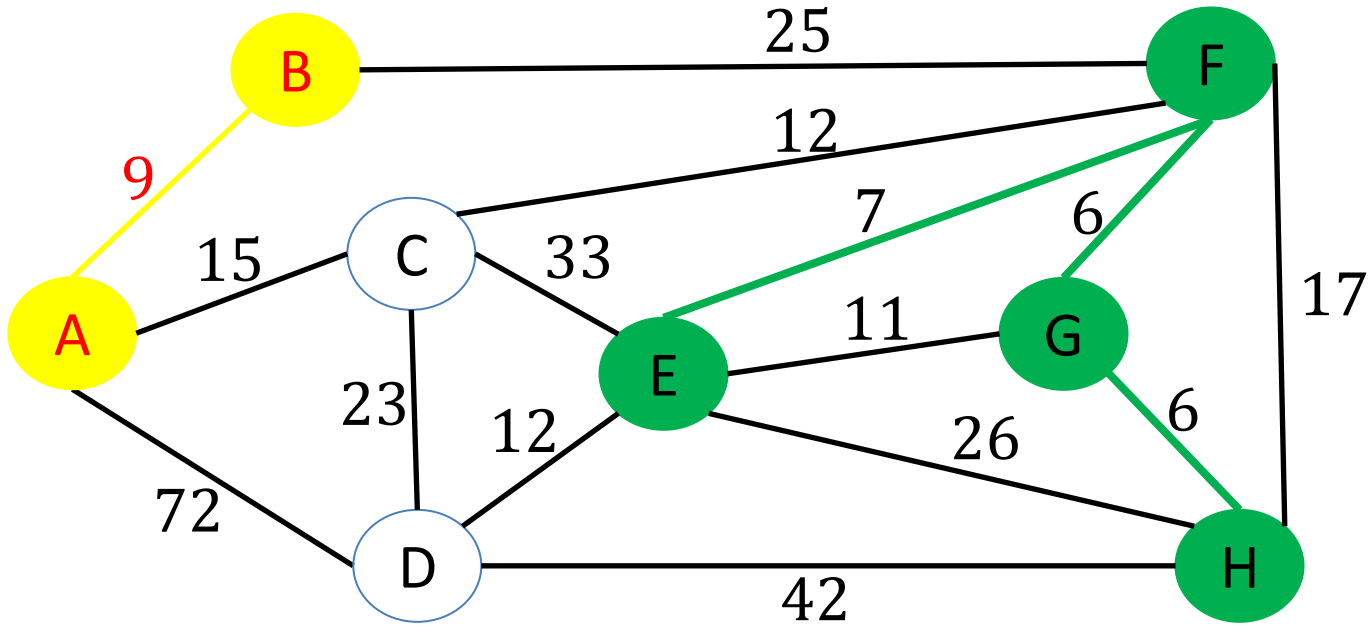


Example (7/12)

Sorted Weight	6	6	7	9	11	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	(E, G)	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B)\}, \text{cost}(T) = 28$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

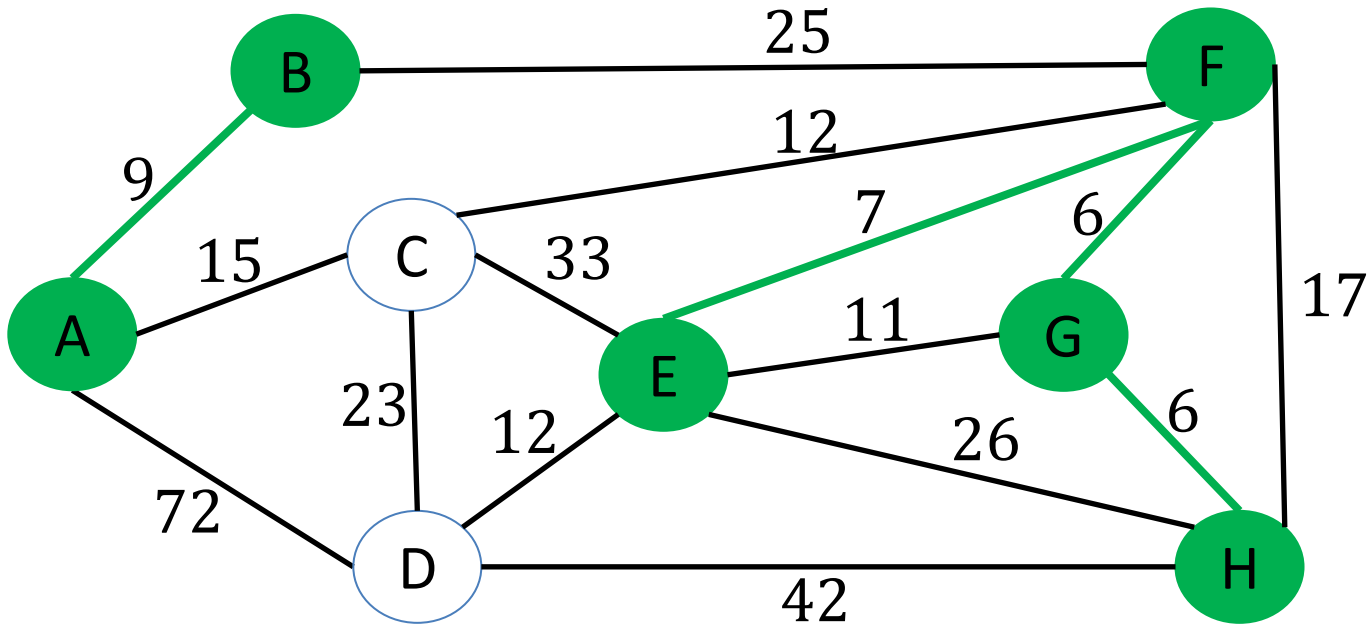


Example (8/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B)\}, \text{cost}(T) = 28$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

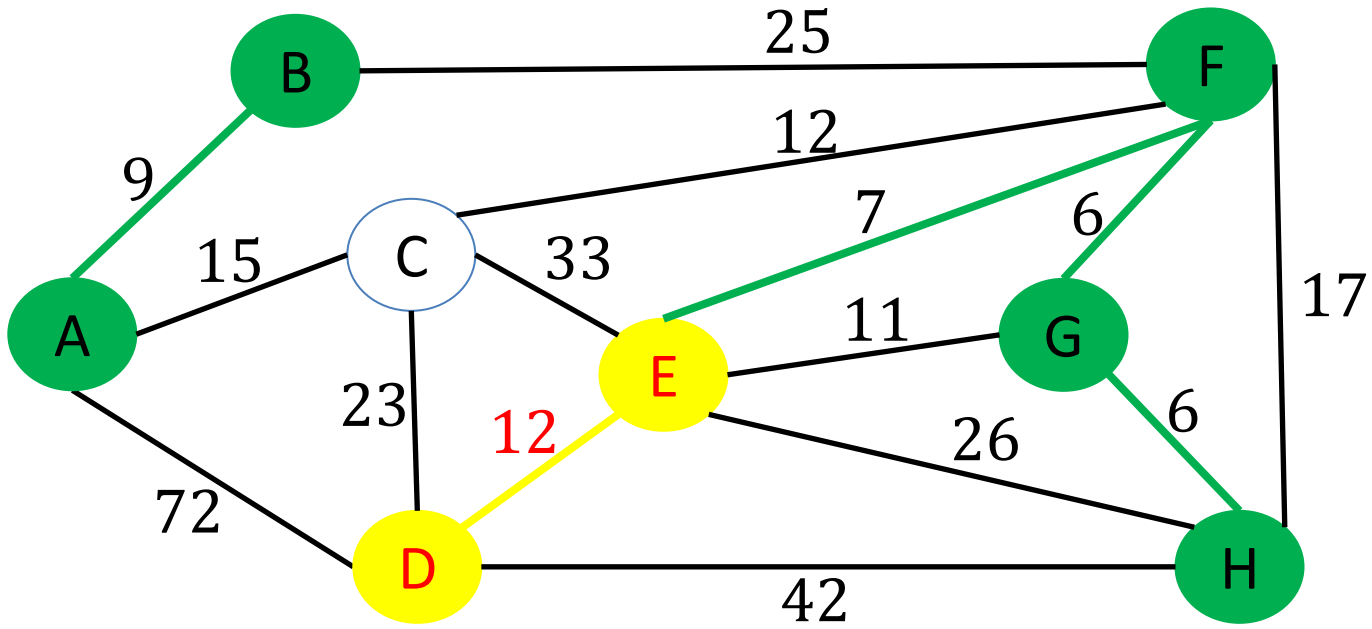


Example (9/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B), (D, E)\}, \text{cost}(T) = 40$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

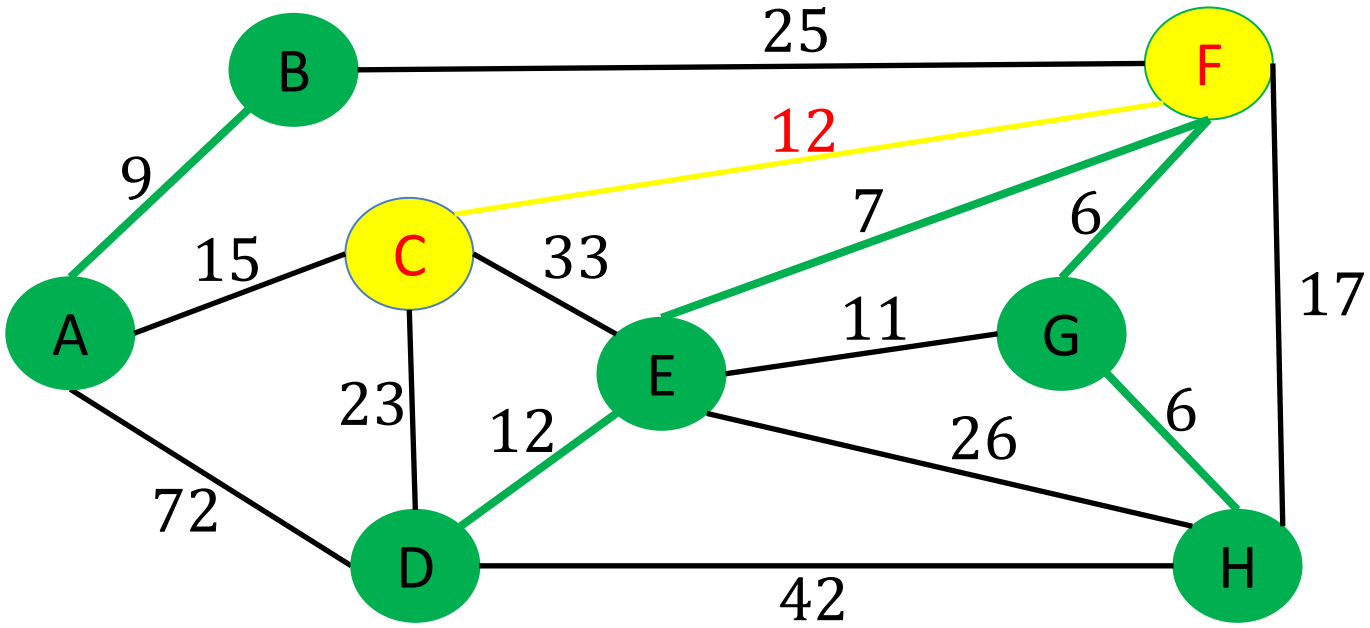


Example (10/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F,G),(G,H),(E,F),(A,B),(D,E),(C,F)\}, \text{cost}(T) = 52$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

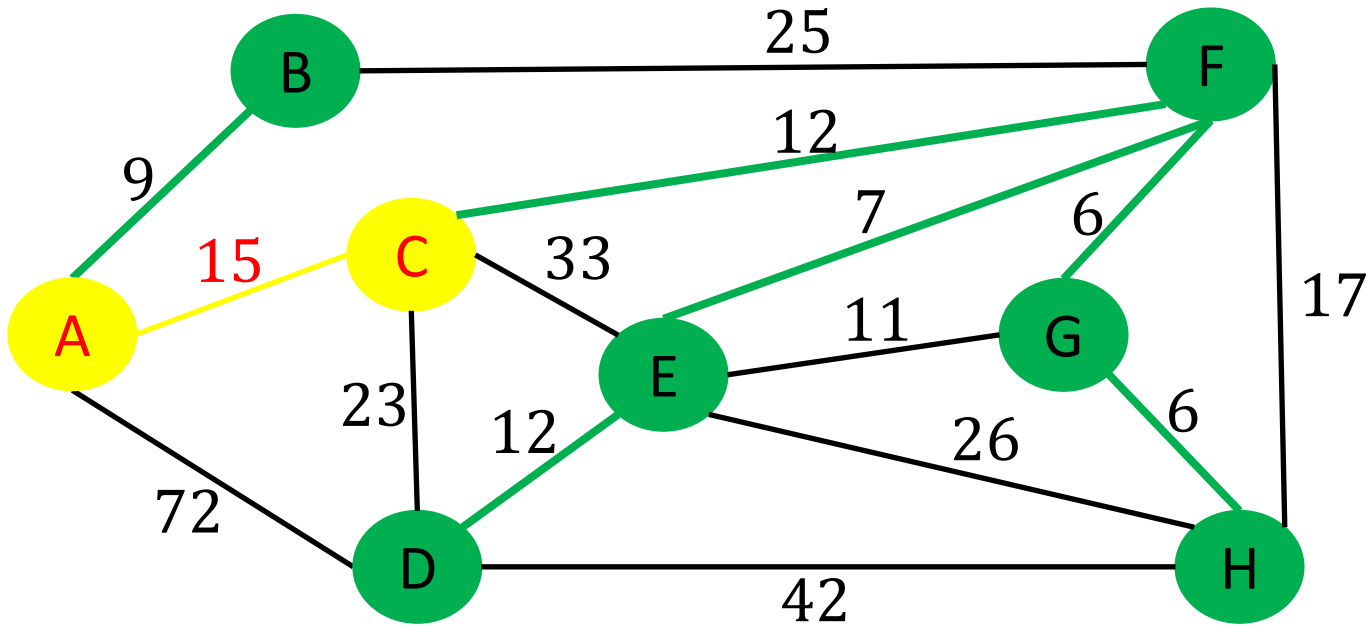


Example (11/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F,G),(G,H),(E,F),(A,B),(D,E),(C,F),(A,C)\}, \text{cost}(T) = 67$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.

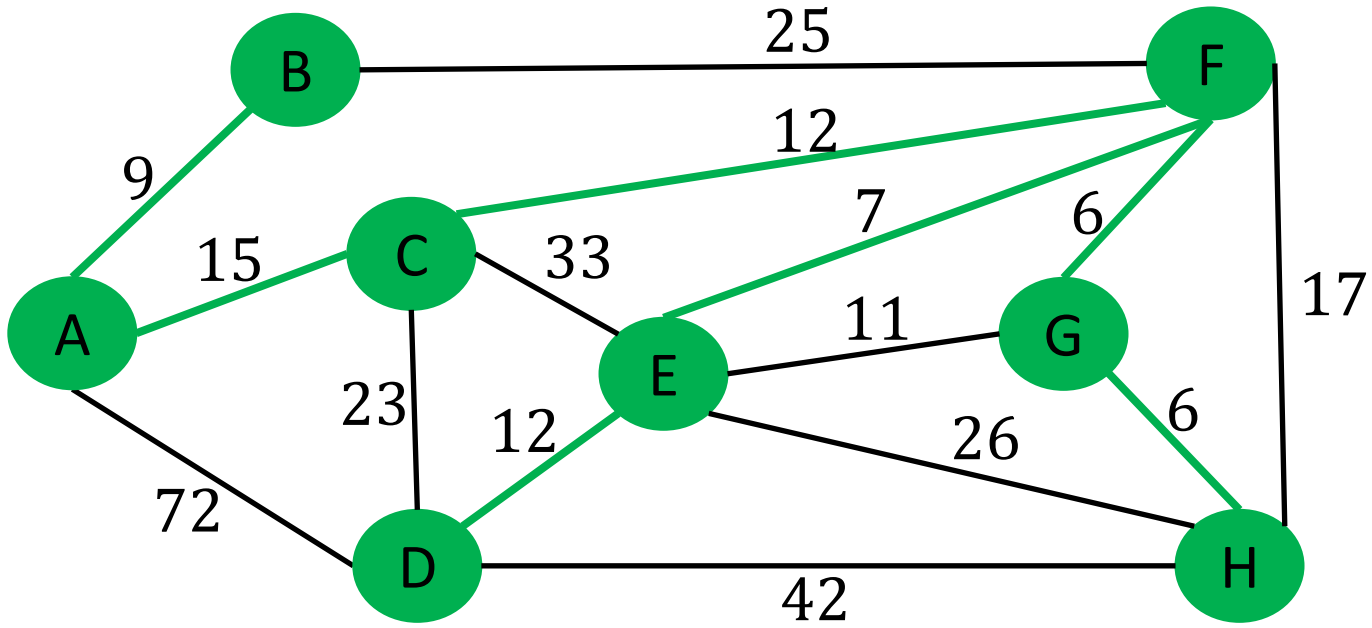


Example (12/12)

Sorted Weight	6	6	7	9	~~11~~	12	12	15	17	23	25	26	33	42	72
Sorted Edge	(F, G)	(G, H)	(E, F)	(A, B)	~~(E, G)~~	(D, E)	(C, F)	(A, C)	(F, H)	(C, D)	(B, F)	(E, H)	(C, E)	(D, H)	(A, D)

$$T = \{(F, G), (G, H), (E, F), (A, B), (D, E), (C, F), (A, C)\}, \text{cost}(T) = 67$$

1. Sort all edges in the graph in non-decreasing order of their weights.
2. Initialize MST T as an empty set.
3. For each edge in the sorted list:
4. If the edge doesn't form a cycle in the MST, add it to the MST.
5. Stop when the MST contains exactly $(|V| - 1)$ edges.



Q & A